



University of
Sheffield

A Reproducible Benchmarking Framework for Evaluating TLS and mTLS Performance Overhead in Zero Trust-Aligned Apache Kafka Deployments

Nick Efe Oni

Supervisor: Mahshid Delavar

*A report submitted in fulfilment of the requirements
for the degree of MEng in Software Engineering*

in the

School of Computer Science

May 5, 2026

Declaration

All sentences or passages quoted in this report from other people's work have been specifically acknowledged by clear cross-referencing to author, work and page(s). Any illustrations that are not the work of the author of this report have been used with the explicit permission of the originator and are specifically acknowledged. I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this project and the degree examination as a whole.

Name: Nick Efe Oni

Signature: Nick Efe Oni

Date: May 5, 2026

Acknowledgements

I would like to thank my mother, Patricia Ojo, for her love, sacrifice, and constant support throughout my life and my university journey. I am also grateful to my sister, Eve Eki Oni, for her encouragement, and to my late father, Frederick Oni, whose memory continues to motivate me.

I would also like to thank my flatmates, Elijah Ogundadegbe, Ojumirireoluwa Onifade, and Michael Iwuji, for their friendship, support, and the moments of calm they provided during a demanding third-year dissertation period. I am grateful to my close friends Olivia Isama, Tanay Patel, Leroy Barnie, and Edesiri Ohwomado for standing by me, encouraging me, and helping me keep perspective when the work became difficult.

I would like to thank the best supervisor in town, Mahshid Delavar for her time, support and patience, my colleagues at Travelodge for their support and understanding, and Andrew Charlesworth, one of the most supportive lecturers I have had, for his guidance and encouragement. I am also grateful to Bali Posner Tal, my programme coordinator at UpReach, for the opportunities, advice, and support provided during my development.

Finally, I am thankful for the experiences I have had at university. They gave me chances to learn, grow, and decompress from personal pressures and wider challenges. The hackathons, exclusive events, and development opportunities I attended through organisations such as KPMG, Man Group, Google, and BNY helped broaden my perspective and gave me experiences I will carry beyond university.

Abstract

Apache Kafka is widely used in high-throughput distributed systems, but enabling stronger transport security can affect the performance mechanisms that make Kafka efficient. This dissertation evaluates the overhead introduced by TLS and mutual TLS (mTLS) as Zero Trust-aligned transport-security mechanisms in Apache Kafka.

A reproducible benchmarking artefact was designed and implemented using Terraform, AWS EC2, Kafka KRaft, configuration-driven orchestration, checkpointed execution, structured result processing, and host-level telemetry collection. The artefact deployed Kafka in plaintext, TLS, and mTLS modes and executed matched workloads so that security-mode overhead could be separated from workload variation. The final evaluation included 5,184 successful producer benchmark rows and 72 successful consumer validation rows.

The results show that Kafka transport-security overhead is measurable but not fixed. In the matched producer campaign, TLS reduced mean throughput by 10.83% and mTLS by 9.57% relative to plaintext. Latency was affected more sharply, with mean average-latency penalties exceeding 950% for both secure modes. mTLS was not consistently worse than TLS; its relative cost depended on the metric and workload condition being examined. Producer concurrency, target throughput, and batch size were the strongest overhead amplifiers, while larger records did not produce a simple proportional increase in throughput penalty.

Host telemetry showed that the overhead should not be interpreted as a simple broker CPU-saturation effect. Instead, secure transport changed the interaction between producer pacing, broker work, network transfer, and achieved throughput. The dissertation contributes both Kafka-specific empirical evidence and a reproducible framework for evaluating transport-security overhead under matched high-throughput conditions.

Contents

1	Introduction	1
1.1	Background and Motivation	1
1.2	Problem Statement and Research Gap	2
1.3	Research Aim, Question, and Objectives	2
1.4	Contribution and Scope	3
1.5	Artefact Overview	4
1.6	Dissertation Structure	4
2	Literature Review	5
2.1	Apache Kafka & Distributed Streaming Systems	5
2.2	Performance in Kafka Systems	7
2.3	Security Mechanisms in Kafka	8
2.4	Zero Trust Architecture	10
2.5	Zero Trust in Distributed Streaming Systems	11
2.6	TLS/mTLS Performance Overhead	12
2.7	Benchmarking Methodologies	13
2.8	Synthesis & Research Gap	14
3	Design, Implementation, and Testing	16
3.1	Artefact Requirements	16
3.2	System Architecture	17
3.3	Reproducing the Artefact’s Environment	20
3.3.1	Infrastructure provisioning	20
3.3.2	Benchmark-client preparation	20
3.3.3	Kafka deployment	20
3.3.4	Benchmark-plan generation	20
3.3.5	Campaign execution	20
3.3.6	Result processing	21
3.4	Infrastructure Provisioning	23
3.5	Kafka Cluster Deployment	23
3.6	Security Mode Design	24
3.7	Benchmark Workload Design	25

3.8	Factorial Experiment Design	26
3.9	Execution and Orchestration	26
3.10	Telemetry and Metric Collection	28
3.11	Result Processing and Export	29
3.12	Fault Tolerance and Idempotency	32
3.13	Testing and Validation	32
3.14	Chapter Summary	34
4	Results and Discussion	36
4.1	Chapter Overview	36
4.2	Result-Set Completion and Validity	36
4.3	Comparison Method	37
4.4	Producer Throughput Results	39
4.5	Producer Latency Results	42
4.6	Host Telemetry Results	46
4.7	Consumer Validation Results	50
4.8	Discussion	53
4.9	Chapter Summary	54
5	Evaluation, Conclusions, and Further Work	56
5.1	Chapter Overview	56
5.2	Evaluation Against Objectives	56
5.3	Summary of Findings	58
5.4	Methodological Contribution	58
5.5	Limitations	59
5.6	Further Work	60
5.7	Answer to the Research Question	60
5.8	Chapter Summary	61
	Appendices	64
A	Ethics and GenAI Declaration	65
A.1	Ethics Statement	65
A.2	Generative AI Declaration	65
A.2.1	Acknowledge	65
A.2.2	Describe	66
A.2.3	Evidence	66
A.2.4	Declaration	67
B	Artefact Configuration	68
B.1	Configuration Files	68
B.2	Producer Factorial Campaign	69
B.3	Security Mode Configuration	70

B.4	Validity Constraints	71
B.5	Consumer Validation Slice	71
C	Result Schema	73
C.1	Run-Level Output	73
C.2	Metadata Fields	74
C.3	Parsed Result Fields	75
C.4	Host Telemetry Fields	75
C.5	Result-Set Output	76
C.6	Matched and Statistical Analysis Outputs	77
C.7	Metric Interpretation Caveat	77
D	Additional Result Tables and Generated Artefacts	79
D.1	Campaign Completion and Matched Retention	79
D.2	Matched Producer Performance Tables	80
D.3	Producer Workload Interaction Tables	81
D.4	Host Telemetry Tables	82
D.5	Consumer Validation Tables	83
D.6	Highest Observed Producer Overhead Cases	83
D.7	Selected Supporting Figures	84
E	Repository and Reproducibility Notes	98
E.1	Repository Structure	98
E.2	Configuration-Driven Execution	99
E.3	Deployment Reproducibility	100
E.4	Checkpointing and Resumability	100
E.5	Result Traceability	101
E.6	Generated State and Version Control	101
E.7	Analysis Export Reproducibility	101
E.8	Rerunning the Artefact	102
E.9	Reproducibility Limitations	102

List of Figures

2.1	Simplified Apache Kafka architecture showing producer applications publishing events to a cluster of brokers, and consumer applications pulling events from the cluster. Figure generated using ChatGPT image generation from a prompt requesting a producer-broker-consumer Apache Kafka architecture diagram, April 2026.	6
2.2	Comparison of Kafka data movement under plaintext, TLS, and mTLS. Plaintext can use page-cache-backed <code>sendfile</code> zero-copy transfer, while TLS and mTLS require user-space cryptographic processing that prevents this path.	8
2.3	Kafka security data path showing how TLS/mTLS, certificate-based identity verification, principal mapping, and ACL authorisation are applied before Kafka request processing, with broker-to-broker communication also secured where configured.	10
2.4	An illustration of the conceptual Zero Trust control model for a distributed streaming pipeline.	12
3.1	System architecture of the benchmarking artefact, showing local orchestration, Terraform-provisioned AWS infrastructure, private Kafka traffic, SSH control paths, broker instances, benchmark client, and result collection.	19
3.2	Tutorial-style reproduction workflow for the benchmarking artefact, showing how infrastructure provisioning, deployment, execution, checkpointing, and result processing are connected.	22
3.3	Benchmark execution pipeline from JSON configuration to dissertation-ready result exports.	28
3.4	Result directory and data-flow schema showing how raw per-run logs, metadata, telemetry, campaign state files, parsed results, aggregated summaries, and dissertation-ready exports are connected.	31
4.1	Matched producer throughput penalty for TLS and mTLS relative to plaintext, measured in records/s. Positive values mean lower throughput than the matched plaintext run; negative values mean higher throughput. The orange line is the median, the box is the middle 50% of runs, whiskers show the non-outlier range, and circles are outliers.	40

4.2	Mean matched producer throughput penalty by record size, measured in records/s relative to plaintext. Positive values mean TLS or mTLS achieved lower throughput than the matched plaintext run. Penalty is highest at 1 KB and 10 KB records, then falls at 100 KB records, suggesting that throughput overhead is not simply proportional to payload size.	41
4.3	Matched producer throughput penalty by producer count, measured in records/s relative to plaintext. Each point compares TLS or mTLS with the equivalent plaintext run. Positive values mean lower throughput than plaintext; negative values mean higher throughput. Penalties spread wider and reach higher values as producer count increases.	42
4.4	Matched producer average-latency penalty for TLS and mTLS relative to plaintext. Positive values mean higher latency than the matched plaintext run. The boxplot shows that secure transport produces substantially larger latency penalties than the throughput penalties reported earlier.	44
4.5	Mean matched producer average-latency penalty by record size, relative to plaintext. Positive values mean TLS or mTLS had higher average latency than the matched plaintext run. Latency penalty peaks at 10 KB records, then falls for 100 KB records.	45
4.6	Mean matched producer average-latency penalty by producer count, relative to plaintext. Positive values mean higher latency than the matched plaintext run. Penalty rises sharply from 1 to 6 producers, then increases more slowly from 6 to 12 producers.	46
4.7	Matched CPU utilisation by security mode for producer runs. The orange line is the median, the box shows the middle 50% of runs, whiskers show the non-outlier range, and points are outliers. Client CPU remains high across modes, while broker CPU is generally lower for TLS and mTLS than plaintext.	48
4.8	Correlation heatmap across producer metrics and host telemetry indicators. Red shows positive correlation, blue shows negative correlation, and stronger colour indicates a stronger relationship. The heatmap shows that throughput, latency, CPU, network, and disk metrics form distinct clusters rather than varying independently.	49
4.9	Matched consumer validation overhead relative to plaintext. Positive throughput penalty means lower records/s than plaintext; positive fetch-time penalty means slower consumption. Both TLS and mTLS show read-path overhead in the targeted consumer slice.	52
D.1	Additional producer throughput-overhead distribution. Positive values mean lower throughput than the matched plaintext run; negative values mean higher throughput.	84
D.2	Additional producer throughput-overhead view by record size, showing how the mean penalty changes across payload sizes.	85

D.3	Additional producer throughput-overhead scatter plot by producer count. This supports the workload-interaction result that throughput penalty spreads more widely as producer concurrency increases.	85
D.4	Additional producer average-latency overhead view by record size, showing that latency overhead is not uniform across payload sizes.	86
D.5	Additional producer average-latency overhead view by producer count. This supports the latency result that secure-transport overhead becomes more visible under higher producer concurrency.	86
D.6	Additional benchmark-client CPU distribution by security mode for producer runs.	87
D.7	Additional broker CPU distribution by security mode for producer runs. . . .	87
D.8	Additional broker CPU heatmap by record size and security mode for producer runs. This supports the host telemetry finding that broker CPU does not simply increase under TLS and mTLS.	88
D.9	Additional consumer throughput-overhead distribution for the targeted validation slice. Positive values mean lower throughput than the matched plaintext run.	89
D.10	Additional consumer fetch-time overhead distribution for the targeted validation slice.	89
D.11	Additional benchmark-client network transmit overhead distribution for producer runs. Positive values mean higher transmit volume than the matched plaintext run.	90
D.12	Additional broker network receive heatmap by record size and security mode for producer runs. This supports the comparison of network behaviour across matched security modes.	91
D.13	Additional benchmark-client disk-write overhead distribution for producer runs. Positive values mean more write-sector activity than the matched plaintext run.	91
D.14	Additional broker disk-write overhead distribution for producer runs. This supports the host telemetry comparison without treating disk activity as the main source of transport-security overhead.	92

List of Tables

3.1	Main Functional Requirements	16
3.2	Main Non-Functional Requirements	17
3.3	Kafka transport-security modes evaluated by the artefact	25
3.4	Testing and validation evidence for the benchmarking artefact	34
4.1	Final benchmark completion and matched-analysis counts. Started counts can exceed planned counts because interrupted or failed attempts were retried during checkpointed execution. The failure counts, therefore, record failed attempts, not final invalid planned rows.	37
4.2	Matched producer throughput summary in records/s. Penalty values compare TLS and mTLS against equivalent plaintext runs under the same workload, deployment, and trial configuration. Positive penalty means lower throughput than plaintext; negative penalty means higher throughput than plaintext. . .	41
4.3	Matched producer latency summary	44
4.4	Bootstrap and matched-pair summary for key producer penalties. Confidence intervals are 95% bootstrap intervals for mean penalty. Worse-row percentages show the proportion of matched rows where the secure mode was worse than plaintext for the same metric.	45
4.5	Matched producer host-telemetry summary. Values are means across matched producer rows; broker values use the per-run broker mean, and network counters are converted from bytes using decimal MB/GB.	47
4.6	Targeted consumer validation summary. Penalty values compare TLS and mTLS against equivalent plaintext consumer runs. Positive penalty means lower throughput or higher fetch time than plaintext; negative CPU penalty means lower CPU utilisation than plaintext.	51
5.1	Evaluation of dissertation objectives	57
A.1	Representative examples of GenAI use during the project	67
B.1	Main artefact configuration and generated plan files	69
B.2	Producer factorial campaign dimensions	70
B.3	Security modes used by the artefact	71

C.1	Run-level benchmark output files	74
C.2	Metadata field groups	74
C.3	Primary parsed result metrics	75
C.4	Host telemetry field groups	76
C.5	Result-set output files	76
C.6	Matched comparison and statistical analysis outputs	77
D.1	Producer campaign completion and matched-analysis counts	79
D.2	Consumer validation completion and matched-analysis counts	80
D.3	Matched producer absolute performance and CPU summary	80
D.4	Matched producer relative overhead summary against plaintext	80
D.5	Matched producer overhead range across the factorial campaign	81
D.6	Matched producer throughput penalty by record size	81
D.7	Matched producer throughput penalty by producer count	81
D.8	Matched producer throughput penalty by compression mode	81
D.9	Matched producer throughput penalty by acknowledgement mode	82
D.10	Matched producer average-latency penalty by record size	82
D.11	Matched producer average-latency penalty by producer count	82
D.12	Matched producer average-latency penalty by batch size	82
D.13	Matched producer average-latency penalty by broker count	82
D.14	Matched producer host-telemetry summary	83
D.15	Targeted consumer validation summary	83
D.16	Highest observed producer overhead cases by metric	84
D.17	Bootstrap confidence intervals for key producer penalties	93
D.18	Matched-pair effect summary for key producer penalties	94
D.19	Bootstrap confidence intervals for key consumer penalties	94
D.20	Matched-pair effect summary for key consumer penalties	95
D.21	Top factor-sensitivity results for selected producer and consumer penalties . .	96
E.1	Repository structure	99

Chapter 1

Introduction

1.1 Background and Motivation

Modern distributed systems increasingly rely on event-streaming platforms to connect cloud services, data pipelines, and enterprise applications [1, 2]. Apache Kafka is widely used for this role because it combines high throughput, low latency, fault tolerance, and horizontal scalability [3–5]. These properties make Kafka a critical component in data-driven systems where large volumes of events must move reliably between producers, brokers, and consumers.

Kafka’s performance is not accidental. It depends on partition-level parallelism, batching, append-only logs, page-cache utilisation, and zero-copy data transfer [3, 6, 7]. These mechanisms reduce unnecessary coordination and data movement, but they also make Kafka sensitive to additional processing in the data path. Transport security is therefore not only a security setting. It is also a performance variable because encryption and authentication can change the communication and computation paths on which Kafka relies.

At the same time, cloud and multi-service systems can no longer assume that internal network traffic is trustworthy. Insider threats, compromised services, and lateral movement attacks can originate within the network boundary [8–10]. This has increased interest in Zero Trust Architecture, commonly summarised as “never trust, always verify” [8–11]. Zero Trust deployments usually emphasise strong identity, mutual authentication, least-privilege access, encrypted communication, monitoring, and policy-based control [9, 11, 12].

Zero Trust Architecture is not a single Kafka feature. However, Kafka supports several mechanisms that align with Zero Trust principles, including transport encryption, certificate-based authentication, broker-to-broker security, SASL authentication, and ACL-based authorisation [7, 13–15]. TLS and mutual TLS (mTLS) are particularly important because they directly affect Kafka’s network data path and allow plaintext, server-authenticated encrypted transport, and mutually authenticated encrypted transport to be compared.

1.2 Problem Statement and Research Gap

Existing work provides useful foundations for Kafka performance and Zero Trust security. Kafka has been analysed as a high-throughput event-streaming platform [4, 5, 16], while Zero Trust has been discussed in distributed and streaming environments [8, 12, 17]. However, these areas are often treated separately. There is still limited systematic empirical evidence showing how Zero Trust-aligned transport security affects Kafka performance when workload and deployment variables are controlled.

This gap matters because TLS and mTLS add cryptographic work to paths that Kafka is designed to keep efficient. Kafka’s official design documentation states that SSL/TLS prevents use of the `sendfile` zero-copy optimisation because SSL libraries operate in user space [6]. Wider TLS literature also shows that secure transport can introduce computational and latency overhead, but that the scale depends on workload, connection behaviour, and system configuration [18–20]. The overhead of TLS and mTLS in Kafka, therefore, cannot be treated as a single fixed penalty.

The unresolved problem is methodological as well as empirical. A comparison between unrelated plaintext and secured Kafka runs can confuse security overhead with workload variation. Broker count, replication factor, acknowledgements, batching, compression, producer concurrency, message size, and target throughput can all affect Kafka performance. A defensible evaluation must therefore compare plaintext, TLS, and mTLS only where these parameters are matched. Without this control, it is difficult to determine whether observed differences are caused by transport security itself or by the surrounding workload configuration.

More specifically, the literature reviewed in this dissertation does not provide a matched factorial evaluation of plaintext, TLS, and mTLS Kafka deployments across producer concurrency, message size, batching, compression, acknowledgement mode, replication configuration, and broker count, while also collecting both application-level metrics and host-level telemetry. Existing Kafka studies explain Kafka’s performance mechanisms and benchmark behaviour [4, 5, 16], Zero Trust studies motivate stronger identity and transport controls [8, 9, 12], and TLS studies show that encrypted transport can impose workload-dependent overhead [18–20]. However, these strands do not yet provide a reproducible Kafka-specific comparison of plaintext, TLS, and mTLS under matched high-throughput workload and deployment conditions.

1.3 Research Aim, Question, and Objectives

This dissertation aims to design, implement, and evaluate a reproducible benchmarking framework for quantifying the performance overhead introduced by TLS and mTLS in

high-throughput Apache Kafka deployments.

The research question is:

“How do TLS and mutual TLS affect Apache Kafka throughput, latency, and host-resource behaviour under matched high-throughput workload and deployment conditions?”

The objectives are to:

1. Analyse Kafka’s performance mechanisms and identify how transport security may affect them.
2. Implement a cloud-based Kafka benchmarking artefact supporting plaintext, TLS, and mTLS.
3. Provision reproducible Kafka deployments using infrastructure-as-code and automated orchestration.
4. Execute matched benchmark workloads across security modes using explicit configuration files.
5. Measure throughput, latency, and host-level telemetry for each benchmark run.
6. Evaluate how TLS and mTLS overhead varies across Kafka workload and deployment parameters.
7. Include targeted consumer-side validation to assess whether secure transport also affects Kafka’s read path.

1.4 Contribution and Scope

This dissertation makes three main contributions. First, it provides a Kafka-specific empirical evaluation of plaintext, TLS, and mTLS using matched workload and deployment configurations. Second, it contributes a reproducible cloud-based benchmarking artefact built around Terraform, AWS EC2, Kafka KRaft deployments, configuration-driven orchestration, checkpointed execution, and structured result processing. Third, it combines application-level metrics with host-level telemetry, allowing throughput and latency changes to be interpreted alongside CPU, memory, network, and disk behaviour.

The scope is deliberately precise. This dissertation does not evaluate Zero Trust Architecture as a complete enterprise security model. It evaluates TLS and mTLS as Zero Trust-aligned transport-security mechanisms that provide encryption, server authentication, and mutual certificate-based authentication in Kafka deployments. This narrower scope makes the evaluation measurable, reproducible and directly comparable against a plaintext baseline.

1.5 Artefact Overview

The artefact provisions Kafka clusters on AWS EC2 using Terraform, deploys Kafka in KRaft mode, and supports plaintext, TLS, and mTLS transport-security configurations. Benchmark execution is driven by structured configuration files so that equivalent workloads can be repeated across security modes and compared under matched conditions.

The main evaluation uses a producer-side factorial campaign varying broker count, replication factor, minimum in-sync replicas, message size, target throughput, producer count, acknowledgement mode, batch size, and compression type. A smaller targeted consumer-side validation study is also included to test whether secure transport affects Kafka’s read path under selected workloads.

For each run, the artefact records throughput and latency alongside host telemetry from the benchmark client and Kafka brokers. Raw logs are parsed into structured JSON and CSV outputs, allowing reproducible comparison of plaintext, TLS, and mTLS results.

1.6 Dissertation Structure

Chapter 2 reviews the literature on Kafka performance, Kafka security, Zero Trust Architecture, TLS/mTLS overhead, and benchmarking methodology. Chapter 3 describes the artefact design, implementation, and testing. Chapter 4 presents and discusses the benchmark results. Chapter 5 evaluates the objectives, summarises the findings, discusses limitations, and identifies further work.

Chapter 2

Literature Review

This chapter reviews the literature needed to evaluate the performance impact of Zero Trust-aligned transport security in Apache Kafka. The review does not treat Kafka performance and Zero Trust security as separate topics. Instead, it focuses on the point where they interact: Kafka’s high-throughput data path, and the effect of TLS and mutual TLS (mTLS) when cryptographic processing is added to that path.

The chapter first reviews Kafka as a distributed event-streaming system, then examines the mechanisms that make Kafka efficient, including append-only logs, batching, partitioning, page-cache use, and zero-copy transfer. It then analyses Kafka security mechanisms, Zero Trust Architecture, TLS/mTLS overhead, and benchmarking methodology. The purpose is to establish the research gap addressed by this dissertation: the lack of a reproducible, Kafka-specific, matched evaluation of plaintext, TLS, and mTLS under high-throughput workload and deployment conditions.

2.1 Apache Kafka & Distributed Streaming Systems

Distributed streaming systems are designed to store, process, and move continuous streams of data between distributed applications [3, 4, 6]. Apache Kafka is widely used for this purpose because it provides high-throughput event delivery, fault tolerance, high availability, and horizontal scalability [3, 4, 16]. These properties make it suitable for cloud and enterprise systems where many services exchange data reliably and at scale.

Kafka is built around brokers, producers, consumers, topics, and partitions [3, 4]. Producers publish events to topics, consumers read those events from brokers, and topics are divided into partitions that can be distributed across multiple brokers [3, 6]. This allows Kafka to scale horizontally because different brokers and clients can process different partitions in parallel. Replication improves availability and durability by storing copies of partition data across brokers [3, 6]. A simplified version of this architecture is shown in Figure 2.1.

Kafka also uses a pull-based consumption model. Consumers fetch records at their own rate rather than receiving records pushed by brokers [6]. This decouples producers from consumers and supports batching, but it also means that performance depends heavily on client behaviour, partitioning, broker configuration, and workload shape. Kafka throughput and latency are therefore affected by acknowledgement settings, replication, batching, compression, partitions, producer concurrency, and client configuration [6, 7, 13, 14, 16].

This matters because Kafka performance is not a fixed platform property. It is a configurable outcome produced by deployment choices, workload characteristics, and coordination costs [6, 7, 13, 14, 16]. Security mechanisms such as TLS and mTLS must therefore be evaluated inside Kafka’s architecture rather than treated as generic network overhead. They affect the same distributed data path that Kafka relies on for high-throughput event movement [7, 13, 14, 18].

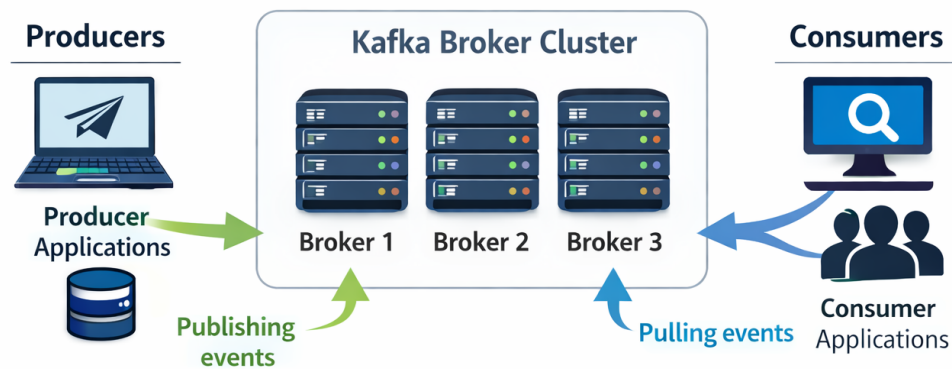


Figure 2.1: Simplified Apache Kafka architecture showing producer applications publishing events to a cluster of brokers, and consumer applications pulling events from the cluster. Figure generated using ChatGPT image generation from a prompt requesting a producer-broker-consumer Apache Kafka architecture diagram, April 2026.

2.2 Performance in Kafka Systems

Kafka’s performance comes from architectural choices that reduce coordination and unnecessary data movement. Kafka stores records in append-only logs rather than random-access data structures, allowing efficient sequential writes [3, 6, 16]. It also uses the operating system page cache and zero-copy I/O, specifically the `sendfile` system call, to transfer data from the page cache to the network socket with minimal copying [6]. These optimisations are central to Kafka’s throughput, but they also make Kafka sensitive to mechanisms that move processing back into user space, including TLS and mTLS [6, 13, 18, 19]. This contrast is shown in Figure 2.2.

Partitioning, batching, compression, and pull-based consumption further shape Kafka performance [3, 4, 6, 16]. Partitioning allows parallelism across brokers and clients. Batching reduces network round trips and improves throughput [6, 7, 16]. Compression can reduce network and storage pressure, but shifts computational work to producers and consumers [6, 7, 14]. This is directly relevant to TLS and mTLS because compression and encryption can compete for CPU resources while also changing the volume of data transferred.

Kafka configuration also creates throughput-latency-durability trade-offs. Stronger acknowledgement settings and higher replication improve durability, but can increase latency compared with weaker acknowledgement settings [6, 7, 16]. Batch size, producer concurrency, message size, broker count, replication factor, and minimum in-sync replicas can all change the balance between throughput, delay, and reliability [6, 7, 13, 14, 16]. This means a security-overhead result from one Kafka configuration cannot be assumed to generalise to another.

This relationship can be framed using a simple queueing model. In publish-subscribe systems, average latency (L) can be expressed in terms of processing time (τ) and system utilisation (U) as:

$$L = \frac{\tau}{1 - U} \quad (2.1)$$

[4, 5, 19]. As processing time increases, utilisation moves closer to saturation and latency can grow non-linearly [4, 5, 16, 19]. TLS and mTLS can increase processing time by adding encryption, decryption, and certificate-based authentication to Kafka communication paths [6, 13, 18, 20].

The performance literature therefore establishes two requirements for this dissertation. First, Kafka security overhead must be measured under controlled Kafka configurations rather than inferred from generic platform claims. Second, throughput and latency must be interpreted alongside the workload variables that shape utilisation, such as batching, compression, producer count, message size, replication, and acknowledgement mode.

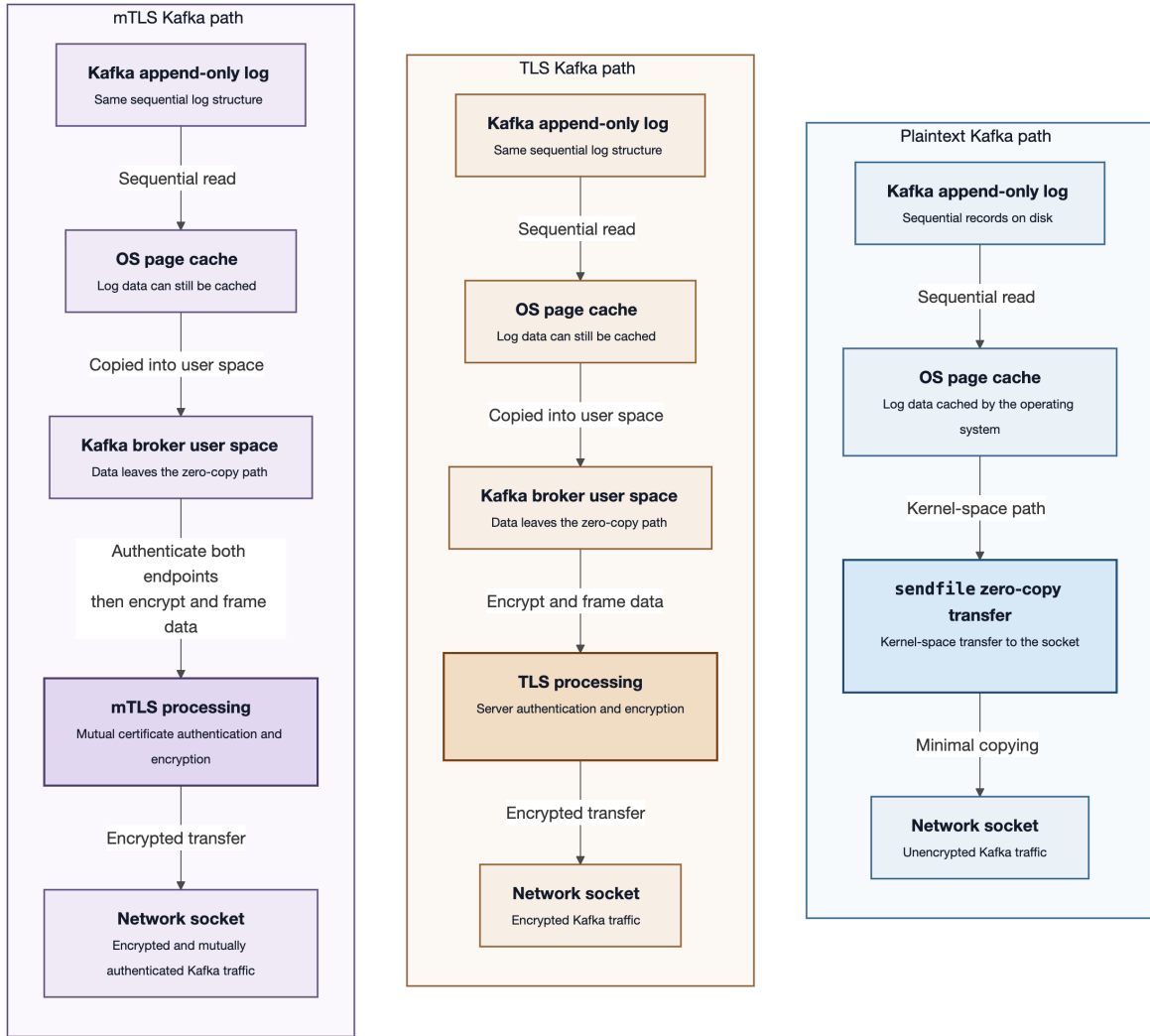


Figure 2.2: Comparison of Kafka data movement under plaintext, TLS, and mTLS. Plaintext can use page-cache-backed `sendfile` zero-copy transfer, while TLS and mTLS require user-space cryptographic processing that prevents this path.

2.3 Security Mechanisms in Kafka

Kafka provides native security mechanisms for encryption, authentication, and authorisation [3, 7, 13–15]. These mechanisms are important because Kafka is often deployed across distributed services that exchange sensitive data. However, enabling security can also change Kafka’s data path by adding cryptographic processing, identity verification, and access-control checks [6, 7, 13, 14].

Encryption in transit is provided through Transport Layer Security (TLS). TLS protects traffic between Kafka clients and brokers, and can also secure broker-to-broker

communication [7,13,14]. Kafka exposes this through listener and SSL configuration options, including `ssl.enabled.protocols` and `ssl.cipher.suites`, allowing operators to control TLS protocol and cipher behaviour [7,13,14]. This improves confidentiality and protects against eavesdropping or man-in-the-middle attacks, but it also introduces encryption and decryption work into Kafka’s communication path.

The performance implication is Kafka-specific. Kafka’s plaintext path can use `sendfile`, but Kafka’s design documentation states that this zero-copy optimisation is not used when SSL is enabled because SSL libraries operate in user space [6]. TLS therefore does not only add cryptographic cost. It can also change how data moves between disk, memory, and the network. This makes generic TLS overhead results from web servers or simple client-server systems insufficient for explaining Kafka behaviour [16,18,19].

Kafka can also support mTLS. In this mode, clients present certificates to brokers, and brokers verify those identities using Kafka SSL configuration such as `ssl.client.auth` [7,13,14]. This allows producers, consumers, and brokers to authenticate using X.509 certificates. A simplified view of this security path is shown in Figure 2.3.

mTLS is especially relevant to Zero Trust-aligned deployments because it provides encrypted communication and certificate-based identity verification. However, it may also increase cryptographic work, particularly when client-broker and broker-to-broker communication are both secured [6,13,20]. Reconnection events can also create additional handshake load, especially when many clients reconnect close together [13,18,19].

Kafka security mechanisms therefore create a direct performance-security trade-off. They improve confidentiality, authentication, and access control, but may increase CPU usage, latency, and user-space data movement. TLS and mTLS are therefore suitable mechanisms for this dissertation because they are measurable, Kafka-native, and aligned with Zero Trust principles while directly affecting the data path that enables Kafka’s throughput.

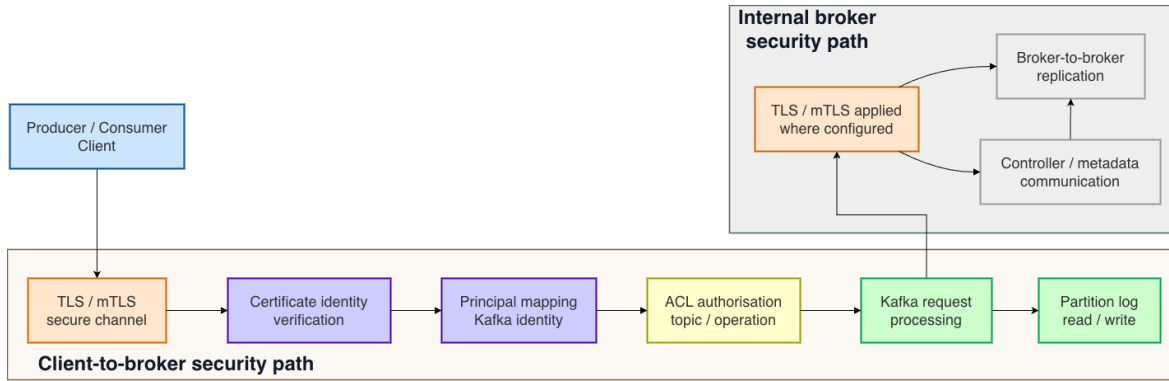


Figure 2.3: Kafka security data path showing how TLS/mTLS, certificate-based identity verification, principal mapping, and ACL authorisation are applied before Kafka request processing, with broker-to-broker communication also secured where configured.

2.4 Zero Trust Architecture

Zero Trust Architecture (ZTA) responds to the limits of perimeter-based security [8–10, 12]. Traditional models often assume that systems inside a network boundary are trustworthy. ZTA rejects this assumption and instead applies the principle of “never trust, always verify”, where access decisions depend on identity, context, policy, and service state rather than network location alone [8–11].

The core principles of ZTA include strong identity, least-privilege access, continuous verification, encrypted communication, monitoring, and fine-grained policy enforcement [8–11]. These principles are often implemented through Policy Decision Points (PDPs) and Policy Enforcement Points (PEPs), which separate policy decision from policy enforcement [8, 9, 11]. ZTA should therefore be understood as a system-level security approach rather than as a single protocol or product feature.

Transport security forms one measurable part of this wider model. TLS provides encrypted channels and server authentication, while mTLS requires both sides of a connection to authenticate using certificates [8–10, 20]. This makes mTLS relevant to Zero Trust-aligned machine-to-machine communication, where services should not be trusted purely because they operate inside the same network.

However, ZTA can introduce operational and performance costs. Identity management, certificate handling, monitoring, segmentation, and access-control design add complexity [8–12]. In performance-sensitive distributed systems, some controls may add processing or coordination overhead. This creates the central tension for this dissertation: the mechanisms that improve identity assurance and transport protection may also affect the latency and throughput of Apache Kafka [6, 18, 19].

2.5 Zero Trust in Distributed Streaming Systems

The challenges of ZTA are especially relevant to distributed streaming systems [1, 8–10, 12]. Streaming pipelines can span cloud services, remote devices, multi-cloud environments, and third-party networks. Producers, brokers, stream processors, and consumers cannot be assumed trustworthy solely because they operate within the same network boundary. They may each require authentication, authorisation, encryption, and monitoring [8–10, 12]. Figure 2.4 illustrates this control model.

This creates a direct tension. Streaming platforms are designed for continuous data movement, low latency, and high throughput, while Zero Trust-aligned controls such as mutual authentication, encryption, segmentation, and policy enforcement add work to the communication paths that carry the data [12, 18–20]. The analytical question is therefore not whether these controls are valuable, but how their cost changes under realistic streaming workloads.

Access control illustrates this tension. Role-Based Access Control is simpler to operate, but may lack the granularity needed in dynamic distributed systems [8–10]. Attribute-Based Access Control can support finer-grained decisions based on identity, device state, time, location, or environmental context [8, 9, 12], but can be harder to configure and may add latency if run-time policy evaluation sits on the critical data path [8–10, 12].

Transport security creates a similar performance question. Modern TLS can have low steady-state overhead under persistent connections and adequate hardware [15, 19]. However, streaming systems may involve large message volumes, many clients, broker-to-broker communication, compressed batches, and reconnect events. Under these conditions, TLS and mTLS overhead may depend on workload shape, certificate verification, connection behaviour, and CPU availability [12, 18–20].

For Kafka, this cannot be inferred from generic TLS studies alone. Kafka is designed around batching, partitioning, replication, and zero-copy data transfer [6, 16]. Zero Trust-aligned transport controls can move work into user space and add cryptographic processing to client-broker and broker-to-broker communication. This justifies measuring plaintext, TLS, and mTLS Kafka deployments under controlled and reproducible workloads.

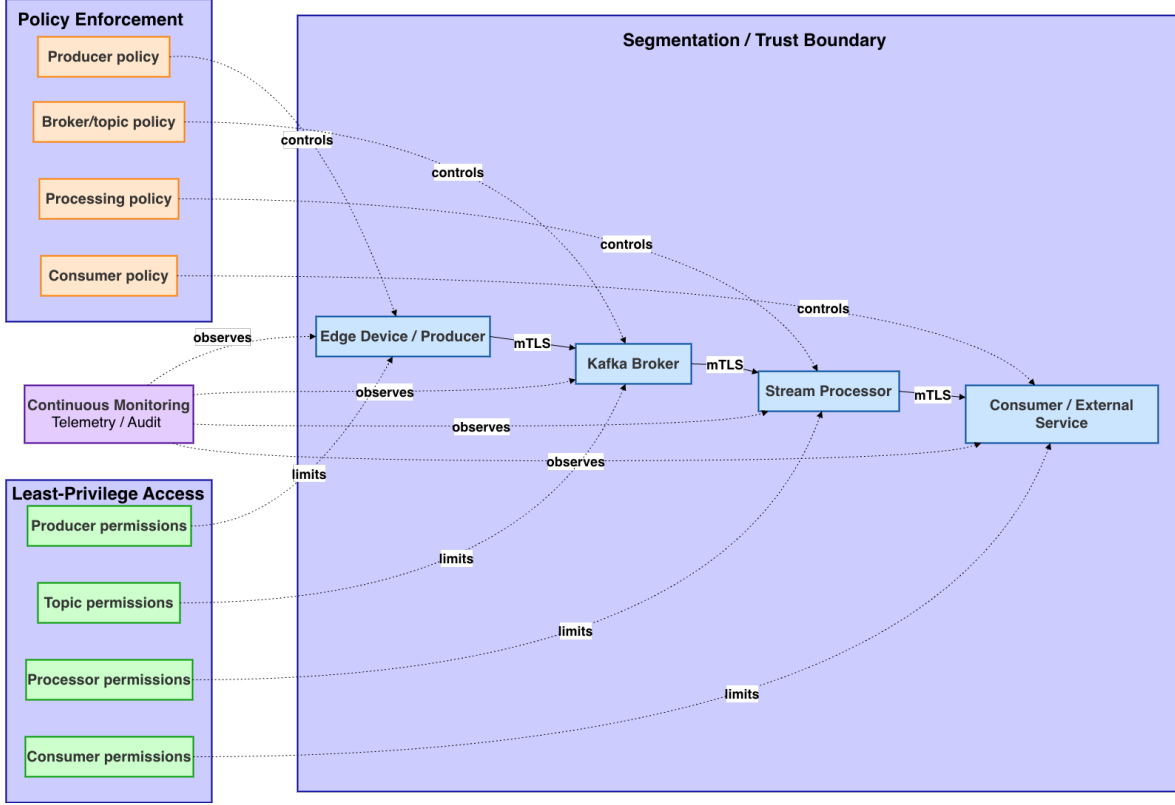


Figure 2.4: An illustration of the conceptual Zero Trust control model for a distributed streaming pipeline.

2.6 TLS/mTLS Performance Overhead

Existing TLS studies show that encrypted transport can introduce measurable overhead [18, 19]. Coarfa et al. separate TLS cost into connection setup, which uses asymmetric cryptography, and bulk data transfer, which uses symmetric encryption [18]. Their web-server experiments report a throughput degradation factor between 3.4 and 9 compared with plaintext baselines, depending on workload and session reuse [18]. They also identify RSA-based operations as a major processing cost under the evaluated workloads [18].

These findings are useful, but their transferability to Kafka is limited. Web-server TLS benchmarks often model short-lived request-response workloads, where new connections are frequent and data is already handled in user space [18, 19]. Kafka uses long-lived client-broker connections, batching, partitions, replication, page-cache reuse, and zero-copy I/O [6, 7, 14]. Because Kafka disables `sendfile` when SSL is enabled, TLS affects both cryptographic work and the data-movement path [6].

Persistent-connection studies provide a second perspective. Iorio et al. show that persistent

encrypted channels can reduce repeated handshake costs, stabilise TCP behaviour, and make steady-state TLS latency small on sufficiently provisioned devices [19]. This is relevant because Kafka also uses long-lived connections. However, these studies still do not capture Kafka’s broker-mediated, replicated, batch-oriented data path [6, 19].

In Kafka, TLS and mTLS can apply to producer-to-broker, consumer-to-broker, and broker-to-broker communication [3, 6, 13]. With mTLS, certificate-based authentication is added to encryption [13, 20]. The cost is therefore not limited to one channel. It may interact with replication, batching, compression, partitioning, client concurrency, and broker resources [6, 7, 13, 14].

The Kafka benchmarking literature adds a further limitation. Mohammad reports that many Kafka studies omit configuration details such as partition count, replication factor, batch size, linger time, compression settings, and workload characteristics [16]. This reduces reproducibility and makes it difficult to distinguish configuration effects from the effect being measured [7, 13, 16].

The literature therefore supports the expectation that TLS and mTLS overhead is workload-dependent, but does not provide the Kafka-specific matched evidence needed for this dissertation. What is missing is a reproducible comparison of plaintext, TLS, and mTLS where Kafka workload and deployment parameters are held constant and where both application-level metrics and host-level telemetry are collected.

2.7 Benchmarking Methodologies

Performance evaluation in distributed systems often begins with theoretical models. Network studies may use bisection bandwidth or sparsest cut to estimate throughput limits [21]. Kafka studies may use queueing models to relate latency to processing time and utilisation [5]. These models explain why saturation can increase latency and reduce throughput, but they are not sufficient on their own.

Jyothi et al. show that cut-based metrics do not always predict worst-case throughput, since a topology with a stronger cut metric can still perform worse under certain traffic patterns [21]. Similarly, Kafka producer evaluations show that queueing models can overestimate performance because they do not fully capture operating-system overhead, thread coordination, CPU contention, and network I/O contention [5]. Theoretical models are therefore useful for framing performance behaviour, but empirical benchmarking is required to measure Kafka under load [6, 16].

Latency measurement also requires care. Network-level round-trip time, measured through tools such as ping or traceroute, can show whether the network path is stable, but

does not capture the full application-level path [19]. Iorio et al. argue that application-level Flow Completion Time is more representative because it includes protocol processing, endpoint behaviour, buffering, proxying, and other effects that network-level probes can miss [19]. This matters because systems can appear stable at median latency while diverging at the tail due to endpoint processing, buffering, and shared-infrastructure effects [19].

Kafka benchmarking also has a reproducibility problem. Standardised suites such as TPCx-Kafka and synthetic frameworks such as the Yahoo Streaming Benchmark provide recognised baselines, while custom workloads can model domain-specific pipelines [12, 15, 16]. However, Mohammad’s review of 42 Kafka event-streaming studies shows that benchmark reporting is often incomplete and inconsistent [16].

This is especially problematic for security-overhead evaluation. Kafka performance depends on partition count, replication factor, message size, batch size, compression, producer concurrency, consumer lag, and acknowledgement settings [7, 13, 16]. If these are not controlled or reported, performance differences may be caused by configuration variation rather than TLS or mTLS overhead. Aggregate metrics can also hide client-side effects such as producer queueing and outbuffer contention [5].

A defensible Kafka security-overhead benchmark must therefore be transparent, reproducible, and configuration-explicit. It must use explicit workload definitions, matched comparisons, structured outputs, and enough metric detail to connect throughput and latency changes with resource-level behaviour. This directly motivates the artefact developed in Chapter 3.

2.8 Synthesis & Research Gap

The literature establishes that Kafka performance is produced by interacting mechanisms: append-only logs, page-cache utilisation, batching, partitioning, pull-based consumption, compression, replication, and zero-copy data transfer [3, 6, 7, 14]. These mechanisms make Kafka efficient, but also sensitive to configuration and additional data-path processing [5–7, 13, 16]. Kafka performance should therefore be understood as a configurable outcome rather than a fixed platform property.

The literature also establishes that Zero Trust Architecture responds to the weakness of implicit trust inside network boundaries [8–10]. Its principles include strong identity, least-privilege access, continuous verification, encrypted communication, monitoring, and fine-grained control [8–10]. Kafka does not implement Zero Trust as a complete architecture, but it provides mechanisms that can support Zero Trust-aligned deployments, including TLS encryption, mTLS authentication, SASL authentication, ACL authorisation, and broker-to-broker security configuration [7, 13, 14].

TLS and mTLS are the most relevant mechanisms for this dissertation because they directly affect Kafka’s data path. They provide a measurable basis for comparing plaintext, server-authenticated encrypted transport, and mutually authenticated encrypted transport [6, 13]. This creates a performance-security tension. Kafka’s plaintext path can use `sendfile`, but SSL prevents this optimisation because SSL libraries operate in user space [6]. TLS and mTLS can therefore introduce both cryptographic processing and additional user-space data movement.

Existing TLS studies provide evidence that encrypted transport can impose overhead, but they do not fully answer the Kafka-specific problem. Web-server studies emphasise connection setup and asymmetric cryptography [18, 20], while persistent-connection studies show that steady-state overhead can be smaller once handshakes are amortised [19]. Kafka is different because it is a replicated, partitioned, batch-oriented distributed log with producer-broker, consumer-broker, and broker-to-broker communication paths [3, 6, 13].

Existing Kafka performance studies also do not fully resolve the problem. They often examine throughput, batching, replication, producer parallelism, or benchmark-suite design without isolating TLS, mTLS, client authentication, and encrypted broker communication under matched configurations [5, 13, 16]. Where Kafka security is discussed, evaluations are often domain-specific, configuration-limited, or insufficiently reproducible [15, 16].

The research gap is therefore precise. The literature reviewed in this dissertation does not provide a reproducible matched factorial evaluation of plaintext, TLS, and mTLS Kafka deployments across producer concurrency, message size, batching, compression, acknowledgement mode, replication configuration, and broker count, while also collecting application-level metrics and host-level telemetry. This dissertation addresses that gap by designing and evaluating a Kafka benchmarking artefact that provisions cloud-based Kafka clusters, executes matched security-mode workloads, records producer throughput and latency, captures client and broker telemetry, and includes targeted consumer-side validation.

The contribution is both methodological and empirical. Methodologically, the artefact provides a reproducible approach for evaluating Kafka transport-security overhead under matched conditions. Empirically, it provides Kafka-specific evidence about how TLS and mTLS affect throughput, latency, and resource utilisation across workload and deployment parameters.

Chapter 3

Design, Implementation, and Testing

3.1 Artefact Requirements

The artefact requirements were derived from the research question and the methodological gap identified in Chapter 2. Since the dissertation evaluates Kafka transport-security overhead, the artefact must support controlled deployment, matched workload execution, structured metric capture, and repeatable comparison across plaintext, TLS, and mTLS.

The design was therefore guided by four methodological constraints: reproducibility, comparability, observability, and resumability. Reproducibility required infrastructure-as-code, explicit configuration files, and deterministic result directories. Comparability required matched workload rows across security modes. Observability required both application-level metrics and host-level telemetry. Resumability was required because the final benchmark campaign was too large to rely on uninterrupted cloud execution.

#	Main Functional Requirements
1	Provision cloud-based Kafka infrastructure using repeatable infrastructure-as-code.
2	Deploy Kafka in plaintext, TLS, and mTLS configurations.
3	Execute matched producer workloads across security modes.
4	Execute targeted consumer-side validation workloads.
5	Capture throughput, latency, and host-level telemetry.
6	Parse raw benchmark logs into structured result files.
7	Export results in formats suitable for analysis, statistical comparison, and dissertation reporting.

Table 3.1: Main Functional Requirements

#	Main Non-Functional Requirements
1	Reproducibility, so benchmark rows can be regenerated from explicit configuration files.
2	Comparability, so plaintext, TLS, and mTLS runs use matched workload and deployment parameters.
3	Resumability, so interrupted campaigns can continue without rerunning completed rows.
4	Fault tolerance, so transient failures do not corrupt local result state.
5	Configuration transparency, so workload and deployment parameters are recorded with each result.
6	Cost control, so cloud resources and benchmark scope remain manageable.
7	Extensibility, so additional workloads, metrics, statistical summaries, or security modes can be added later.

Table 3.2: Main Non-Functional Requirements

These requirements led to four central design decisions:

1. Terraform was used to make infrastructure provisioning repeatable and inspectable.
2. JSON configuration files were used to separate experiment definition from execution.
3. Checkpointing was added so long-running campaigns could resume after interruption.
4. Host telemetry was collected because throughput and latency alone cannot explain whether overhead appears through CPU pressure, network activity, disk activity, or client-side bottlenecks.

3.2 System Architecture

The artefact uses a cloud-based architecture built around a local workstation, Terraform infrastructure code, orchestration scripts, an AWS Virtual Private Cloud (VPC), Kafka broker instances, and a dedicated benchmark client. This design allows Kafka to be evaluated on virtual machines while also collecting host-level CPU, memory, disk, and network telemetry.

The local workstation acts as the control node. It stores the Terraform configuration, orchestration scripts, workload configuration files, generated execution plans, checkpoints, copied result files, and export outputs. Terraform provisions the AWS infrastructure, while the orchestration scripts prepare the benchmark client, deploy Kafka to the brokers, configure the selected security mode, execute benchmark runs, collect telemetry, and copy results back locally.

The AWS environment contains Kafka broker EC2 instances and one benchmark-client EC2 instance. The broker cluster supports both three-broker and five-broker phases, allowing broker count to be evaluated as an experimental variable. Kafka data-plane traffic uses private VPC addresses, while SSH orchestration uses public IP addresses. This keeps benchmark traffic inside the cloud network while still allowing the local workstation to manage the cluster.

The dedicated benchmark client runs Kafka producer and consumer benchmark tools. It connects to the brokers using the listener for the selected security mode: plaintext, TLS, or mTLS. Separating the benchmark client from the broker machines avoids producer-side CPU and network load competing with broker execution. This improves attribution because TLS and mTLS overhead may appear on the client, the broker, or both.

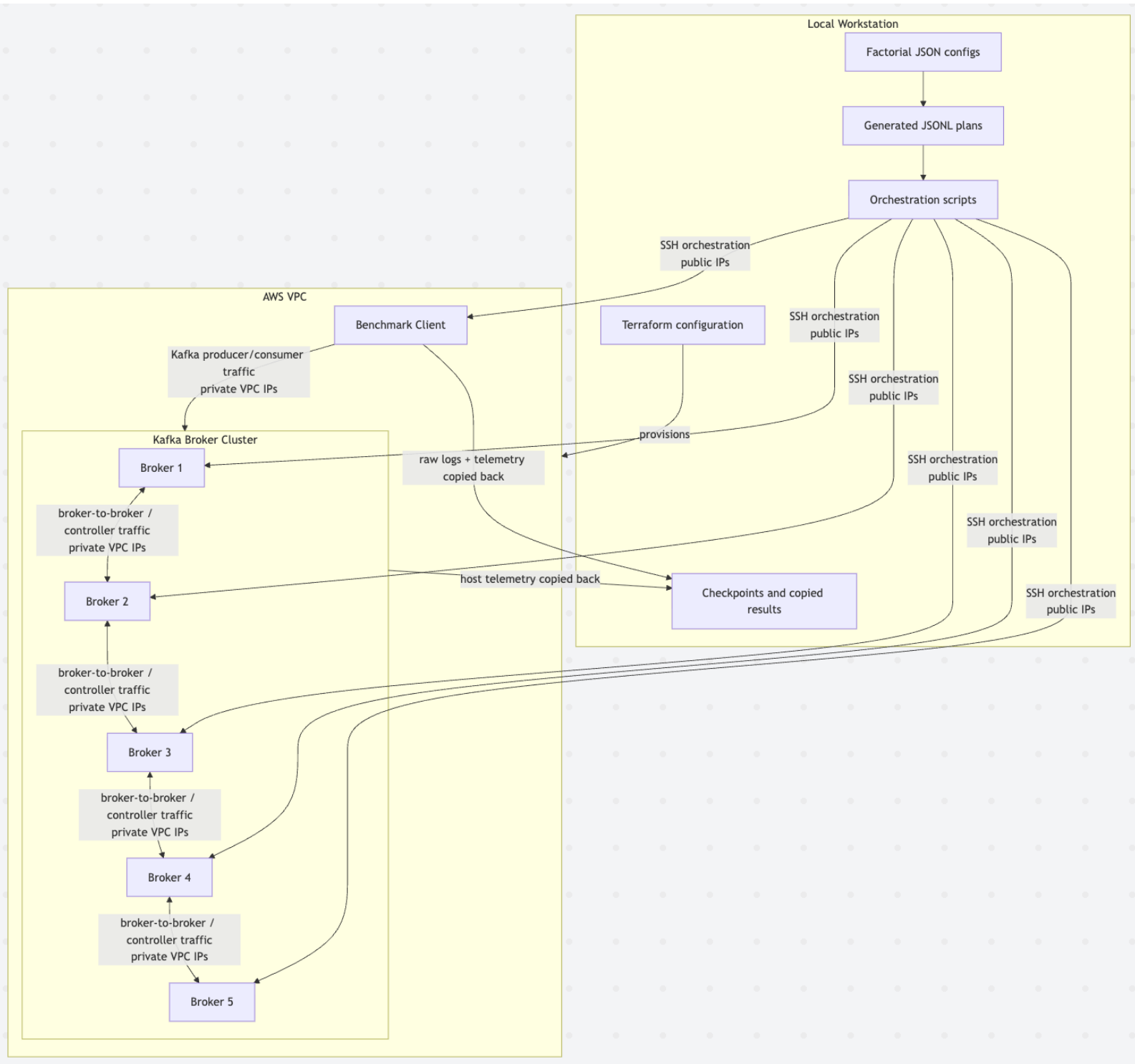


Figure 3.1: System architecture of the benchmarking artefact, showing local orchestration, Terraform-provisioned AWS infrastructure, private Kafka traffic, SSH control paths, broker instances, benchmark client, and result collection.

3.3 Reproducing the Artefact’s Environment

The artefact source code is available at github.com/VictoriousWealth/apache-kafka-artefact. The repository contains the Terraform infrastructure configuration, Kafka deployment scripts, orchestration pipeline, workload configuration files, result-processing scripts, and supporting documentation used for the dissertation.

The artefact was designed as a staged workflow rather than as a single monolithic script. Reproducing it therefore involves infrastructure provisioning, benchmark-client preparation, Kafka deployment, benchmark-plan generation, campaign execution, and result aggregation.

A researcher requires an AWS account, Terraform, SSH access, and a local clone of the artefact’s repository. The local workstation remains the control node, while the cloud environment contains the Kafka broker instances and the benchmark-client instance.

3.3.1 Infrastructure provisioning

Terraform creates the AWS VPC, subnet, security groups, Kafka broker EC2 instances, and benchmark-client EC2 instance. Helper scripts then generate host inventory and cluster metadata.

3.3.2 Benchmark-client preparation

A preparation script installs the Kafka distribution, benchmark scripts, telemetry utilities, and supporting runtime dependencies onto the benchmark-client host.

3.3.3 Kafka deployment

The cluster can be deployed in plaintext, TLS, or mTLS mode using the appropriate deployment script. The script configures Kafka for the chosen mode and validates broker readiness using Kafka API-level checks before benchmarks are started.

3.3.4 Benchmark-plan generation

Workloads are defined in JSON configuration files and expanded into JSONL execution plans. Each row represents one complete benchmark configuration, including message size, target throughput, batch size, acknowledgement mode, producer count, compression mode, replication factor, broker count, security mode, and trial index.

3.3.5 Campaign execution

The executor reads the JSONL plan, applies filters such as broker count or security mode, and executes rows that have not already completed. For each row, it creates the topic, starts

telemetry collection, runs the benchmark on the benchmark client, copies logs and telemetry back locally, parses the output, and records completion state. Campaign state is preserved in `started.jsonl`, `completed.jsonl`, `failures.jsonl`, and a checkpoint file.

3.3.6 Result processing

Each completed run produces a deterministic result directory containing metadata, raw logs, telemetry, and parsed `result.json`. Aggregation scripts combine valid per-run outputs into `summary.json` and `summary.csv`, while export scripts generate CSV tables, LaTeX tables, and figures. Reproducing the artefact therefore means reproducing the full path from deployment to analysis-ready outputs.

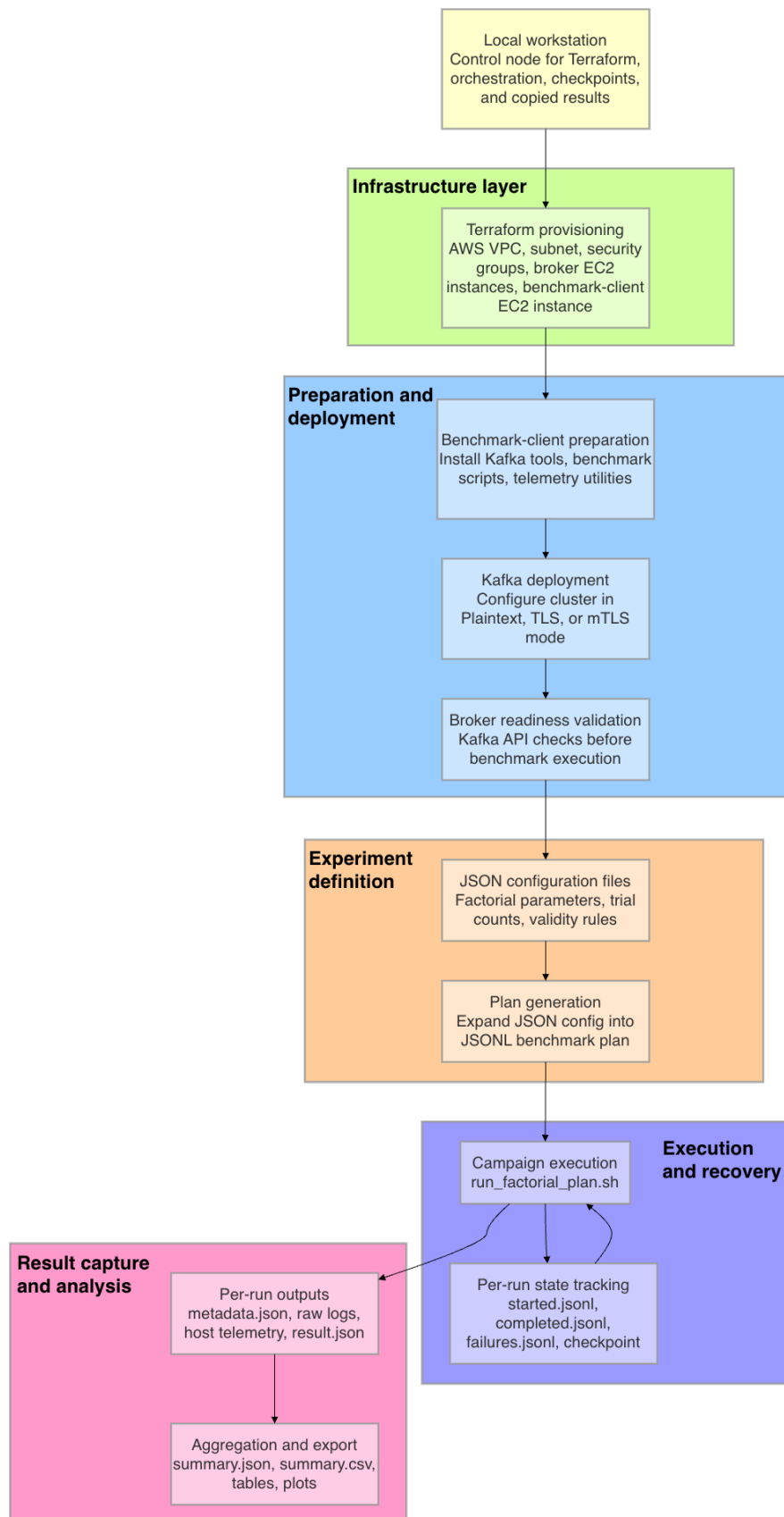


Figure 3.2: Tutorial-style reproduction workflow for the benchmarking artefact, showing how infrastructure provisioning, deployment, execution, checkpointing, and result processing are connected.

3.4 Infrastructure Provisioning

The infrastructure layer was implemented using Terraform and AWS EC2 to create a reproducible and inspectable experimental environment. Terraform enables infrastructure-as-code, allowing deployments to be version-controlled and recreated consistently. This is important for benchmarking because manual provisioning can introduce configuration drift and weaken performance comparisons.

The Terraform configuration provisions an AWS VPC, subnet, security groups, Kafka broker instances, and a benchmark client. The VPC isolates experimental traffic, while security groups restrict SSH, Kafka listener access, and inter-broker communication. Public IP addresses are used only for orchestration. Kafka benchmark traffic remains on private addresses, reducing external routing variability and better reflecting a real distributed deployment.

The infrastructure supports both three-broker and five-broker configurations. This prevents broker count from being hidden inside a single deployment choice and allows the experiment to test how deployment scale interacts with security overhead.

A separate benchmark client instance is used to isolate workload generation from broker execution. Running producers or consumers on broker nodes would make it harder to distinguish TLS/mTLS overhead from local resource contention. This separation improves measurement fidelity and allows client-side and broker-side telemetry to be interpreted independently.

Each EC2 instance uses a 40 GB gp3 root volume. This was introduced after early large-message experiments showed that insufficient storage could interrupt runs or create variability through forced cleanup. The selected volume size balances cost with stability across sustained workloads.

Virtual machines were selected over containers to provide clearer host-level telemetry. Containers improve portability, but can obscure CPU scheduling, memory accounting, and network I/O behaviour. Since this dissertation links Kafka performance to host-resource use, EC2 instances provide a more transparent measurement environment. The trade-off is that EC2 offers less control over noisy-neighbour effects than dedicated hardware, so results are interpreted as controlled cloud measurements rather than bare-metal measurements.

3.5 Kafka Cluster Deployment

Kafka was deployed in KRaft mode, where brokers participate directly in the metadata quorum without a separate ZooKeeper ensemble. This reduces external dependencies,

simplifies automated deployment, and limits the main experimental variables to Kafka workload, security mode, and infrastructure behaviour. The trade-off is that the evaluation focuses on modern Kafka cluster management and does not compare against legacy ZooKeeper-based deployments.

Kafka broker traffic uses private VPC IP addresses for producer, consumer, broker-to-broker, and controller communication. Public IP addresses are used only for SSH orchestration. This avoids routing Kafka data-plane traffic through the public internet and keeps communication within the intended experimental environment.

Benchmark topics are created using explicit workload parameters. Each run defines topic configuration, partition count, replication factor, and `min.insync.replicas`. Topics are deleted after runs where appropriate, reducing the risk that previous benchmark state or retained test data affects later runs.

Broker readiness is validated using Kafka API checks rather than relying only on `systemd` process status. A broker process can be running before the broker is ready to serve metadata requests or client traffic. API-level readiness checks reduce the risk of starting benchmark workloads against a partially available cluster.

3.6 Security Mode Design

The artefact supports three Kafka transport-security modes: plaintext, TLS, and mutual TLS (mTLS). These modes allow the experiment to compare an unsecured baseline, an encrypted server-authenticated configuration, and an encrypted mutually authenticated configuration under matched workloads.

Plaintext mode provides the baseline. Kafka clients and brokers communicate without transport encryption or certificate-based authentication. This is not a secure production configuration, but it is necessary for measuring the additional cost of TLS and mTLS.

TLS mode enables encrypted communication between Kafka clients and brokers. The broker presents a certificate, and the client verifies the broker's identity. This measures the overhead of encrypted server-authenticated transport.

mTLS mode extends TLS by requiring client certificate authentication. Clients also present certificates to the broker, allowing both sides of the connection to be authenticated. This is the closest evaluated configuration to a Zero Trust-aligned transport model because it combines encryption with certificate-based identity verification.

The TLS and mTLS configurations permit TLS 1.3 and TLS 1.2, with protocol negotiation

handled by the Kafka/JDK SSL stack. The artefact therefore measures deployed Kafka transport-security behaviour rather than isolating TLS-version-specific overhead.

Mode	Description
Plaintext	Baseline with no transport encryption.
TLS	Server-authenticated encrypted transport.
mTLS	Encrypted transport with client certificate authentication.

Table 3.3: Kafka transport-security modes evaluated by the artefact

3.7 Benchmark Workload Design

The workload design contains a producer-side factorial campaign and a targeted consumer-side validation campaign. The producer campaign is the main evaluation because the research question focuses on Kafka’s high-throughput write path. The consumer campaign is included as a smaller validation study to test whether secure transport also affects the read path.

The producer benchmark uses Kafka’s `kafka-producer-perf-test.sh`. This tool was selected because it is part of the Kafka distribution and reports producer throughput and latency under configurable workload settings. The artefact extends it with structured metadata, deterministic run identifiers, concurrent producer support, telemetry capture, parsing, aggregation, and checkpointed execution.

The producer campaign varies message size, target throughput, batch size, acknowledgement mode, producer count, compression type, replication factor, minimum in-sync replicas, broker count, security mode, and trial index. These parameters were selected because Chapter 2 identified them as important to Kafka’s throughput-latency-durability trade-off and because they are expected to interact with transport-security overhead.

Other parameters, such as retention time, log segment size, JVM options, and operating-system tuning, were kept outside the factorial design. They may affect Kafka performance, but they are less directly connected to immediate transport-security cost and would substantially increase campaign size. The design therefore prioritises parameters most relevant to the research question.

The consumer-side benchmark is intentionally smaller. A full consumer factorial campaign would increase runtime and cloud cost while duplicating much of the producer-side parameter space. The consumer slice therefore provides supporting read-path evidence without making the overall experiment infeasible.

3.8 Factorial Experiment Design

The producer campaign was designed as a controlled factorial experiment. This allows the dissertation to evaluate how Kafka transport-security overhead changes across workload and deployment conditions rather than measuring only one fixed configuration.

Experiment definition is separated from execution. Factorial designs are described in JSON configuration files and expanded into JSONL execution plans, where each line represents one full benchmark row. Each row is assigned a deterministic run identifier encoding the main parameters. This supports traceability because the same configuration maps to the same run identity.

Validity constraints are applied during plan generation. For example, replication factor and `min.insync.replicas` must be compatible with the selected broker count. This prevents invalid Kafka configurations and ensures that matched comparisons are made across meaningful rows.

The final producer campaign design contains 5,184 planned rows after validity constraints are applied: 1,728 plaintext rows, 1,728 TLS rows, and 1,728 mTLS rows. This balanced structure supports matched security-mode comparison because equivalent workload and deployment configurations can be evaluated under each security mode.

A fully exhaustive Kafka benchmark across all possible parameters would be infeasible within the dissertation's time and cost limits. The factorial design therefore focuses on parameters most likely to affect the interaction between Kafka throughput and cryptographic overhead. Repeated trials are included because cloud benchmark runs can be affected by transient behaviour, including JVM warm-up, network variation, and background operating-system activity. The matched row structure also supports paired analysis, because each TLS and mTLS result can be compared with a plaintext result using the same workload and deployment parameters.

3.9 Execution and Orchestration

The execution layer connects infrastructure provisioning, Kafka deployment, benchmark execution, telemetry capture, parsing, aggregation, and export. This keeps the experiment reproducible because the same configuration files and scripts can regenerate plans, resume campaigns, and rebuild summaries.

The `generate_factorial_plan.sh` script expands a factorial JSON configuration into a JSONL execution plan. This allows campaigns to be inspected, counted, filtered, and validated before cloud execution.

The main executor, `run_factorial_plan.sh`, reads the JSONL plan, applies filters such as broker count or security mode, and executes rows that have not already completed. For each row, it prepares metadata, creates the benchmark topic, starts telemetry, runs the producer or consumer benchmark, copies outputs back locally, parses the result, and records completion state.

Campaign state is recorded in `started.jsonl`, `completed.jsonl`, `failures.jsonl` where applicable, and a checkpoint file. Parser scripts convert raw benchmark and telemetry output into `result.json`. Aggregation scripts produce `summary.json` and `summary.csv`, while export scripts generate dissertation-ready CSV, LaTeX, and SVG outputs where required.

This design supports auditability because each run has explicit metadata, each campaign has recorded state, and each exported summary can be traced back to raw benchmark output. Figure 3.3 summarises the workflow.

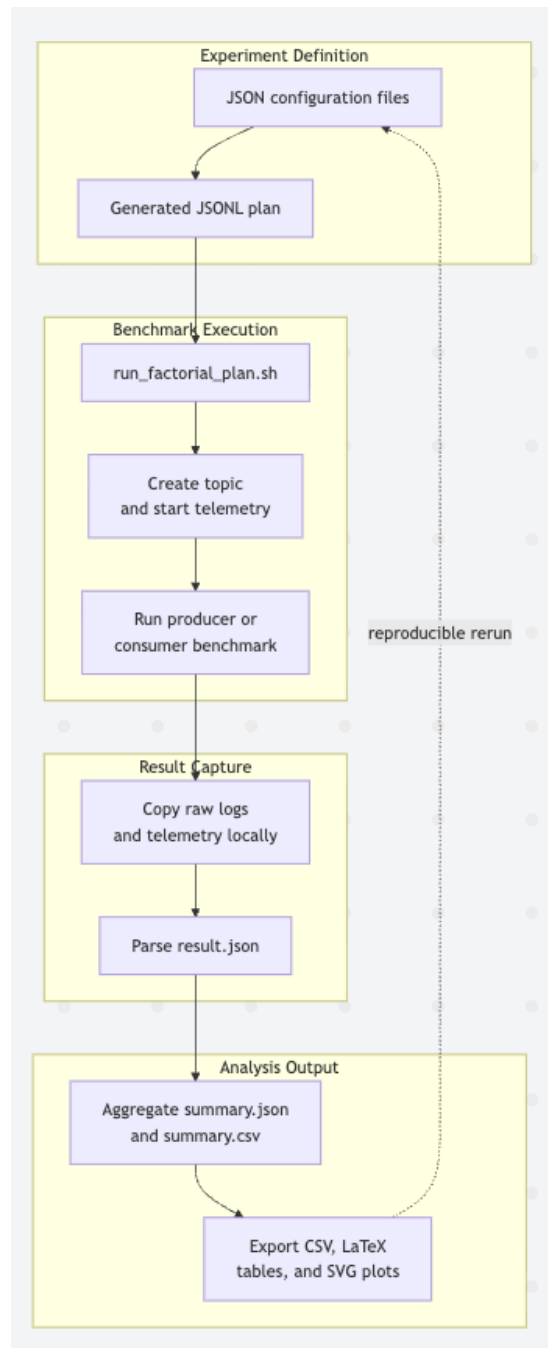


Figure 3.3: Benchmark execution pipeline from JSON configuration to dissertation-ready result exports.

3.10 Telemetry and Metric Collection

The artefact collects application-level metrics and host-level telemetry. Throughput and latency show benchmark outcome, while telemetry provides evidence about CPU, memory,

network, and disk behaviour under plaintext, TLS, and mTLS.

For producer runs, the main application metrics are throughput in records per second, throughput in MB per second, average latency, and maximum latency. For consumer runs, the parsed output records consumer throughput and read-path metrics where available.

Host telemetry is collected from both the benchmark client and Kafka brokers during execution. The telemetry includes CPU utilisation, memory utilisation, network receive/transmit activity, and disk read/write sector counters. Collecting telemetry from both sides of the connection is important because secure-transport overhead may appear on the client, the broker, or both.

Each run records workload and deployment metadata, including run identifier, security mode, broker count, replication factor, minimum in-sync replicas, message size, target throughput, batch size, acknowledgement mode, producer count, compression type, trial index, and topic configuration. This metadata allows results to be grouped and compared under matched conditions.

Some result files include interval-derived p95 and p99 fields. These are treated as diagnostic indicators rather than true per-record latency percentiles because they are derived from interval-level benchmark output rather than a full per-record latency distribution.

The combined metric set supports both comparison and interpretation. Throughput and latency measure the application-level effect of each security mode, while telemetry helps relate those effects to host-resource utilisation.

3.11 Result Processing and Export

The artefact separates raw execution from result processing. Each run produces a deterministic result directory containing raw benchmark output, metadata, telemetry files, and parsed result data. This preserves original evidence while also providing structured outputs for analysis.

Raw logs are retained so parsed results can be checked against Kafka benchmark output. Each run also contains a `metadata.json` file recording workload, deployment, and security parameters. The parser combines this metadata with raw benchmark output to produce `result.json`, the canonical structured result for that run.

At campaign level, aggregation scripts combine valid `result.json` files into `summary.json` and `summary.csv`. The JSON summary preserves structured fields, while the CSV summary provides a flat table for analysis, plotting, matched comparison, and statistical processing.

The export layer produces dissertation-ready outputs, including CSV tables, LaTeX tables, SVG plots, matched security-comparison files, and statistical summaries. The matched wide comparison files align plaintext, TLS, and mTLS rows that share the same workload and deployment parameters. These files are then used to calculate percentage penalties relative to plaintext. Positive penalty means worse than the matched plaintext baseline. For throughput, this means lower throughput. For latency and resource metrics, this means a higher value.

The statistical export stage adds three forms of evidence. First, bootstrap confidence intervals estimate uncertainty around key mean and median penalties using resampling with a fixed random seed. Second, matched-pair effect summaries report how often TLS or mTLS is worse than plaintext, alongside paired effect values and sign-test indicators. Third, factor-sensitivity summaries show which workload and deployment factors create the largest spread in mean overhead. These outputs support Chapter 4 without replacing the main matched comparison method.

Figure 3.4 shows how raw benchmark evidence is transformed into structured outputs.

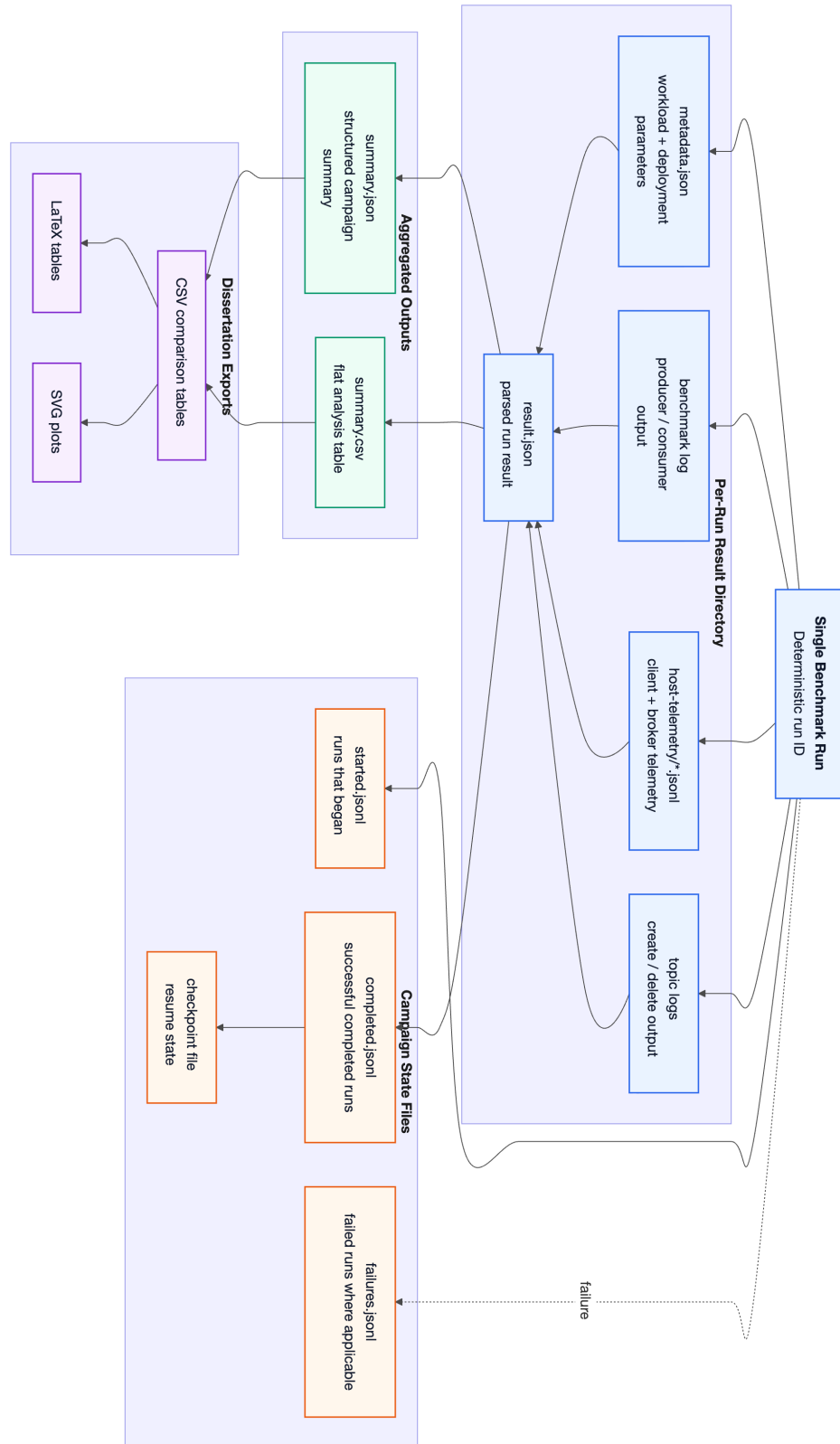


Figure 3.4: Result directory and data-flow schema showing how raw per-run logs, metadata, telemetry, campaign state files, parsed results, aggregated summaries, and dissertation-ready exports are connected.

3.12 Fault Tolerance and Idempotency

Fault tolerance and idempotency were necessary because the campaigns were long-running and executed on remote cloud infrastructure. Execution could be interrupted by SSH failure, local process termination, network instability, benchmark errors, or deliberate pauses to control cost.

Deployment checkpointing records which preparation stages have already completed, including inventory generation, benchmark-client preparation, broker bootstrap, Kafka configuration, and readiness validation. If deployment is interrupted, it can resume from recorded state.

Benchmark checkpointing records row-level execution state. The executor writes `started.jsonl`, `completed.jsonl`, `failures.jsonl` where applicable, and a checkpoint file. Before starting a run, it checks whether the deterministic run identifier has already completed and skips it if so.

A row is treated as complete only after remote execution finishes, result files are copied back, raw output is parsed, and the checkpoint is updated. This prevents interrupted rows from being treated as valid results.

Local writes and result copying use temporary files or temporary directories where possible, with outputs moved into final locations only after copying or parsing succeeds. Topic cleanup prevents retained benchmark data from exhausting broker disks during long-running campaigns.

Together, deployment checkpoints, benchmark checkpoints, deterministic run identifiers, temporary result copying, topic cleanup, retry wrappers, and Kafka API readiness checks make the artefact resilient to interruption and suitable for repeated execution.

3.13 Testing and Validation

Testing was carried out at multiple levels because the artefact combines infrastructure provisioning, Kafka deployment, security configuration, benchmark execution, telemetry capture, and result processing. The aim was to validate the full benchmarking pipeline rather than only individual scripts.

Infrastructure testing checked whether Terraform could provision the required AWS resources, including the VPC, subnet, security groups, broker instances, benchmark client, and storage volumes. Kafka deployment testing checked broker availability using Kafka API-level readiness checks.

Security-mode testing used smoke tests for plaintext, TLS, and mTLS. The plaintext smoke test validated the baseline producer path, the TLS smoke test validated encrypted server-authenticated transport, and the mTLS smoke test validated encrypted transport with client-certificate authentication.

Consumer-side testing validated the read path. The consumer smoke test confirmed that topics could be populated and consumed successfully under the selected security mode. The targeted mTLS consumer validation slice confirmed that consumer execution, telemetry capture, and structured result processing worked together.

Resumability was tested by restarting checkpointed campaigns and confirming that completed rows were skipped. Heavy-row testing validated larger message sizes and concurrent producer configurations. Parser and aggregation testing confirmed that raw Kafka logs could be converted into `result.json`, `summary.json`, and `summary.csv`.

Testing also changed the artefact. Early larger-message experiments exposed disk-capacity risk, which led to the use of larger gp3 root volumes. Deployment testing showed that `systemd` process state was not sufficient evidence of Kafka readiness, which led to Kafka API-level readiness checks. Long-running campaign interruptions motivated row-level checkpointing and deterministic run identifiers. These changes improved the final artefact by making execution more robust, auditable, and repeatable.

Test type	Purpose	Evidence produced
Terraform validation	Confirm infrastructure provisioning.	EC2 instances, VPC, subnet, security groups, and storage volumes were provisioned successfully.
Broker readiness checks	Confirm Kafka API availability before benchmark execution.	Kafka metadata and topic operations succeeded before benchmark workloads started.
Plaintext smoke test	Validate the baseline producer path.	Producer benchmark completed and produced parsed result output.
TLS smoke test	Validate encrypted server-authenticated transport.	TLS client configuration connected successfully and completed benchmark execution.
mTLS smoke test	Validate client-certificate authentication.	mTLS client certificate authentication completed successfully.
Consumer smoke test	Validate the read-path benchmark.	Consumer workload completed and produced structured output.
Resume test	Confirm checkpointed execution skips completed rows.	Previously completed run identifiers were skipped on resume.
Heavy-row test	Confirm larger workloads can execute.	Large-message and concurrent-producer rows completed without invalidating campaign state.
Parser test	Confirm structured result generation.	Raw logs were converted into <code>result.json</code> , <code>summary.json</code> , and <code>summary.csv</code> .

Table 3.4: Testing and validation evidence for the benchmarking artefact

These tests produced validation evidence across the full pipeline, from infrastructure provisioning and Kafka readiness to secure benchmark execution, checkpoint recovery, and structured result generation. Final campaign counts and benchmark findings are discussed in Chapter 4.

3.14 Chapter Summary

This chapter described the design, implementation, and testing of the Kafka benchmarking artefact. It showed how reproducibility, comparability, observability, and resumability shaped the use of Terraform-based AWS provisioning, EC2 broker and benchmark-client instances, Kafka KRaft deployment, explicit workload configuration, checkpointed orchestration, telemetry collection, and structured result processing.

The chapter also explained how plaintext, TLS, and mTLS were implemented as the transport-security modes used for comparison. Plaintext provides the baseline, TLS adds encrypted server-authenticated transport, and mTLS extends this with client certificate authentication. These modes are evaluated as Zero Trust-aligned transport-security mechanisms rather than as a complete Zero Trust Architecture.

Overall, the artefact defines the infrastructure, workload parameters, security modes, result schema, telemetry collection, checkpointing behaviour, validation process, and statistical export method needed to compare Kafka transport-security overhead under matched conditions. The following chapter uses these outputs to evaluate the performance overhead of TLS and mTLS, including descriptive results, matched comparisons, uncertainty estimates, and workload-sensitivity evidence.

Chapter 4

Results and Discussion

4.1 Chapter Overview

This chapter presents the benchmark results produced by the artefact described in Chapter 3. It evaluates how TLS and mTLS affect Apache Kafka throughput, latency, and host-level resource behaviour under matched workload and deployment configurations.

The analysis uses controlled comparison. Plaintext, TLS, and mTLS are compared only where message size, target throughput, batch size, acknowledgement mode, producer count, compression mode, replication factor, minimum in-sync replicas, broker count, and trial structure are matched. This allows differences to be interpreted as transport-security overhead rather than workload variation.

The chapter first reports result-set completeness and the matched comparison method, including the statistical summaries added to the final export stage. It then analyses producer throughput, producer latency, host telemetry, and targeted consumer-side validation before discussing the main patterns.

4.2 Result-Set Completion and Validity

The final producer campaign contained 5,184 planned rows after validity constraints: 1,728 plaintext rows, 1,728 TLS rows, and 1,728 mTLS rows. These rows were generated from the factorial configuration described in Chapter 3 and executed through the checkpointed orchestration pipeline.

At the time of final analysis, the completed result set contained 5,184 successful producer rows and 72 successful consumer validation rows. Table 4.1 summarises the final completion state by campaign phase and security mode.

A row was treated as valid only if the benchmark completed successfully, results were

copied back locally, raw output was parsed, and the row appeared in the completed execution state. Rows with incomplete output, parsing failure, or missing execution state were excluded because partial rows would weaken the security-mode comparison.

The analysis also required controlled comparability across security modes. Where a row did not have valid plaintext, TLS, and mTLS counterparts, it was excluded from comparative overhead calculations. The core overhead analysis therefore uses the matched subset rather than all completed rows regardless of comparability.

This stricter rule matters because a large Kafka benchmark can complete successfully while still being unsuitable for direct security-mode comparison. For example, a plaintext row and a TLS row are only comparable if they share the same workload and deployment parameters. The validity process therefore protects the analysis from mixing security effects with differences caused by broker count, replication, producer concurrency, compression, or trial structure.

Result set	Planned	Started	Completed	Failures	JSON	Matched
plaintext_broker5	1296	1328	1296	29	1296	1296
tls_broker5	1296	1301	1296	2	1296	1296
mtls_broker5	1296	1299	1296	0	1296	1296
plaintext_broker3	432	432	432	0	432	432
tls_broker3	432	432	432	0	432	432
mtls_broker3	432	432	432	0	432	432
consumer_plaintext_broker5	24	24	24	0	24	24
consumer_tls_broker5	24	28	24	2	24	24
consumer_mtls_broker5	24	24	24	0	24	24

Table 4.1: Final benchmark completion and matched-analysis counts. Started counts can exceed planned counts because interrupted or failed attempts were retried during checkpointed execution. The failure counts, therefore, record failed attempts, not final invalid planned rows.

The checkpointed execution model described in Chapter 3 was important for validity. Long-running campaigns could resume without rerunning completed rows, while interrupted rows were not counted until copying, parsing, and checkpoint updates had completed.

4.3 Comparison Method

The main analytical goal is to quantify transport-security overhead. Results were therefore compared using matched benchmark rows rather than broad averages across the whole campaign. Each comparative unit holds workload and deployment configuration constant

while changing only the security mode.

A matched producer comparison row contains the following shared attributes: broker count, replication factor, minimum in-sync replicas, message size, target messages per second, batch size, acknowledgement mode, producer count, compression mode, and trial index. The only field that changes across the matched set is the security mode. This produces a three-way comparison of plaintext, TLS, and mTLS under otherwise equivalent conditions.

The same principle was used for consumer validation, although that slice is smaller and treated as supporting evidence. Consumer comparisons were made only where consumer count, message size, compression mode, trial index, and security mode were aligned.

For each matched set, the export retained raw percentage change and penalty percentage. For throughput, positive penalty means lower throughput than plaintext. For latency and resource metrics, positive penalty means a higher value than plaintext. Positive penalty therefore consistently means worse than the matched plaintext run.

The final export stage also produced statistical summaries from the matched comparison files. Bootstrap 95% confidence intervals estimate uncertainty around key mean and median penalties. Matched-pair summaries record how often TLS or mTLS is worse than plaintext under the same row configuration. A sign-test approximation is included as supporting evidence, but interpretation focuses on effect size, confidence intervals, and penalty direction because large samples can make small effects statistically detectable.

This statistical layer is used to qualify the descriptive results rather than to replace them. The mean and median penalties show the size of the observed effect, the bootstrap intervals show how stable that estimate is within the completed dataset, and the matched-pair summaries show whether the effect is widespread or concentrated in a smaller number of rows. This is useful because Kafka results can include both extreme runs and configurations where secure transport has little measured penalty.

The chapter uses five main result perspectives:

1. Absolute application-level performance, measured through throughput and latency.
2. Relative security overhead, measured as change from the matched plaintext baseline.
3. Bootstrap uncertainty, measured through 95% confidence intervals for key penalties.
4. Host-level telemetry behaviour, measured through CPU, memory, network, and disk activity.

5. Cross-factor sensitivity, measured by checking which workload and deployment variables create the largest overhead spread.

This means the chapter does not only ask whether TLS or mTLS is slower than plaintext. It asks how large the penalty is, how stable it is, which factors amplify it, and how host telemetry helps interpret it.

4.4 Producer Throughput Results

Producer throughput is the primary metric because the main campaign evaluates Kafka's write path. The analysis begins with throughput in records per second, with MB/s retained in the wider exports for supporting inspection.

Figure 4.1 summarises matched throughput behaviour. Plaintext provides the baseline, TLS adds encrypted server-authenticated transport, and mTLS adds client certificate authentication.

Across the matched producer rows, TLS and mTLS reduced mean throughput relative to plaintext, but mTLS was not consistently worse than TLS. Plaintext averaged 2,848.49 records/s, compared with 2,336.51 for TLS and 2,356.73 for mTLS. TLS had a mean throughput penalty of 10.83%, while mTLS had 9.57%.

The bootstrap intervals support this interpretation. TLS had a mean throughput penalty interval of 10.17–11.49%, while mTLS had 8.90–10.23%. These intervals show that the penalty is not driven by a small number of outliers. However, plaintext did not win every row: TLS was worse than plaintext in 74.31% of matched rows, while mTLS was worse in 67.71%.

The results therefore show measurable secure-transport overhead, but not a simple ordering where mTLS is always worse than TLS. The additional client-certificate mechanism did not translate into consistently lower producer throughput.

Producer concurrency amplified throughput penalties, while larger records did not necessarily increase them. Mean penalty rose from 2.71% for TLS and 1.55% for mTLS at one producer to 17.02% and 16.05% at twelve producers. By contrast, penalty was lower at 100 KB records than at 1 KB or 10 KB records.

The factor-sensitivity export gives the same result more systematically. For producer throughput, target throughput and producer count created the largest mean-penalty spread. Under TLS, the spreads were 16.68 percentage points across target throughput and 14.32 across producer count. Under mTLS, the corresponding spreads were 15.48 and 14.50 points.

This pattern is important because it shows that throughput overhead was not just a property of the security protocol. It became more visible when the workload placed greater pressure on the producer path. Therefore, a low-concurrency Kafka deployment may observe a smaller secure-transport cost than a deployment with many producers and higher target throughput, even when the same TLS or mTLS configuration is used.

Compression also affected relative overhead. Mean throughput penalties were lower with lz4 than with no compression for both TLS and mTLS, suggesting that reduced data volume can offset part of the secure-transport cost.

Table 4.2 reports the main matched throughput summary. Figures 4.2 and 4.3 show how overhead changes with message size and producer concurrency.

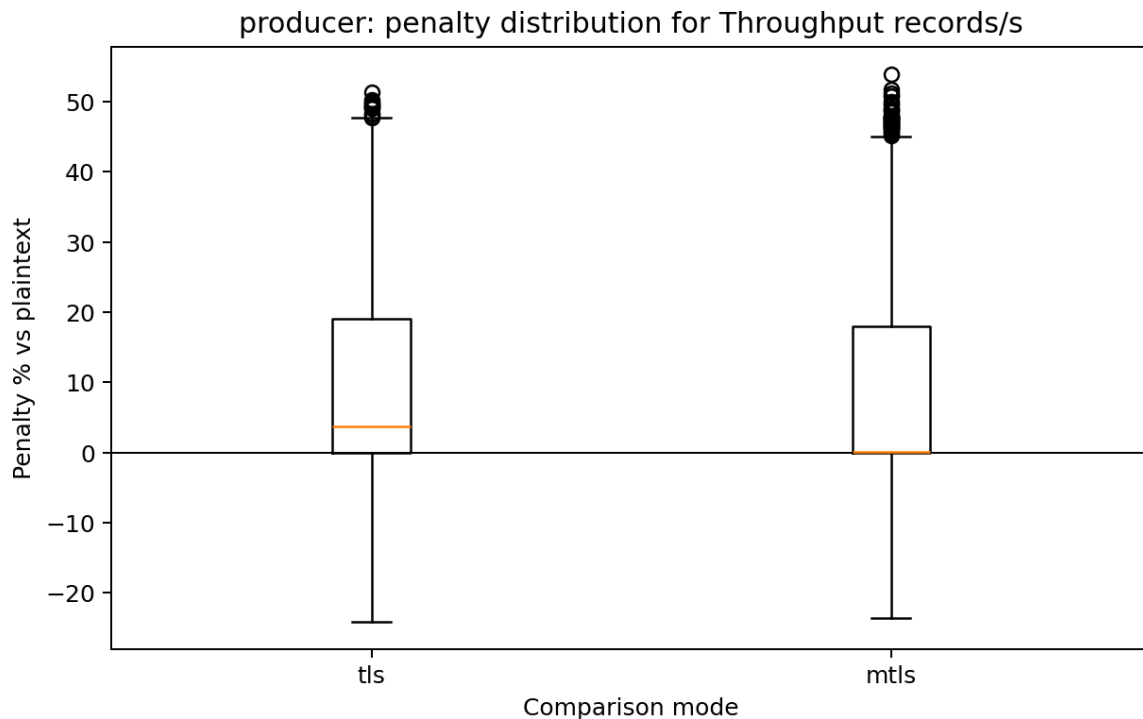


Figure 4.1: Matched producer throughput penalty for TLS and mTLS relative to plaintext, measured in records/s. Positive values mean lower throughput than the matched plaintext run; negative values mean higher throughput. The orange line is the median, the box is the middle 50% of runs, whiskers show the non-outlier range, and circles are outliers.

Metric	Plaintext	TLS	mTLS
Matched rows	1,728	1,728	1,728
Mean throughput (records/s)	2,848.49	2,336.51	2,356.73
Median throughput (records/s)	999.64	999.64	999.63
Mean penalty vs plaintext	—	10.83%	9.57%
Median penalty vs plaintext	—	3.77%	0.12%
Worst penalty vs plaintext	—	51.35%	53.98%
Best penalty vs plaintext	—	-24.07%	-23.65%

Table 4.2: Matched producer throughput summary in records/s. Penalty values compare TLS and mTLS against equivalent plaintext runs under the same workload, deployment, and trial configuration. Positive penalty means lower throughput than plaintext; negative penalty means higher throughput than plaintext.

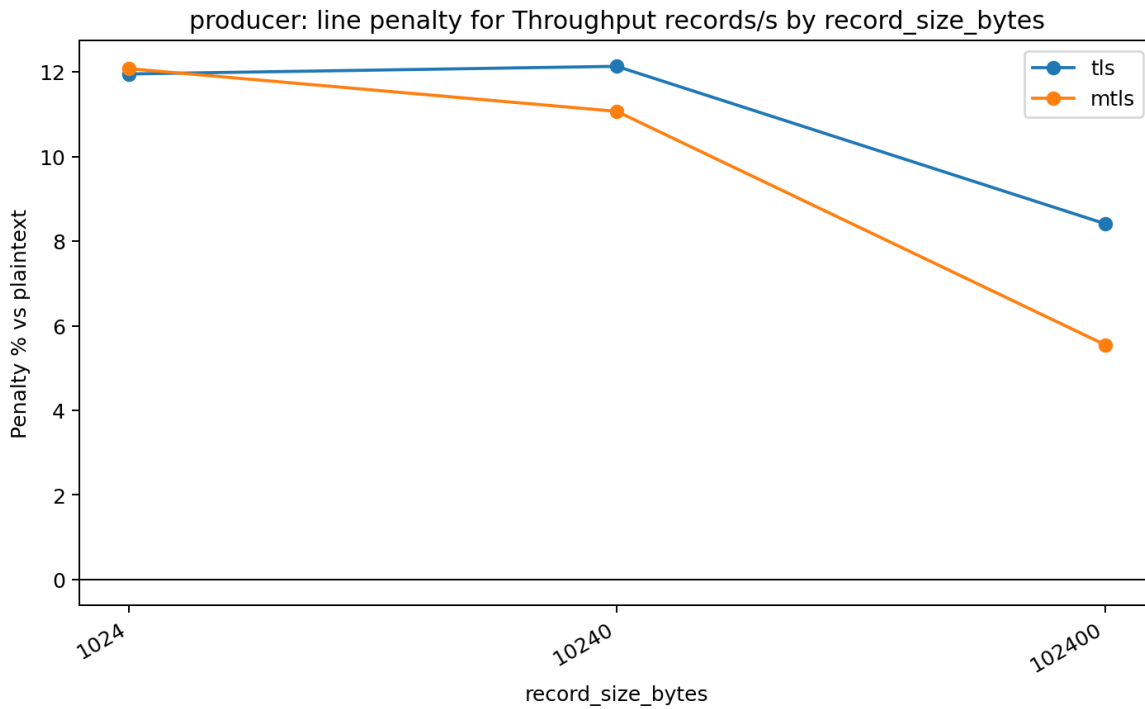


Figure 4.2: Mean matched producer throughput penalty by record size, measured in records/s relative to plaintext. Positive values mean TLS or mTLS achieved lower throughput than the matched plaintext run. Penalty is highest at 1 KB and 10 KB records, then falls at 100 KB records, suggesting that throughput overhead is not simply proportional to payload size.

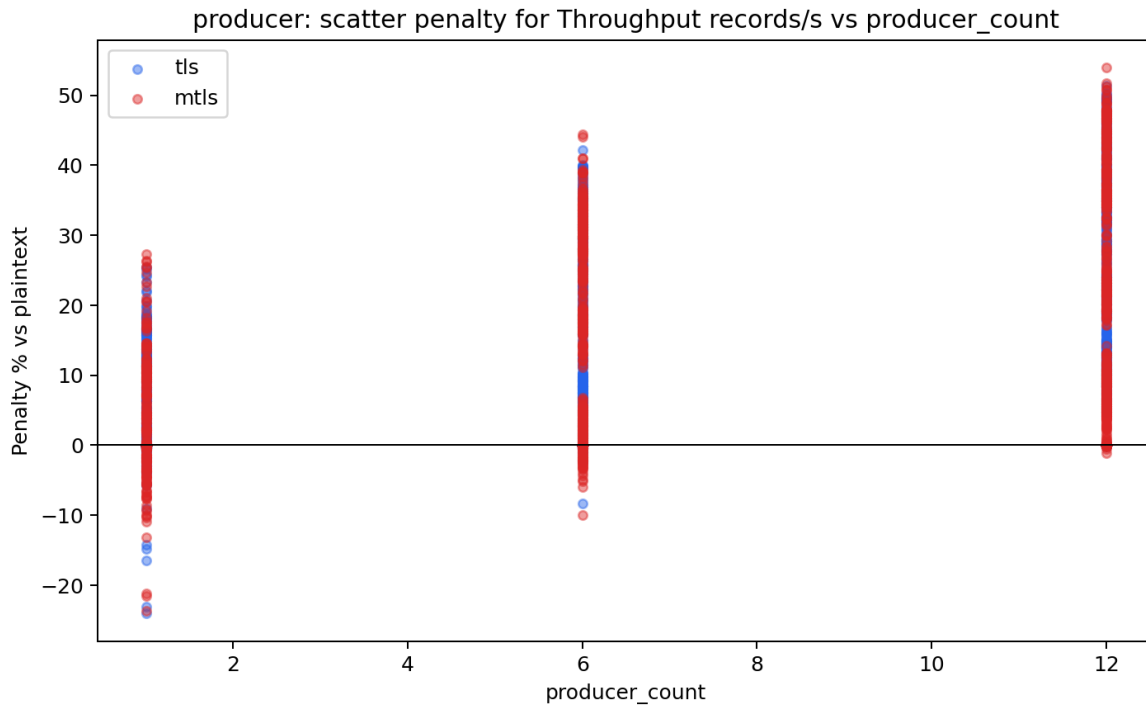


Figure 4.3: Matched producer throughput penalty by producer count, measured in records/s relative to plaintext. Each point compares TLS or mTLS with the equivalent plaintext run. Positive values mean lower throughput than plaintext; negative values mean higher throughput. Penalties spread wider and reach higher values as producer count increases.

Overall, secure transport reduced Kafka write-path throughput, but the size of that reduction depended strongly on workload shape. Producer count and target throughput were the clearest amplifiers, while larger records did not automatically produce larger penalties.

4.5 Producer Latency Results

Throughput alone is not sufficient to evaluate Kafka transport-security overhead. A configuration can sustain similar throughput while introducing higher delay, especially near saturation. The producer analysis therefore also considers average latency and maximum latency.

Average latency gives the central delay behaviour. Maximum latency captures the worst observed delay during a run, which is useful when the system experiences transient contention or queue build-up. Interval-derived `p95` and `p99` fields are treated as diagnostic indicators rather than true per-record percentiles, consistent with Chapter 3 and Appendix C.

Figure 4.4 summarises the matched latency comparison. The important point is not only that secure transport increased latency, but that the increase depended on the same

workload factors that shaped throughput.

Secure transport affected latency more sharply than throughput. Mean average-latency penalty exceeded 950% for both TLS and mTLS, while mean maximum-latency penalty reached 79.30% for TLS and 91.80% for mTLS. Average-latency penalty rose sharply from one to six producers, then more slowly from six to twelve.

The confidence intervals show that these latency penalties were stable despite extreme individual rows. TLS had a mean average-latency interval of 901.37–1004.04%, while mTLS had 905.55–1010.04%. For maximum latency, the intervals were 77.89–80.69% and 90.09–93.45%. Secure transport increased average latency in more than 99% of matched producer rows and maximum latency in more than 98%.

Maximum latency is important for performance-sensitive streaming systems because a modest throughput loss can still be operationally significant if outlier delay rises sharply. Latency therefore has to be interpreted alongside throughput rather than as a secondary metric.

The factor-sensitivity results show that batch size and producer count were the clearest latency amplifiers. For average latency, the mean penalty spread between batch-size levels was 945.20 percentage points for TLS and 946.41 for mTLS. The spread between one and twelve producers was 811.94 points for TLS and 841.51 for mTLS.

This supports a queueing-based interpretation of the results. Secure transport adds processing and data-movement cost, but the latency effect becomes most visible when the producer side is already handling larger batches or more concurrent producers. Under these conditions, small increases in processing time can translate into much larger waiting-time effects. This explains why latency changed more sharply than throughput.

Table 4.3 summarises matched latency outcomes, while Figures 4.5 and 4.6 show how overhead changes across message size and producer count.

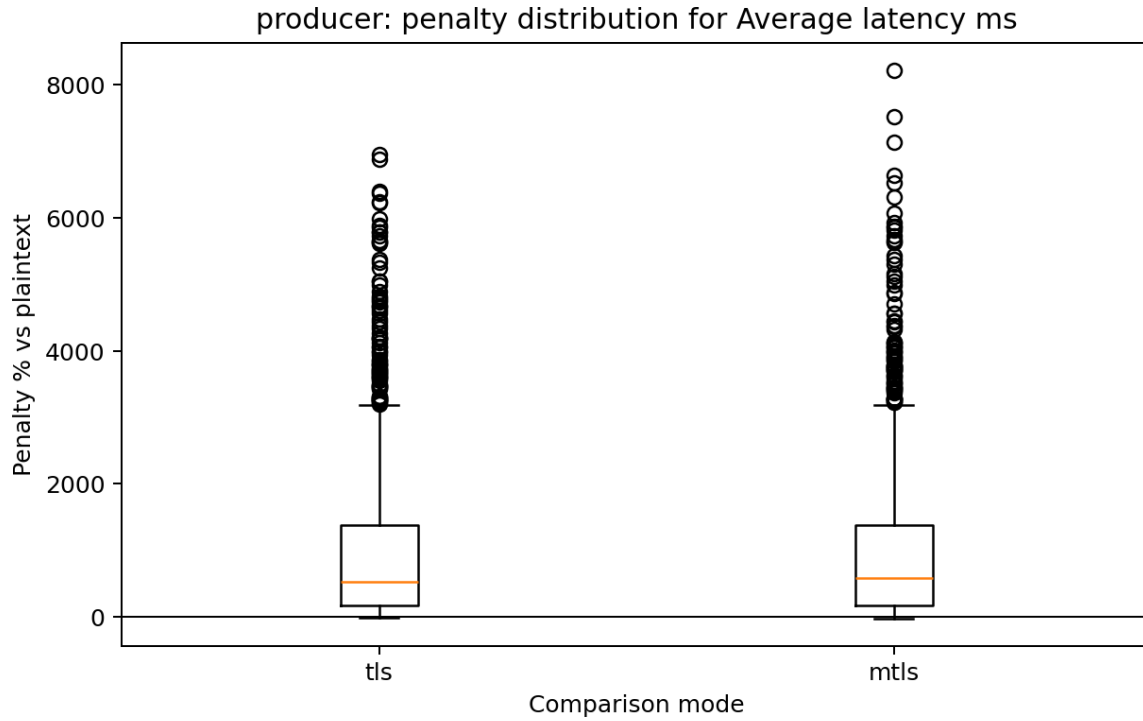


Figure 4.4: Matched producer average-latency penalty for TLS and mTLS relative to plaintext. Positive values mean higher latency than the matched plaintext run. The boxplot shows that secure transport produces substantially larger latency penalties than the throughput penalties reported earlier.

Comparison metric	Plaintext	TLS	mTLS
Matched rows	1,728	1,728	1,728
Mean average latency (ms)	537.41	1,861.07	1,837.07
Median average latency (ms)	55.68	484.21	473.59
Mean average-latency penalty	–	952.23%	957.81%
Median average-latency penalty	–	530.30%	576.31%
Mean maximum latency (ms)	5,343.48	9,544.81	10,129.98
Median maximum latency (ms)	5,074.00	8,971.00	9,434.50
Mean maximum-latency penalty	–	79.30%	91.80%
Median maximum-latency penalty	–	79.36%	88.03%

Table 4.3: Matched producer latency summary

Metric	TLS mean 95% CI	mTLS mean 95% CI	TLS worse rows	mTLS worse rows
Throughput records/s	10.17–11.49%	8.90–10.23%	74.31%	67.71%
Average latency ms	901.37–1004.04%	905.55–1010.04%	99.42%	99.31%
Maximum latency ms	77.89–80.69%	90.09–93.45%	98.15%	98.55%
Benchmark-client CPU	-0.80–8.42%	-3.94–5.31%	59.64%	48.93%
Broker CPU mean	-18.40 to -15.75%	-28.87 to -26.04%	26.55%	18.19%

Table 4.4: Bootstrap and matched-pair summary for key producer penalties. Confidence intervals are 95% bootstrap intervals for mean penalty. Worse-row percentages show the proportion of matched rows where the secure mode was worse than plaintext for the same metric.

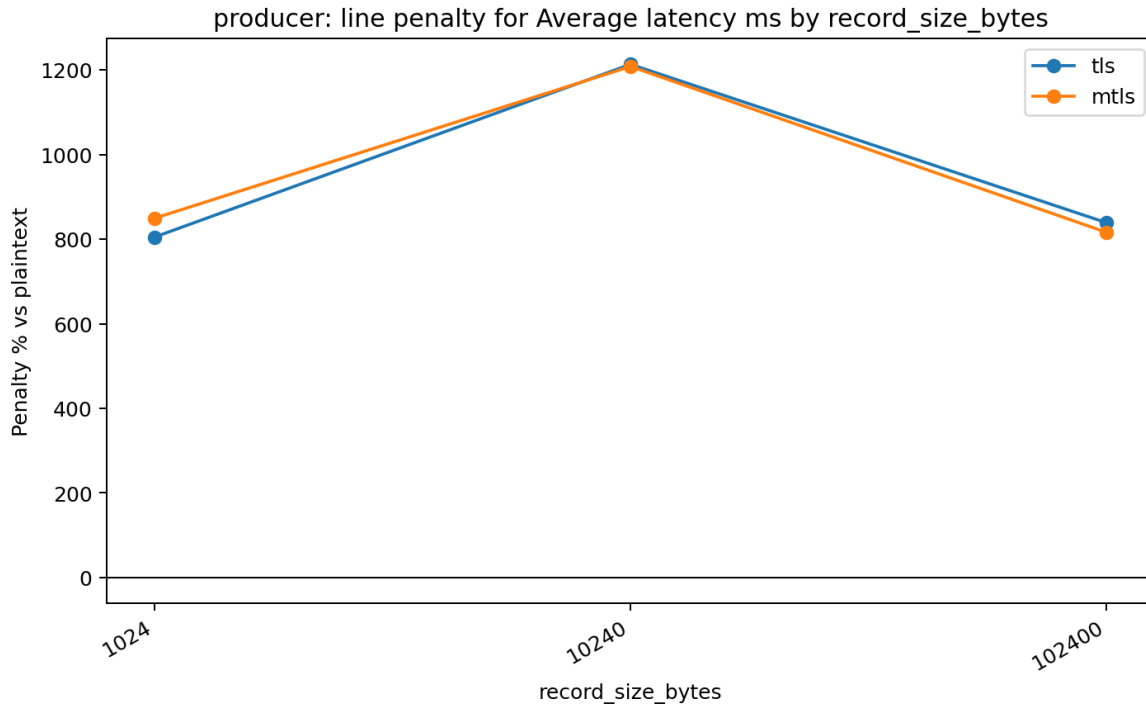


Figure 4.5: Mean matched producer average-latency penalty by record size, relative to plaintext. Positive values mean TLS or mTLS had higher average latency than the matched plaintext run. Latency penalty peaks at 10 KB records, then falls for 100 KB records.

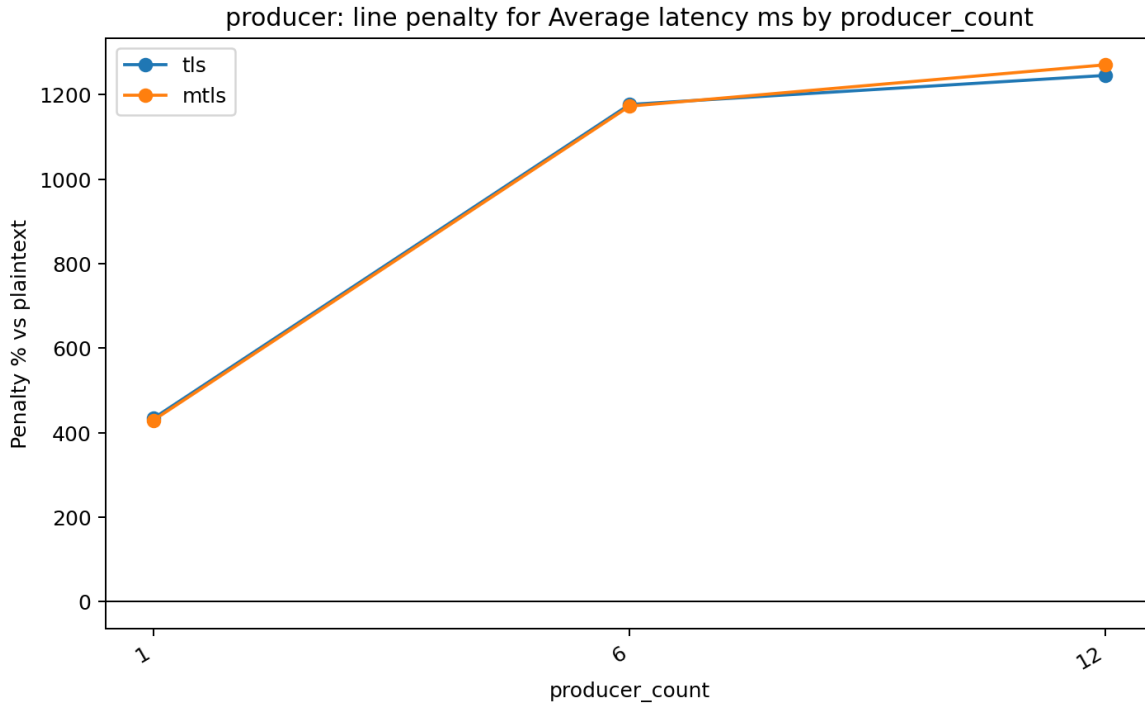


Figure 4.6: Mean matched producer average-latency penalty by producer count, relative to plaintext. Positive values mean higher latency than the matched plaintext run. Penalty rises sharply from 1 to 6 producers, then increases more slowly from 6 to 12 producers.

Taken together, the latency results show that secure transport affected not only write rate, but also the producer delay profile under heavier batching and concurrency.

4.6 Host Telemetry Results

Application-level metrics show that overhead exists, but not where it appears. The artefact therefore captured telemetry from the benchmark client and brokers, allowing throughput and latency to be interpreted alongside CPU, memory, network, and disk behaviour.

The main analytical value of telemetry is attribution, but attribution is not straightforward. TLS and mTLS reduced throughput and increased latency, yet this did not appear as a simple increase in all CPU metrics. Benchmark-client CPU remained high across modes, while broker CPU was generally lower under TLS and mTLS than plaintext.

Figure 4.7 compares benchmark-client and broker CPU utilisation. It shows that resource behaviour does not follow a simple system-wide increase under secure transport.

The statistical summaries reinforce this caution. TLS benchmark-client CPU had a mean penalty of 2.49%, but its 95% interval crossed zero at -0.80–8.42%. mTLS had -0.55%,

with -3.94–5.31%. Client CPU is therefore a weak explanation for the main throughput and latency penalties. Broker CPU showed clearer negative penalties, which is consistent with lower achieved throughput and changed pacing rather than reduced cryptographic work.

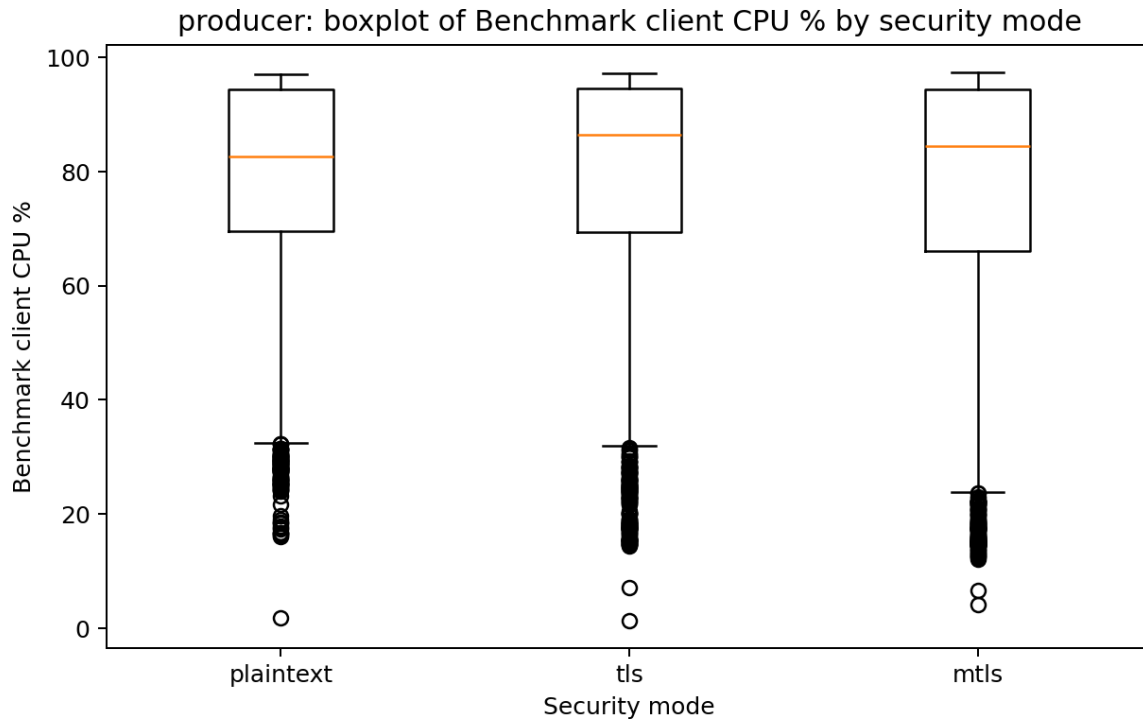
Network, disk, and memory were also supporting rather than decisive. Network counters did not increase in proportion to latency penalties. Broker disk write sectors were broadly similar across modes, and broker memory remained close across plaintext, TLS, and mTLS. The main security effect therefore appears at system level, not as a single isolated host counter.

This does not make telemetry unimportant. Instead, it prevents overclaiming. The telemetry shows that secure transport changed the operating profile of the benchmark, but it does not justify attributing the full overhead to one host metric. The more defensible conclusion is that TLS and mTLS changed the interaction between producer pacing, broker work, network transfer, and achieved throughput.

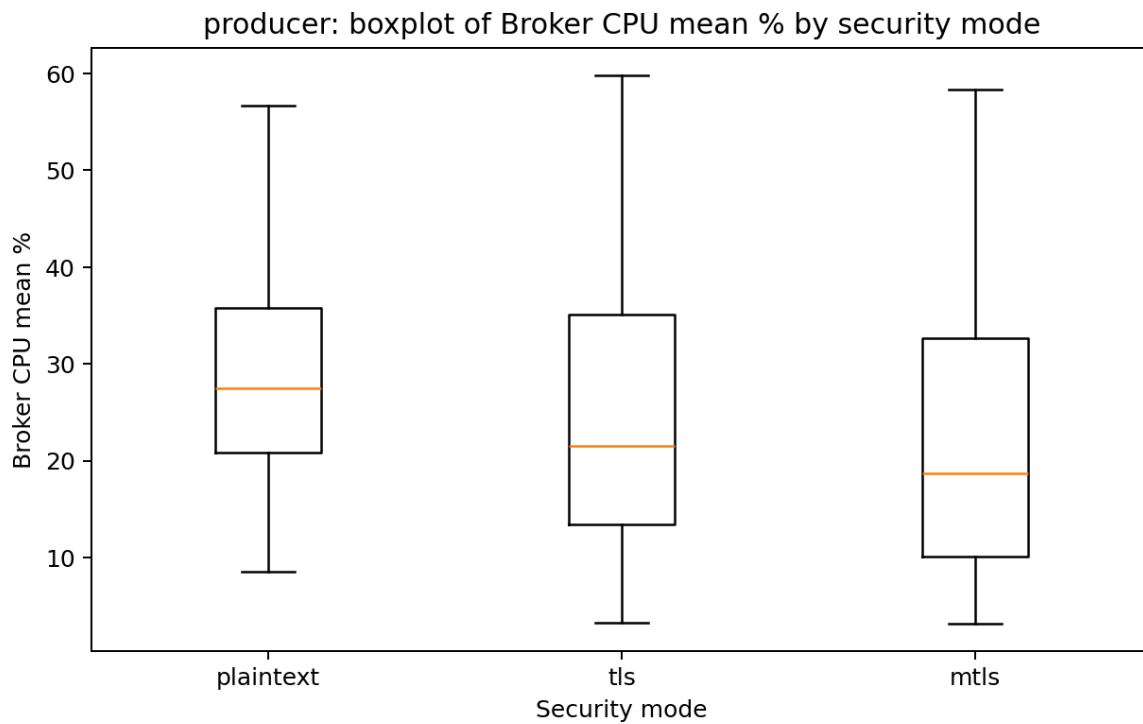
Table 4.5 reports the telemetry summary, while Figures 4.7 and 4.8 show the highest-value relationships for interpretation.

Telemetry metric	Plaintext	TLS	mTLS
Benchmark-client CPU (%)	76.96	77.40	75.76
Broker CPU (%)	28.71	24.96	22.35
Benchmark-client memory (%)	17.86	19.87	19.02
Broker memory (%)	23.41	23.31	23.22
Benchmark-client network RX (MB)	18.04	21.99	22.35
Benchmark-client network TX (GB)	3.84	3.86	3.87
Broker network RX (GB)	3.44	3.47	3.47
Broker network TX (GB)	2.56	2.58	2.58
Benchmark-client disk read sectors	106	126	147
Benchmark-client disk write sectors	44,262	44,478	49,353
Broker disk read sectors	8,109	7,239	6,510
Broker disk write sectors	6,421,375	6,422,382	6,463,730

Table 4.5: Matched producer host-telemetry summary. Values are means across matched producer rows; broker values use the per-run broker mean, and network counters are converted from bytes using decimal MB/GB.



(a) Benchmark client CPU



(b) Broker CPU

Figure 4.7: Matched CPU utilisation by security mode for producer runs. The orange line is the median, the box shows the middle 50% of runs, whiskers show the non-outlier range, and points are outliers. Client CPU remains high across modes, while broker CPU is generally lower for TLS and mTLS than plaintext.

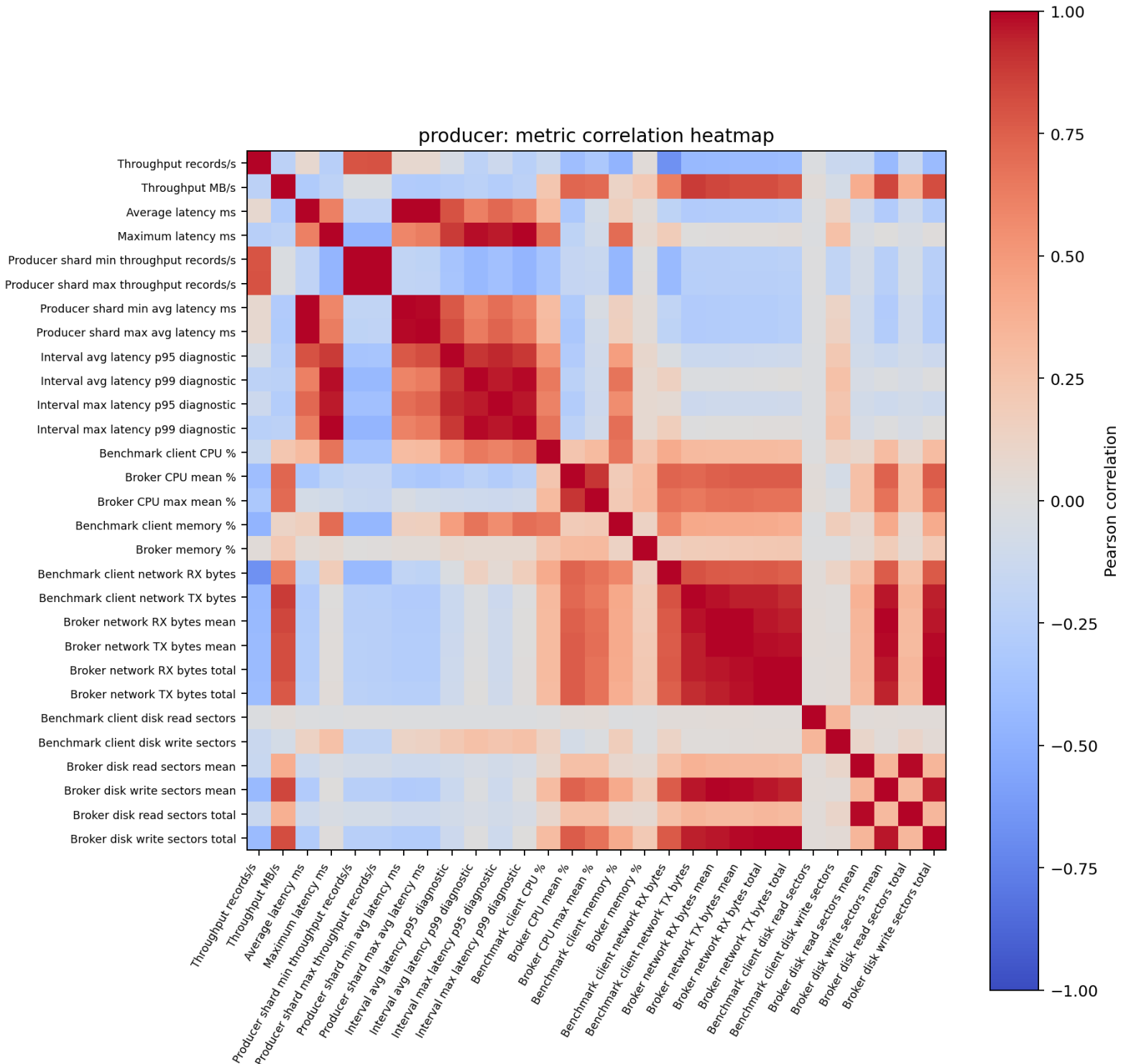


Figure 4.8: Correlation heatmap across producer metrics and host telemetry indicators. Red shows positive correlation, blue shows negative correlation, and stronger colour indicates a stronger relationship. The heatmap shows that throughput, latency, CPU, network, and disk metrics form distinct clusters rather than varying independently.

The telemetry links the security-mode comparison to observable resource behaviour, while the statistical summaries prevent weak telemetry differences from being overinterpreted.

4.7 Consumer Validation Results

The consumer-side benchmark was a targeted validation slice rather than a full factorial campaign. Its purpose was to test whether transport security also affects Kafka’s read path under selected workloads.

The consumer analysis is therefore supporting evidence rather than the main experimental contribution. The producer campaign remains central because the dissertation focuses primarily on Kafka’s high-throughput write path.

Table 4.6 reports the main consumer comparison results, while Figure 4.9 shows plaintext, TLS, and mTLS under the selected validation workloads.

The results indicate that transport-security overhead was also visible on the read path. TLS reduced mean consumer throughput by 38.70% relative to plaintext and mTLS by 31.81%, while mean fetch-time penalties reached 79.58% for TLS and 69.10% for mTLS.

This does not make the consumer slice equivalent to the producer campaign. The consumer result set is smaller and covers fewer workload combinations. Its role is to check whether the producer-side conclusion is isolated to writes. The read-path results suggest that it is not, because secure transport also reduced consumption throughput and increased fetch time under the selected validation workloads.

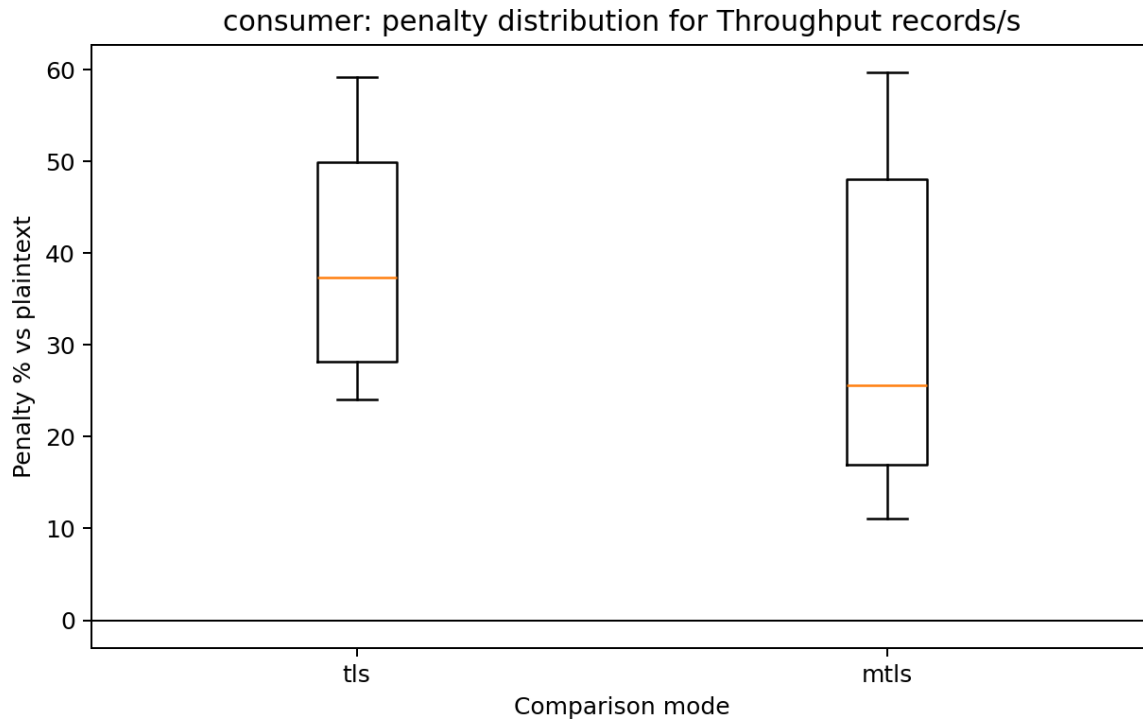
The bootstrap intervals support the same conclusion, although they are wider because the slice contains only 24 matched rows per secure mode. TLS consumer throughput penalty had a mean 95% interval of 34.18–43.40%, while mTLS had 25.37–38.46%. Fetch-time intervals were 60.22–101.12% for TLS and 46.04–93.70% for mTLS. Every matched consumer row had lower throughput and higher fetch time under TLS and mTLS.

Consumer telemetry should be interpreted carefully for the same reason as producer telemetry. Mean client and broker CPU were lower under TLS and mTLS than plaintext, so CPU should be read alongside achieved throughput and fetch time rather than as a direct standalone measure of cryptographic cost.

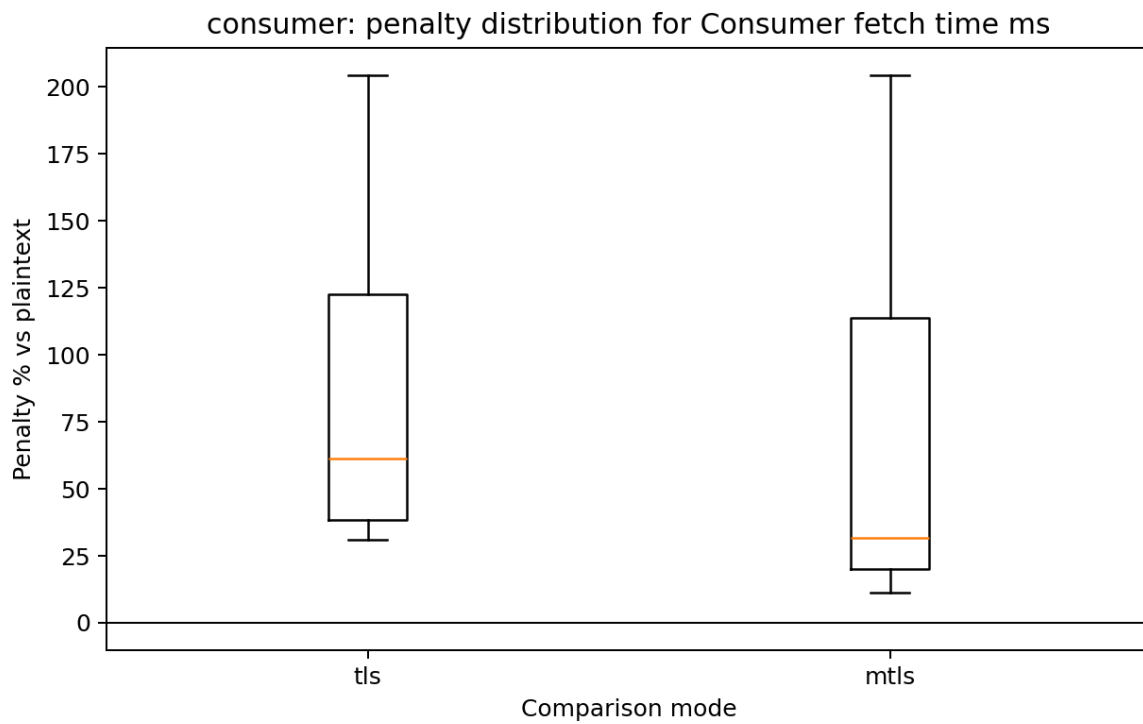
This section therefore provides secondary validation that secure-transport overhead is also visible when data is consumed.

Consumer metric	Plaintext	TLS	mTLS
Matched rows	24	24	24
Mean throughput (records/s)	28,734.78	14,819.16	15,488.13
Median throughput (records/s)	22,860.97	13,205.10	12,606.30
Mean throughput penalty	–	38.70%	31.81%
Median throughput penalty	–	37.36%	25.55%
Mean fetch time (ms)	14,237.75	20,292.42	17,472.92
Median fetch time (ms)	13,445.00	19,003.00	16,895.00
Mean fetch-time penalty	–	79.58%	69.10%
Median fetch-time penalty	–	61.31%	31.55%
Mean client CPU (%)	50.21	48.04	40.02
Mean broker CPU (%)	25.40	22.34	15.14

Table 4.6: Targeted consumer validation summary. Penalty values compare TLS and mTLS against equivalent plaintext consumer runs. Positive penalty means lower throughput or higher fetch time than plaintext; negative CPU penalty means lower CPU utilisation than plaintext.



(a) Throughput penalty



(b) Fetch-time penalty

Figure 4.9: Matched consumer validation overhead relative to plaintext. Positive throughput penalty means lower records/s than plaintext; positive fetch-time penalty means slower consumption. Both TLS and mTLS show read-path overhead in the targeted consumer slice.

4.8 Discussion

The first major result is that TLS and mTLS imposed measurable overhead, but the cost appeared more strongly in latency than throughput. Producer throughput fell by around 10% on average, while average-latency penalties exceeded 950% and maximum-latency penalties were around 79–92%. Bootstrap intervals show that these penalties are stable rather than explained only by extreme rows.

Plaintext did not dominate every matched row, but the aggregate results show a clear secure-transport cost. This makes generic claims such as “TLS overhead is small” or “mTLS overhead is always worse” incomplete without workload context.

The second major result is that overhead was workload-dependent. Producer count and target throughput amplified throughput penalties, while batch size and producer count were the strongest average-latency amplifiers. Larger records did not automatically produce larger throughput penalties.

A third result is that mTLS was not consistently worse than TLS. mTLS had a slightly lower mean producer-throughput penalty than TLS, but worse maximum latency. The difference between TLS and mTLS therefore depends on the metric and workload condition.

The telemetry results qualify the application-level findings. Secure transport did not simply increase CPU utilisation across all hosts. Client CPU effects were weak, while broker CPU was lower under TLS and mTLS. This suggests that lower achieved throughput, pacing, batching, and secure-transport processing interacted to produce the observed profile.

The findings also refine the literature discussed in Chapter 2. Earlier TLS studies show that encrypted transport can create major overhead when connection setup and asymmetric cryptography dominate [18]. Persistent-connection studies show that steady-state overhead can be smaller once handshakes are amortised [19]. The producer throughput results here are closer to the persistent-connection interpretation because Kafka uses long-lived client-broker connections. However, the latency results show that Kafka can still experience large penalties because batching, producer concurrency, broker coordination, and zero-copy limitations interact with the encrypted data path [6, 16].

The statistical summaries strengthen this comparison with the literature. The producer throughput penalty was moderate and tightly bounded, which supports the argument that persistent Kafka connections avoid the most severe TLS setup costs. At the same time, the latency penalties were large and widespread across matched rows, which shows that low connection-setup overhead does not imply low application-level delay. This is a Kafka-specific result because it depends on Kafka’s batching, partitioned writes, producer concurrency, and broker-mediated data path.

The consumer validation slice provides smaller but useful read-path evidence. TLS and mTLS reduced consumer throughput and increased fetch time, but the slice should be treated as supporting evidence because it is much smaller than the producer campaign.

These findings also clarify the dissertation’s position on Zero Trust. The results do not evaluate a complete Zero Trust Architecture. They evaluate TLS and mTLS as Zero Trust-aligned transport-security mechanisms, showing how selected Kafka transport controls affect throughput, latency, and resource utilisation under controlled conditions.

This distinction is important for interpretation. The dissertation does not claim that Zero Trust has one measurable Kafka performance cost. It shows that two transport mechanisms commonly aligned with Zero Trust principles introduce measurable, workload-dependent overhead. Broader Zero Trust mechanisms, such as external policy decisions, identity lifecycle management, and fine-grained authorisation, would require separate measurement.

The main implication is that secure Kafka deployment involves measurable trade-offs. TLS and mTLS improve confidentiality and identity assurance, but may reduce throughput, increase latency, and change the observed resource profile. Whether that trade-off is acceptable depends on workload shape and operational priorities.

For operators, the results suggest that enabling TLS or mTLS should be evaluated under the expected workload rather than under a single default benchmark. Producer count, target throughput, and batch size were more informative than security mode alone when explaining where overhead became large. This means capacity planning should include secure-mode benchmarks for representative message sizes, producer concurrency, compression settings, and durability requirements.

The dissertation therefore contributes both empirical evidence and practical interpretation. It quantifies Kafka transport-security overhead under matched workloads, reports uncertainty around the main estimates, and shows which workload factors most affect the cost of TLS or mTLS.

4.9 Chapter Summary

This chapter presented the benchmark results for plaintext, TLS, and mTLS Kafka deployments under matched workload and deployment configurations. It established result-set validity and then evaluated producer throughput, producer latency, host telemetry, and targeted consumer validation.

The results show that Kafka transport-security overhead is not fixed. It changes

across message size, target throughput, producer concurrency, batching, compression, and durability-related settings. Bootstrap intervals and matched-pair summaries show that the main throughput and latency penalties are stable at campaign level, while factor sensitivity identifies the workload variables that most strongly explain overhead variation.

Host telemetry further shows that resource behaviour does not map directly onto throughput and latency penalties, making telemetry useful as interpretive context rather than standalone proof of cryptographic cost. Overall, the chapter shows that TLS and mTLS affect Kafka performance under controlled high-throughput conditions, but that the magnitude and location of overhead depend on workload and deployment context.

Chapter 5

Evaluation, Conclusions, and Further Work

5.1 Chapter Overview

This chapter evaluates the dissertation against its objectives, summarises the main findings, discusses limitations, and identifies further work. Rather than repeating Chapter 4, it draws together the empirical and methodological contributions of the dissertation and directly answers the research question.

5.2 Evaluation Against Objectives

The dissertation aimed to design, implement, and evaluate a reproducible benchmarking artefact for measuring the performance overhead of Zero Trust-aligned transport-security mechanisms in Apache Kafka. Table 5.1 summarises the objective-level evaluation.

	Objective	Status	Evidence
1	Analyse Kafka mechanisms relevant to throughput, latency, and transport-security overhead.	Achieved	Chapter 2 reviewed batching, replication, compression, page cache, zero-copy transfer, and client/broker configuration.
2	Design and implement a cloud-based Kafka benchmarking artefact.	Achieved	Chapter 3 described the Terraform AWS deployment, Kafka KRaft cluster, benchmark client, orchestration pipeline, and result processing.
3	Support plaintext, TLS, and mTLS Kafka deployments under controlled conditions.	Achieved	The artefact deployed matched security modes and validated connectivity through smoke testing and benchmark execution.
4	Execute matched producer workloads across workload and deployment parameters.	Achieved	The factorial producer campaign was generated from JSON configuration and executed through checkpointed orchestration.
5	Capture throughput and latency metrics for security-overhead analysis.	Achieved	Producer and consumer benchmark outputs were parsed into structured throughput and latency results.
6	Capture host-level telemetry from benchmark client and broker hosts.	Achieved	CPU, memory, network, and disk telemetry were collected and linked to benchmark rows.
7	Evaluate how security overhead changes across Kafka parameters.	Achieved	Chapter 4 used matched comparisons, bootstrap intervals, matched-pair summaries, and factor-sensitivity analysis.
8	Include consumer-side validation rather than only the producer path.	Achieved	The targeted consumer validation slice provided supporting read-path evidence.

Table 5.1: Evaluation of dissertation objectives

Overall, the objectives were achieved within the selected Kafka deployment model, workload design, and cloud environment. The dissertation produced more than a benchmark runner. It produced a controlled experimental framework with explicit configuration, resumable execution, structured result capture, telemetry collection, matched security-mode comparison, and statistical post-processing.

5.3 Summary of Findings

The main finding is that Kafka transport-security overhead is measurable but not fixed. In the matched producer campaign, TLS had a mean throughput penalty of 10.83%, while mTLS had 9.57%. Bootstrap intervals supported the stability of these estimates: 10.17–11.49% for TLS and 8.90–10.23% for mTLS. This shows that secure transport introduced measurable overhead, but also that mTLS was not automatically worse than TLS across the completed benchmark space.

Latency was affected more sharply than throughput. Mean average-latency penalties exceeded 950% for both TLS and mTLS, with bootstrap intervals of 901.37–1004.04% for TLS and 905.55–1010.04% for mTLS. Maximum-latency penalties were also substantial. Matched-pair summaries showed that secure transport increased average latency in more than 99% of matched producer rows. Throughput alone would therefore understate the performance trade-off.

The results also show that overhead is workload-dependent. Producer count and target throughput made throughput penalties more visible, while batch size and producer count were the strongest latency amplifiers. Larger records did not produce a simple proportional increase in throughput penalty. These interactions support the central claim that Kafka security overhead should be interpreted in relation to workload shape rather than as a single constant percentage.

The telemetry results qualify the interpretation further. Secure modes reduced throughput and increased latency, but this did not appear as a simple increase in broker CPU. Benchmark-client CPU effects were weak, while broker CPU was generally lower under TLS and mTLS than plaintext. The resource profile therefore reflects achieved throughput, pacing, cryptographic work, and workload interaction rather than a simple CPU-saturation pattern.

The targeted consumer validation slice showed that the read path was also affected. TLS reduced mean consumer throughput by 38.70%, while mTLS reduced it by 31.81%. Fetch-time penalties were also substantial. Since the consumer slice was smaller than the producer campaign, this result should be treated as supporting evidence, but it strengthens the conclusion that transport-security overhead is not limited to Kafka’s write path.

5.4 Methodological Contribution

The dissertation makes a methodological contribution by providing a reproducible Kafka benchmarking artefact designed specifically for matched transport-security evaluation. This matters because the literature review identified weaknesses in Kafka benchmarking

practice, including incomplete configuration reporting, limited metric granularity, and weak reproducibility.

The artefact addresses these weaknesses through infrastructure-as-code, configuration-driven execution, checkpointed campaigns, deterministic run identifiers, structured per-run outputs, host telemetry, traceable aggregation, matched comparison files, bootstrap confidence intervals, matched-pair summaries, and factor-sensitivity outputs. Its value therefore lies not only in the final measurements, but in the method used to produce, inspect, repeat, and qualify those measurements.

5.5 Limitations

Several limitations affect the scope and interpretation of the results.

First, the experiments were conducted on AWS EC2 rather than on dedicated hardware. This made the artefact practical, repeatable, and realistic for cloud deployment, but introduced possible noisy-neighbour effects, instance-level variation, and shared-network variability. The results should therefore be interpreted as controlled cloud measurements rather than hardware-invariant constants.

Second, the statistical analysis improves confidence in the main overhead estimates, but it does not remove the wider limits of the experiment. Bootstrap intervals quantify uncertainty within the completed cloud dataset. They do not prove that the same percentages will hold across all hardware, Kafka versions, JVM settings, network conditions, or production deployments.

Third, the dissertation evaluates selected Kafka transport-security modes rather than every possible secure Kafka deployment or every aspect of Zero Trust Architecture. Plaintext, TLS, and mTLS provide a clear measurable comparison, but dynamic policy enforcement, identity lifecycle management, external policy engines, and broader access-control architectures remain outside the scope.

Fourth, the workload and metric scope are selective rather than exhaustive. The producer-side factorial campaign covers the parameters most likely to affect Kafka throughput, latency, and transport-security cost, but it does not explore every Kafka setting, JVM option, operating-system tuning choice, or network condition. Telemetry supports host-level interpretation, but does not provide full per-record latency traces or deep JVM-internal attribution for every run.

Finally, the consumer validation slice is intentionally smaller than the producer campaign. It provides useful read-path evidence, but should not be treated as equivalent to the full

producer-side factorial study.

5.6 Further Work

Future work could extend the benchmark space to include additional Kafka parameters, JVM tuning options, storage configurations, constrained network conditions, larger broker clusters, and longer-running stability tests. This would test whether the patterns observed in this dissertation remain stable under broader deployment conditions.

A second direction is deeper protocol and observability analysis. TLS 1.2 and TLS 1.3 could be isolated, cipher-suite choices could be compared, and JVM-level, broker-internal, or kernel-level metrics could be added to improve attribution of cryptographic overhead.

A third direction is to evaluate wider Zero Trust-inspired mechanisms. This could include SASL authentication modes, OAuth-based identity integration, ACL policy complexity, certificate rotation, workload identity systems, external policy decision points, and combinations of authentication, authorisation, and encryption.

A more ambitious systems direction is to investigate whether Kafka can preserve more of its efficient data path under secure transport. This could include kernel TLS, hardware cryptographic offload, or Kafka/JDK transport designs that reduce the need to move data through user space for encryption. Such work would address one of the core tensions identified in this dissertation: TLS and mTLS improve confidentiality and identity assurance, but can disrupt the zero-copy data movement that helps Kafka achieve high throughput.

Finally, the artefact could be extended for wider reuse through richer automation, reusable dashboards, stronger statistical post-processing, and comparison with other distributed streaming or messaging systems.

5.7 Answer to the Research Question

The research question asked how TLS and mTLS affect Apache Kafka throughput, latency, and host-resource behaviour under matched high-throughput workload and deployment conditions.

The dissertation shows that TLS and mTLS introduce measurable Kafka performance overhead, but that the overhead is not a fixed security tax. In the matched producer campaign, secure transport reduced mean throughput by around 10% relative to plaintext, while average-latency penalties were much larger and appeared in almost all matched producer rows. Producer concurrency, target throughput, and batch size made the overhead more visible, whereas larger records did not produce a simple proportional increase in

throughput penalty.

The answer is therefore both empirical and methodological. Empirically, TLS and mTLS affected throughput, latency, and resource behaviour under matched Kafka workloads. Methodologically, the dissertation shows that this overhead is best measured through reproducible matched benchmarking with uncertainty estimates and workload-sensitivity analysis, rather than inferred from generic TLS studies or unmatched Kafka runs.

5.8 Chapter Summary

This chapter evaluated the dissertation objectives, summarised the main findings, discussed limitations, and identified further work. The dissertation achieved its central aim of designing and evaluating a reproducible Kafka benchmarking artefact for measuring TLS and mTLS overhead under controlled conditions.

Overall, the work contributes a configuration-explicit, resumable, telemetry-enabled, and statistically supported framework for Kafka transport-security evaluation. It also shows that secure Kafka deployment involves measurable and configuration-dependent trade-offs best understood through controlled empirical benchmarking.

Bibliography

- [1] A. Kumar, “Distributed systems in modern enterprise architecture: Challenges and solutions,” *IJSAT*, vol. 16, no. 1, p. 2447, 2025.
- [2] “Powered by,” <https://kafka.apache.org/powered-by/>, accessed: 2026-03-24.
- [3] “Introduction,” <https://kafka.apache.org/42/getting-started/introduction/>, accessed: 2026-03-24.
- [4] T. P. Raptis and A. Passarella, “A survey on networked data streaming with apache kafka,” *IEEE Access*, vol. 11, pp. 85 333–85 350, 2023.
- [5] A. Sundbom, “Parallel kafka producer applications: Their performance and its limitations,” Master’s thesis, KTH Royal Institute of Technology, 2023.
- [6] “Design,” <https://kafka.apache.org/42/design/design/>, accessed: 2026-03-24.
- [7] “Producer configs,” <https://kafka.apache.org/42/configuration/producer-configs/>, accessed: 2026-03-26.
- [8] H. Kang, G. Liu, Q. Wang, L. Meng, and J. Liu, “Theory and application of zero trust security: A brief survey,” *Entropy*, vol. 25, no. 12, p. 1595, 2023.
- [9] N. F. Syed, S. W. Shah, A. Shaghaghi, A. Anwar, Z. Baig, and R. Doss, “Zero trust architecture (zta): A comprehensive survey,” *IEEE Access*, vol. 10, pp. 57 143–57 179, 2022.
- [10] V. A. Vora, “Demystifying zero trust security: The no-trust network paradigm,” *Journal of Computer Science and Technology Studies*, vol. 7, no. 3, pp. 141–148, 2025.
- [11] M. Tsai, S. Lee, and S. W. Shieh, “Strategy for implementing of zero trust architecture,” *IEEE Transactions on Reliability*, vol. 73, no. 1, pp. 93–100, 2024.
- [12] H. Bhoite, “Zero-trust architecture in streaming dataflows,” <https://www.authorea.com/doi/full/10.36227/techrxiv.175303721.12297807>, accessed: 2026-03-26.
- [13] “Broker configs,” <https://kafka.apache.org/42/configuration/broker-configs/>, accessed: 2026-03-26.

- [14] “Consumer configs,” <https://kafka.apache.org/42/configuration/consumer-configs/>, accessed: 2026-03-26.
- [15] J. Meka, “Financial services cloud transformation: Securing sensitive data in kafka event streams,” *Journal of Computer Science and Technology Studies*, vol. 7, no. 4, pp. 1023–1028, 2025.
- [16] M. Mohammad, “Analysis of design patterns and benchmark practices in apache kafka event-streaming systems,” in *2025 5th International Conference on Intelligent Cybernetics Technology & Applications (ICICyTA)*, 2025, pp. 612–617.
- [17] A. A. Süzen and O. Ceylan, “The advantages and implementation challenges within the scope of the basic principles of transition to zero trust architecture,” *International Journal of 3D Printing Technologies and Digital Industry*, vol. 8, no. 3, pp. 416–427, 2024.
- [18] C. Coarfa, P. Druschel, and D. S. Wallach, “Performance analysis of tls web servers,” *ACM Transactions on Computer Systems*, vol. 24, no. 1, pp. 39–69, 2006.
- [19] M. Iorio, F. Risso, and C. Casetti, “When latency matters: Measurements and lessons learned,” *SIGCOMM Computer Communication Review*, vol. 51, no. 4, pp. 2–13, 2021.
- [20] C. H. Jack, S. K. Teck, L. T. Ming, and D. Y. Hong, “An overview analysis of authentication mechanism in microservices-based software architecture: A discussion paper,” in *2023 IEEE 8th International Conference on Software Engineering and Computer Systems (ICSECS)*, 2023, pp. 1–6.
- [21] S. A. Jyothi, A. Singla, P. B. Godfrey, and A. Kolla, “Measuring and understanding throughput of network topologies,” in *SC '16: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2016, pp. 761–772.

Appendices

Appendix A

Ethics and GenAI Declaration

A.1 Ethics Statement

This dissertation uses a computer-based experimental methodology. The artefact provisions Apache Kafka clusters on AWS EC2, executes synthetic benchmark workloads, and records application-level performance metrics and host-level telemetry. The project does not involve human participants, user studies, interviews, surveys, behavioural observation, or the collection of personal data.

The benchmark data used in this dissertation is generated synthetically by Kafka producer and consumer tools. The recorded data consists of technical machine metrics, including throughput, latency, CPU utilisation, memory utilisation, disk activity, network activity, workload metadata, and benchmark execution status. No personally identifiable information is collected, processed, or analysed.

The project therefore does not require ethical approval for human-participant research. The main ethical and professional considerations relate to responsible cloud-resource usage, secure handling of credentials, and accurate reporting of experimental limitations. AWS credentials, SSH keys, certificates, and generated security material are excluded from version control where appropriate.

A.2 Generative AI Declaration

A.2.1 Acknowledge

Generative AI tools were used as support tools during this project. Their use included assistance with drafting and refining dissertation wording, reviewing structure against university guidance, improving clarity of technical explanations, and supporting software development planning. Generative AI was also used to create the initial Apache Kafka architecture diagram used in Chapter 2.

The project design, artefact implementation, benchmark execution, result interpretation, and final academic judgement remain the author’s own work. AI-generated material was reviewed, edited, and adapted before inclusion.

A.2.2 Describe

Generative AI was used mainly for support and review rather than as a source of evidence. Its use included:

- suggesting clearer wording for dissertation sections;
- helping restructure paragraphs and chapter sections;
- reviewing whether sections aligned with the university marking criteria;
- helping explain technical trade-offs such as virtual machines versus containers;
- assisting with implementation planning and debugging strategy for the artefact;
- generating an initial Apache Kafka architecture diagram, which was then reviewed for relevance to the dissertation.

Generative AI was not used as an independent factual source. Technical claims were checked against cited literature, official Apache Kafka documentation, the artefact source code, and benchmark results. The author remains responsible for the accuracy, originality, and final form of all submitted work.

A.2.3 Evidence

Representative examples of GenAI use are summarised below.

Use case	Example prompt summary	How the output was used
Dissertation structure review	Asked for Chapter 1 and Chapter 2 to be reviewed against university marking criteria.	Used to identify missing structure, such as explicit aims, objectives, and clearer research gap framing.
Chapter wording support	Asked for sections to be rewritten in a clearer academic style while preserving the author’s tone.	Outputs were edited and adapted before inclusion.
Artefact explanation	Asked for help explaining the Kafka benchmarking artefact, including infrastructure, security modes, telemetry, and result processing.	Used as drafting support for Chapter 3, checked against the implemented repository.
Technical planning	Asked for advice on benchmark design, fault tolerance, idempotency, and telemetry collection.	Used to support development planning; final implementation was tested directly.
Figure generation	Prompt requested a simplified Apache Kafka architecture diagram showing producers, brokers, topics, and consumers.	Output was used as Figure 2.1 and disclosed in the figure caption.

Table A.1: Representative examples of GenAI use during the project

A.2.4 Declaration

The author confirms that no content created by generative AI technologies has been submitted without review, adaptation, and acknowledgement. The author remains responsible for the dissertation argument, artefact implementation, experimental execution, result interpretation, and final submitted content.

Appendix B

Artefact Configuration

This appendix summarises the configuration structure used by the benchmarking artefact. The artefact is driven by explicit configuration files rather than hard-coded experiment settings. This was done to make the benchmark design repeatable, transparent, and easier to compare across plaintext, TLS, and mTLS modes.

The main configuration files are stored under `config/`. Baseline files define fixed workload defaults, while factorial files define the larger parameter spaces used for the final evaluation. Generated execution plans are written as JSONL files under `.orchestration/`, where each line represents one benchmark row. This separates the experiment design from the execution logic and allows the same plan structure to be resumed after interruption.

B.1 Configuration Files

The artefact uses three main categories of configuration file. Baseline configuration files describe default workload settings. Sweep configuration files describe one-factor-at-a-time exploratory experiments. Factorial configuration files describe the final multi-parameter experiment spaces used for the security-overhead evaluation.

The final producer campaign is defined by `config/factorials/security-overhead-final.json`. The targeted consumer validation study is defined by `config/factorials/consumer-security-slice.json`. These files are converted into JSONL execution plans before running. The generated producer plan is stored as `.orchestration/security-overhead-final-plan.jsonl`, while the generated consumer plan is stored as `.orchestration/consumer-security-slice-plan.jsonl`.

Temporary deployment metadata, generated plans, checkpoints, inventory files, and Terraform outputs are also stored under `.orchestration/`. This directory is not part of the permanent artefact configuration because it contains environment-specific state, including generated cluster metadata and credentials. It is therefore excluded from version control

where appropriate.

The main configuration files used by the artefact are listed in Table B.1. The long paths are shown explicitly because they form part of the reproducibility trail for the artefact.

Configuration path	Purpose
<code>config/baselines/plaintext-default.json</code>	Defines the original plaintext baseline workload settings.
<code>config/baselines/plaintext-requested-baseline.json</code>	Defines the requested plaintext baseline used during expanded benchmark development.
<code>config/factorials/security-overhead-final.json</code>	Defines the final producer-side factorial campaign across plaintext, TLS, and mTLS modes.
<code>config/factorials/consumer-security-slice.json</code>	Defines the targeted consumer-side validation slice.
<code>.orchestration/security-overhead-final-plan.jsonl</code>	Generated JSONL execution plan for the final producer campaign.
<code>.orchestration/consumer-security-slice-plan.jsonl</code>	Generated JSONL execution plan for the consumer validation slice.
<code>.orchestration/security-overhead-final-mtls-broker5.checkpoint</code>	Example checkpoint file used to resume the mTLS five-broker producer campaign.
<code>.orchestration/consumer-security-slice-mtls-broker5.checkpoint</code>	Example checkpoint file used to resume the mTLS five-broker consumer slice.

Table B.1: Main artefact configuration and generated plan files

B.2 Producer Factorial Campaign

The main evaluation is based on a producer-side factorial campaign. This campaign is used because the research question requires more than a single plaintext, TLS, and mTLS comparison. Kafka performance depends on workload and deployment configuration, so the security overhead must be measured across a controlled range of Kafka-relevant parameters.

The final producer campaign design contains 5,184 planned rows after validity constraints are applied. These rows are divided evenly across the three security modes, with 1,728 planned rows each for plaintext, TLS, and mTLS. The same workload dimensions are used across the security modes so that comparisons can be made under matched conditions.

The campaign varies broker count, replication factor, minimum in-sync replicas, message size, target throughput, batch size, acknowledgement setting, producer count, compression type, and trial index. These variables were selected because they directly influence Kafka’s throughput-latency-durability trade-off and therefore may affect how visible TLS or mTLS overhead becomes.

The generated producer plan is stored as `.orchestration/security-overhead-final-plan.jsonl`. Each line in this file represents one planned benchmark row. The executor reads this plan, applies runtime filters such as security mode and broker count, skips rows already recorded in the checkpoint, and writes completed rows into the relevant result set.

Dimension	Role in the experiment
Security mode	Compares plaintext, TLS, and mTLS transport configurations.
Broker count	Tests whether cluster size changes the observed overhead.
Replication factor	Controls durability and replication cost.
Minimum in-sync replicas	Controls how many replicas must acknowledge writes.
Message size	Tests whether larger payloads amplify encryption and data-movement costs.
Target throughput	Controls the offered producer load.
Batch size	Controls producer batching and amortisation of network overhead.
Acknowledgement mode	Changes the durability and latency behaviour of producer writes.
Producer count	Controls producer-side concurrency.
Compression type	Tests whether compression interacts with secure transport overhead.
Trial index	Provides repeated measurements for the same configuration.

Table B.2: Producer factorial campaign dimensions

B.3 Security Mode Configuration

The artefact uses three security modes. Plaintext provides the baseline with no transport encryption. TLS enables encrypted transport with server authentication. mTLS extends TLS by requiring certificate-based client authentication as well as encrypted communication.

These modes do not represent a complete Zero Trust Architecture. Instead, TLS and mTLS are evaluated as Zero Trust-aligned transport-security mechanisms because they provide encrypted communication and certificate-based identity verification. This distinction is important because the dissertation evaluates measurable Kafka transport controls rather than claiming to implement a full system-level Zero Trust deployment.

The TLS and mTLS configurations permit TLS 1.3 and TLS 1.2, with protocol negotiation handled by the Kafka and JDK SSL stack. This reflects the implemented artefact configuration and avoids assuming that all benchmarked connections use TLS 1.3 only.

Mode	Description
Plaintext	Baseline Kafka transport with no TLS encryption.
TLS	Encrypted Kafka transport with server-side certificate authentication.
mTLS	Encrypted Kafka transport with mutual certificate authentication between clients and brokers.

Table B.3: Security modes used by the artefact

B.4 Validity Constraints

Not every possible parameter combination is valid in Kafka. The plan generation process therefore applies validity constraints before benchmark execution. For example, the replication factor cannot exceed the number of available brokers, and the minimum in-sync replica setting must be valid for the selected replication factor.

These constraints are applied before the generated JSONL plan is executed. This prevents invalid benchmark rows from being submitted to the cluster and ensures that the final plan represents executable Kafka configurations rather than a purely theoretical parameter grid.

The use of deterministic run identifiers also supports traceability. Each run identifier is derived from the workload configuration, including security mode, broker count, replication factor, minimum in-sync replicas, message size, target throughput, batch size, acknowledgement mode, producer count, compression type, and trial index. This makes it possible to link each plan row to its result directory and summary output.

For example, a generated run directory may include a full configuration-derived identifier such as `security-overhead-final-factorial-mtls-b5-rf5-isr3-msg102400-tps10000-batch131072-acks1-prod12-complz4-trial3`. This long identifier is intentional because it makes the run self-describing and reduces ambiguity when inspecting result directories.

B.5 Consumer Validation Slice

The consumer-side benchmark is intentionally designed as a targeted validation slice rather than a full factorial campaign. A full consumer factorial campaign would greatly increase the experimental cost and runtime, while duplicating much of the producer-side parameter space. The purpose of the consumer slice is therefore narrower: to test whether secure transport also affects Kafka’s read path under selected workloads.

The consumer configuration is defined by `config/factorials/consumer-security-slice.json`. The generated consumer execution plan is stored as `.orchestration/consumer-`

`security-slice-plan.jsonl`. This plan includes plaintext, TLS, and mTLS modes, selected message sizes, compression settings, consumer concurrency, and repeated trials.

The consumer validation results are therefore used as supporting evidence rather than as a second full factorial evaluation. This keeps the dissertation focused on the main producer-side security-overhead campaign while still addressing the possibility that TLS and mTLS affect consumers as well as producers and brokers.

Appendix C

Result Schema

This appendix summarises the structure of the benchmark outputs produced by the artefact. Each benchmark run produces a run directory containing raw benchmark logs, workload metadata, parsed metrics, and host telemetry. The exact metric values differ between runs, but the output structure remains stable across plaintext, TLS, and mTLS modes.

The result schema was designed to support traceability from experiment plan to dissertation output. Each run can be linked back to its configuration row, security mode, workload parameters, raw Kafka benchmark output, parsed result file, host telemetry files, and aggregated summary row. This makes the result pipeline reproducible and reduces the risk of interpreting a metric without its workload context.

C.1 Run-Level Output

Each completed benchmark run is stored in a dedicated result directory. The directory name is derived from the run identifier, which encodes the main workload parameters. This makes the result directory self-describing and allows individual runs to be inspected without relying only on the aggregated CSV output.

The most important run-level files are `metadata.json`, `result.json`, the raw Kafka benchmark logs, and the `host-telemetry/` directory. The metadata file records the configuration used for the run, while the result file stores parsed benchmark metrics and run-level telemetry summaries.

File or directory	Description
<code>metadata.json</code>	Stores workload, deployment, and security-mode configuration for the run.
<code>result.json</code>	Stores parsed application metrics and aggregated telemetry fields for the run.
<code>producer-perf.log</code>	Stores raw output from the Kafka producer performance benchmark.
<code>consumer-perf.log</code>	Stores raw output from the Kafka consumer performance benchmark, where consumer benchmarking is used.
<code>topic-create.log</code>	Stores topic creation output for the run.
<code>topic-delete.log</code>	Stores topic deletion output where per-run cleanup is used.
<code>host-telemetry/</code>	Stores per-host telemetry logs collected from the benchmark client and Kafka brokers.

Table C.1: Run-level benchmark output files

C.2 Metadata Fields

The `metadata.json` file records the workload and deployment context for a benchmark run. This includes the security mode, broker count, partition count, replication factor, minimum in-sync replicas, message size, record count, target throughput, producer count, consumer count, batch size, acknowledgement setting, compression type, and trial index.

These fields are important because throughput and latency are only meaningful when interpreted alongside the workload that produced them. For example, a latency value from a plaintext run with one producer and small messages is not directly comparable with a mTLS run using twelve producers, larger messages, and stricter durability settings unless the configuration is visible.

The metadata fields also support matched comparison across security modes. Plaintext, TLS, and mTLS rows can be grouped using the same workload parameters, allowing the evaluation to isolate transport-security overhead from unrelated workload differences.

Metadata field group	Examples
Security configuration	<code>security_mode</code> , client configuration path, listener protocol.
Cluster configuration	<code>broker_count</code> , <code>partition_count</code> , <code>replication_factor</code> , <code>min_insync_replicas</code> .
Producer workload	<code>message_size_bytes</code> , <code>num_records</code> , <code>target_messages_per_second</code> , <code>producer_count</code> .
Producer tuning	<code>batch_size</code> , <code>linger_ms</code> , <code>acks</code> , <code>compression_type</code> .
Consumer workload	<code>consumer_count</code> and consumer benchmark settings where applicable.
Run identity	<code>run_id</code> , <code>trial_index</code> , <code>trial_count</code> , and result directory.

Table C.2: Metadata field groups

C.3 Parsed Result Fields

The `result.json` file stores parsed metrics extracted from the Kafka benchmark output. For producer runs, this includes throughput in records per second, throughput in MB per second, average latency, maximum latency, and other latency fields reported by the Kafka tool where available. For consumer runs, the parsed output records consumer throughput and read-path metrics where available.

The result file also stores workload metadata and telemetry summaries so that each run remains interpretable even outside the aggregated result set. This is useful when inspecting individual heavy runs, failed runs, or outliers in the final analysis.

The primary application-level metrics used in the dissertation are throughput in records per second, throughput in MB per second, average latency, and maximum latency. These metrics are used because they are directly reported by the Kafka benchmark tools and provide a consistent basis for comparison across plaintext, TLS, and mTLS runs.

Metric	Description
Throughput records/sec	Number of records successfully sent or consumed per second.
Throughput MB/sec	Data throughput converted into megabytes per second.
Average latency	Mean producer latency reported by the Kafka producer benchmark.
Maximum latency	Maximum producer latency reported by the Kafka producer benchmark.
Consumer throughput	Consumer-side throughput for targeted read-path validation runs.
Failure status	Indicates failed or timed-out runs where applicable.

Table C.3: Primary parsed result metrics

C.4 Host Telemetry Fields

Host telemetry is stored under the `host-telemetry/` directory for each run. The artefact collects telemetry from the benchmark client and Kafka brokers during benchmark execution. These files are stored as JSONL logs, with each line representing one interval sample from a host.

The telemetry captures CPU utilisation, memory utilisation, disk activity, and network receive/transmit counters. These measurements are used to link application-level behaviour to host-level resource use. This is important for the dissertation because TLS and mTLS overhead may appear not only as lower throughput or higher latency, but also as increased CPU activity or different network and disk behaviour.

The number of telemetry hosts is also recorded. For a five-broker campaign, the expected telemetry host count is normally six: five Kafka brokers and one benchmark client. This helps identify whether a run has complete host-level coverage.

Telemetry field group	Description
CPU utilisation	Per-host CPU usage sampled during the benchmark run.
Memory utilisation	Per-host memory usage during the benchmark run.
Network activity	Network receive and transmit counters for each telemetry host.
Disk activity	Disk read and write activity, including sector counters where available.
Telemetry host count	Number of hosts with telemetry files for the run.
Telemetry role	Identifies whether the host is a broker or benchmark client.

Table C.4: Host telemetry field groups

C.5 Result-Set Output

At the result-set level, completed runs are aggregated into `summary.json` and `summary.csv`; later export stages derive matched comparison and statistical analysis files from these summaries.

The executor also writes `started.jsonl`, `completed.jsonl`, and `failures.jsonl` where applicable. These files record execution state. The `started.jsonl` file records rows that began execution, while `completed.jsonl` records rows that finished successfully and were copied back locally. The `failures.jsonl` file is only produced when failures occur.

This distinction matters because a started row is not necessarily a completed row. A run is treated as complete only when the benchmark has finished, the result files have been copied back, the raw output has been parsed, and the checkpoint has been updated.

Result-set file	Description
<code>started.jsonl</code>	Records benchmark rows that were started by the executor.
<code>completed.jsonl</code>	Records benchmark rows that completed successfully.
<code>failures.jsonl</code>	Records failed or timed-out rows when failures occur.
<code>summary.json</code>	Stores structured summary data across completed runs.
<code>summary.csv</code>	Stores flat tabular summary data for analysis and export.
<code>export/table.csv</code>	Dissertation-ready CSV table output where export is enabled.
<code>export/table.tex</code>	Dissertation-ready LaTeX table output where export is enabled.
<code>export/*.svg</code>	Plot outputs where export is enabled.

Table C.5: Result-set output files

C.6 Matched and Statistical Analysis Outputs

After result-set aggregation, the final export stage creates derived analysis files under `results/final-processed/comprehensive-result-pack/`. These files are not raw benchmark outputs. They are produced from completed and matched result rows so that plaintext, TLS, and mTLS can be compared under equivalent workload and deployment conditions.

The matched comparison files include producer and consumer wide-format CSVs, where each row contains equivalent plaintext, TLS, and mTLS measurements for the same workload configuration. The overhead-long CSVs then store penalty values relative to the matched plaintext baseline. These files provide the direct input for the Chapter 4 overhead analysis.

The statistical outputs are stored under `statistics/`. They include bootstrap confidence intervals, matched-pair effect summaries, and factor-sensitivity tables. The accompanying `statistics/manifest.json` records the exported files, bootstrap sample count, and random seed.

Derived output	Description
<code>data/producer_matched_wide_comparison.csv</code>	Matched producer rows with plaintext, TLS, and mTLS values side by side.
<code>data/consumer_matched_wide_comparison.csv</code>	Matched consumer validation rows with security-mode values side by side.
<code>data/producer_overhead_long.csv</code>	Producer penalty values relative to matched plaintext rows.
<code>data/consumer_overhead_long.csv</code>	Consumer penalty values relative to matched plaintext rows.
<code>statistics/csv/*.csv</code>	Bootstrap, matched-pair, and factor-sensitivity outputs in CSV form.
<code>statistics/latex/*.tex</code>	Dissertation-ready LaTeX versions of the statistical summary tables.
<code>statistics/manifest.json</code>	Records statistical export settings and generated output files.

Table C.6: Matched comparison and statistical analysis outputs

C.7 Metric Interpretation Caveat

The benchmark tools provide application-level throughput and latency metrics, while the telemetry pipeline provides interval-based host-resource measurements. These two sources are complementary but not identical. Application metrics describe benchmark behaviour, while telemetry fields describe resource behaviour sampled during the run.

Where interval-derived percentile fields are present in the summary output, they are

treated as diagnostic indicators rather than true per-record latency percentiles. They describe variation across sampled intervals or parsed benchmark summaries and are therefore used to support interpretation, not as a substitute for full per-record latency tracing.

For this reason, the dissertation focuses on metrics that are consistently available across runs: throughput records per second, throughput MB per second, average latency, maximum latency, CPU utilisation, memory utilisation, network activity, disk activity, telemetry host count, workload metadata, and failure status.

Appendix D

Additional Result Tables and Generated Artefacts

This appendix provides additional evidence for Chapter 4. It does not replace the main discussion. Instead, it records supporting completion counts, matched overhead summaries, workload-interaction tables, telemetry summaries, consumer validation results, and selected generated PNG figures.

Unless stated otherwise, producer tables are based on the matched producer comparison set of 1,728 rows per security mode. Positive penalty means worse than the matched plaintext baseline. For throughput, this means lower throughput. For latency and resource metrics, this means a higher value. Negative penalty therefore means that the secure mode recorded a lower value than plaintext for that metric.

D.1 Campaign Completion and Matched Retention

Table D.1 summarises the completed producer campaign. All planned producer rows were completed and retained for matched comparison across plaintext, TLS, and mTLS.

Security mode	Planned	Started	Completed	Failures	Matched
Plaintext	1,728	1,760	1,728	29	1,728
TLS	1,728	1,733	1,728	2	1,728
mTLS	1,728	1,731	1,728	0	1,728
Total	5,184	5,224	5,184	31	5,184

Table D.1: Producer campaign completion and matched-analysis counts

Security mode	Planned	Started	Completed	Failures	Matched
Plaintext	24	24	24	0	24
TLS	24	28	24	2	24
mTLS	24	24	24	0	24
Total	72	76	72	2	72

Table D.2: Consumer validation completion and matched-analysis counts

D.2 Matched Producer Performance Tables

Table D.3 gives the absolute producer means used alongside the relative overhead calculations. The median throughput is close to the configured 1,000 records/s target for many rows, while the mean captures the higher-throughput parts of the factorial design.

Metric	Plaintext	TLS	mTLS
Throughput records/s	2,848.49	2,336.51	2,356.73
Throughput MB/s	40.81	36.39	37.35
Average latency ms	537.41	1,861.07	1,837.07
Maximum latency ms	5,343.48	9,544.81	10,129.98
Benchmark-client CPU %	76.96	77.40	75.76
Broker CPU %	28.71	24.96	22.35

Table D.3: Matched producer absolute performance and CPU summary

Metric	TLS mean	TLS median	mTLS mean	mTLS median
Throughput records/s	10.83%	3.77%	9.57%	0.12%
Average latency ms	952.23%	530.30%	957.81%	576.31%
Maximum latency ms	79.30%	79.36%	91.80%	88.03%
Benchmark-client CPU %	2.49%	0.61%	-0.55%	-0.07%
Broker CPU %	-17.06%	-21.06%	-27.42%	-31.87%

Table D.4: Matched producer relative overhead summary against plaintext

Metric	TLS min	TLS max	mTLS min	mTLS max
Throughput records/s	-24.07%	51.35%	-23.65%	53.98%
Average latency ms	-20.87%	6,954.15%	-21.76%	8,223.00%
Maximum latency ms	-78.48%	189.76%	-72.24%	249.77%
Benchmark-client CPU %	-98.51%	4,603.50%	-95.71%	4,714.11%
Broker CPU %	-83.52%	66.45%	-86.08%	63.66%

Table D.5: Matched producer overhead range across the factorial campaign

D.3 Producer Workload Interaction Tables

The following tables show selected workload interactions behind the Chapter 4 figures. They make the plotted patterns easier to check without moving every generated plot into the main chapter.

Record size	TLS mean	TLS med.	mTLS mean	mTLS med.	Higher
1024	11.95%	0.01%	12.08%	0.01%	mTLS
10240	12.13%	0.14%	11.07%	0.03%	TLS
102400	8.41%	8.47%	5.55%	3.79%	TLS

Table D.6: Matched producer throughput penalty by record size

Producer count	TLS mean	TLS med.	mTLS mean	mTLS med.	Higher
1	2.71%	0.00%	1.55%	0.00%	TLS
6	12.77%	8.02%	11.10%	2.60%	TLS
12	17.02%	12.76%	16.05%	8.95%	TLS

Table D.7: Matched producer throughput penalty by producer count

Compression	TLS mean	TLS med.	mTLS mean	mTLS med.	Higher
lz4	9.99%	2.45%	8.59%	0.06%	TLS
none	11.68%	4.99%	10.55%	1.46%	TLS

Table D.8: Matched producer throughput penalty by compression mode

Acks	TLS mean	TLS med.	mTLS mean	mTLS med.	Higher
acks=1	11.00%	3.77%	9.71%	0.08%	TLS
acks=all	10.66%	3.88%	9.42%	0.17%	TLS

Table D.9: Matched producer throughput penalty by acknowledgement mode

Record size	TLS mean	TLS med.	mTLS mean	mTLS med.	Higher
1024	804.73%	496.57%	849.26%	579.13%	mTLS
10240	1,213.29%	296.45%	1,208.25%	273.52%	TLS
102400	838.68%	744.71%	815.94%	722.30%	TLS

Table D.10: Matched producer average-latency penalty by record size

Producer count	TLS mean	TLS med.	mTLS mean	mTLS med.	Higher
1	433.89%	181.22%	429.47%	174.85%	TLS
6	1,176.97%	688.45%	1,172.99%	698.63%	TLS
12	1,245.84%	1,026.96%	1,270.98%	1,008.94%	mTLS

Table D.11: Matched producer average-latency penalty by producer count

Batch size	TLS mean	TLS med.	mTLS mean	mTLS med.	Higher
16384	479.63%	264.85%	484.61%	247.57%	mTLS
131072	1,424.83%	994.13%	1,431.02%	993.14%	mTLS

Table D.12: Matched producer average-latency penalty by batch size

Broker count	TLS mean	TLS med.	mTLS mean	mTLS med.	Higher
3	1,120.44%	485.35%	1,256.59%	636.67%	mTLS
5	896.16%	543.39%	858.22%	560.70%	TLS

Table D.13: Matched producer average-latency penalty by broker count

D.4 Host Telemetry Tables

Table D.14 reports the main producer host-telemetry means. The broker values are per-run broker means, while network counters are converted from bytes using decimal MB or GB.

Telemetry metric	Plaintext	TLS	mTLS
Benchmark-client CPU %	76.96	77.40	75.76
Broker CPU %	28.71	24.96	22.35
Benchmark-client memory %	17.86	19.87	19.02
Broker memory %	23.41	23.31	23.22
Benchmark-client network RX MB	18.04	21.99	22.35
Benchmark-client network TX GB	3.84	3.86	3.87
Broker network RX GB	3.44	3.47	3.47
Broker network TX GB	2.56	2.58	2.58
Benchmark-client disk read sectors	106	126	147
Benchmark-client disk write sectors	44,262	44,478	49,353
Broker disk read sectors	8,109	7,239	6,510
Broker disk write sectors	6,421,375	6,422,382	6,463,730

Table D.14: Matched producer host-telemetry summary

D.5 Consumer Validation Tables

The consumer campaign is smaller than the producer campaign and is therefore treated as supporting validation. Table D.15 reports the main consumer means and penalties.

Metric	Plain.	TLS	mTLS	TLS pen.	mTLS pen.
Throughput records/s	28,734.78	14,819.16	15,488.13	38.70%	31.81%
Throughput MB/s	201.74	138.85	160.30	38.70%	31.81%
Fetch time ms	14,237.75	20,292.42	17,472.92	79.58%	69.10%
Benchmark-client CPU %	50.21	48.04	40.02	-4.46%	-25.19%
Broker CPU %	25.40	22.34	15.14	-15.76%	-46.34%

Table D.15: Targeted consumer validation summary

D.6 Highest Observed Producer Overhead Cases

Table D.16 records the highest observed producer penalties in the matched comparison set. These values are extreme cases rather than typical behaviour.

Metric	Highest TLS	Highest mTLS
Throughput records/s	51.35%	53.98%
Average latency ms	6,954.15%	8,223.00%
Maximum latency ms	189.76%	249.77%
Benchmark-client CPU %	4,603.50%	4,714.11%
Broker CPU %	66.45%	63.66%

Table D.16: Highest observed producer overhead cases by metric

D.7 Selected Supporting Figures

The figures below are PNG exports from the generated result pack. They are included as supporting evidence for the main Chapter 4 figures and tables, rather than as additional primary claims.

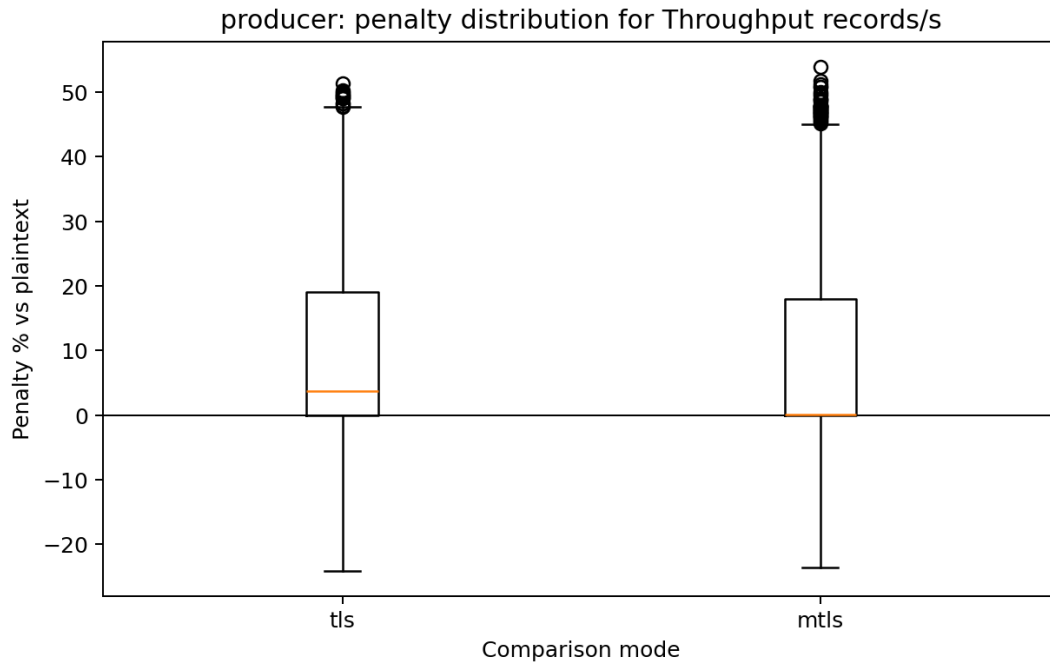


Figure D.1: Additional producer throughput-overhead distribution. Positive values mean lower throughput than the matched plaintext run; negative values mean higher throughput.

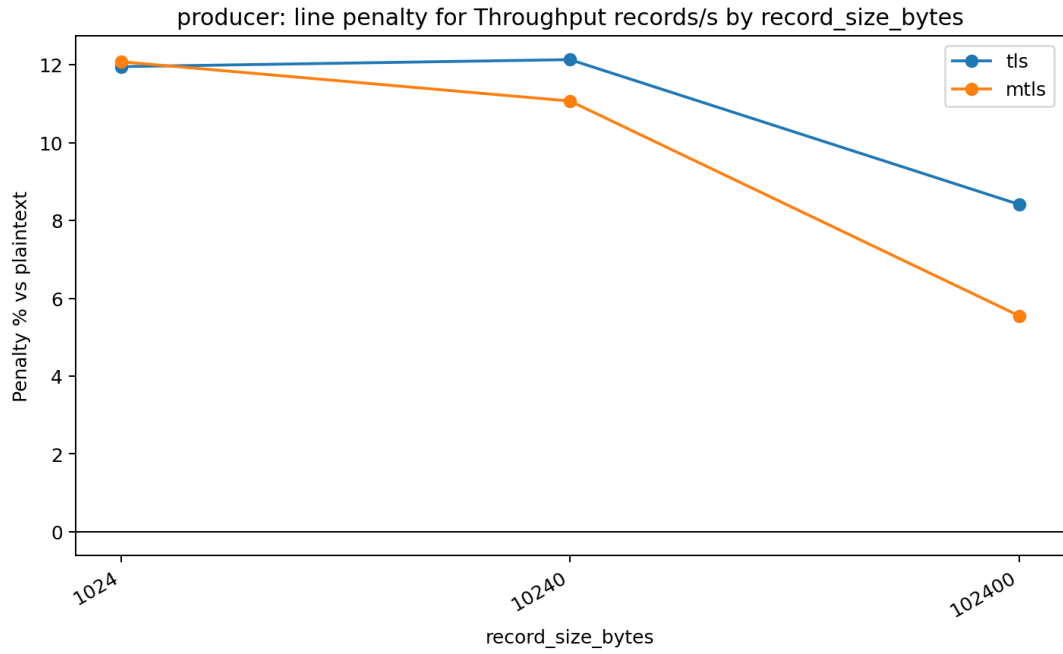


Figure D.2: Additional producer throughput-overhead view by record size, showing how the mean penalty changes across payload sizes.

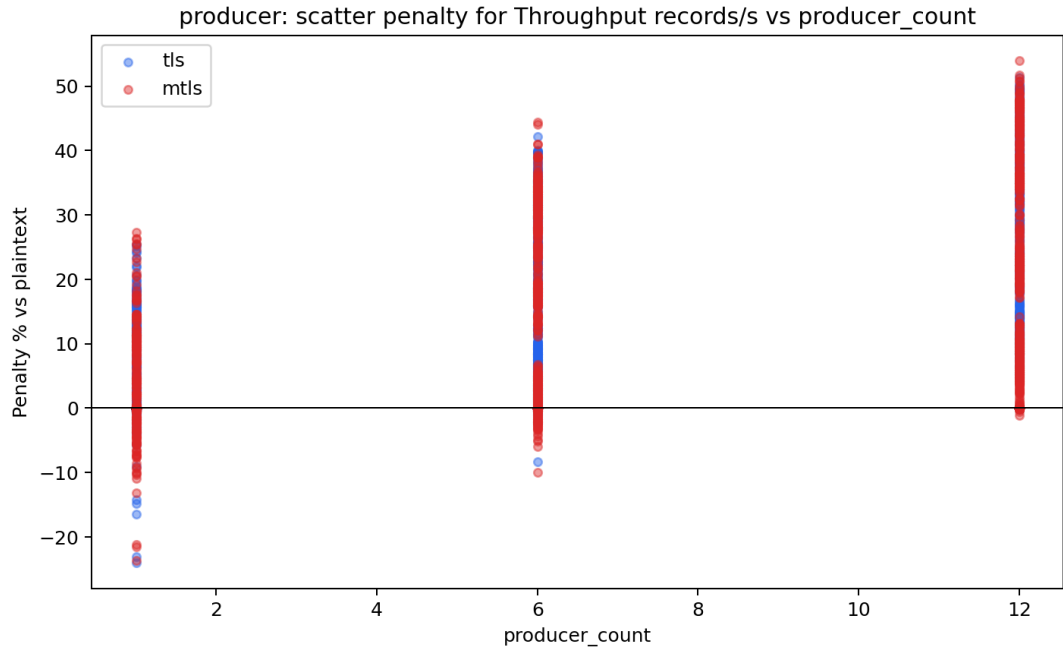


Figure D.3: Additional producer throughput-overhead scatter plot by producer count. This supports the workload-interaction result that throughput penalty spreads more widely as producer concurrency increases.

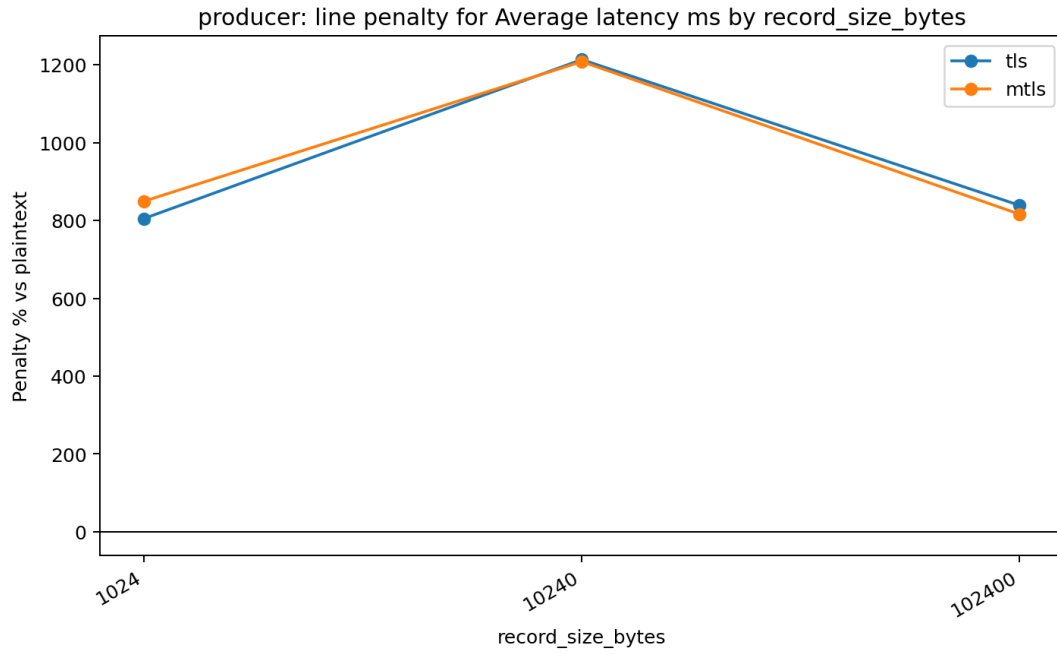


Figure D.4: Additional producer average-latency overhead view by record size, showing that latency overhead is not uniform across payload sizes.

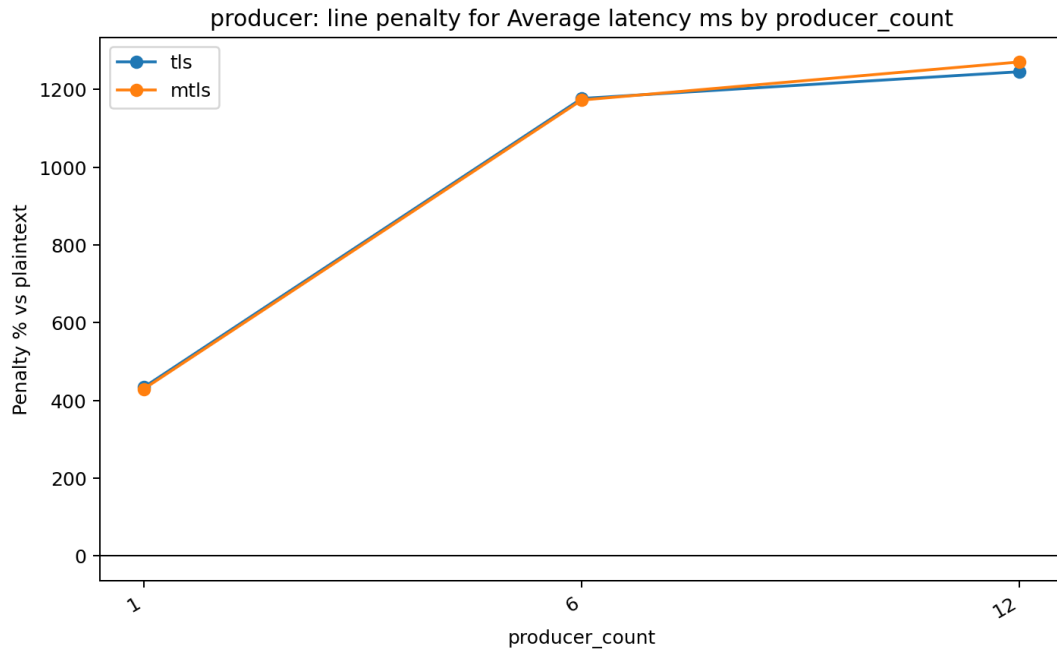


Figure D.5: Additional producer average-latency overhead view by producer count. This supports the latency result that secure-transport overhead becomes more visible under higher producer concurrency.

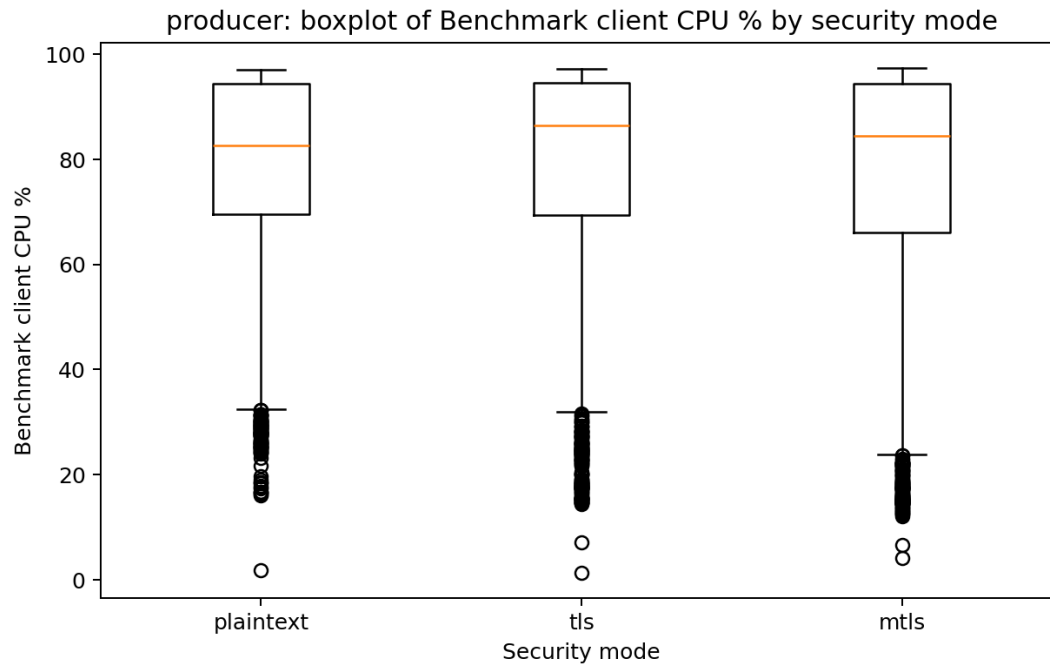


Figure D.6: Additional benchmark-client CPU distribution by security mode for producer runs.

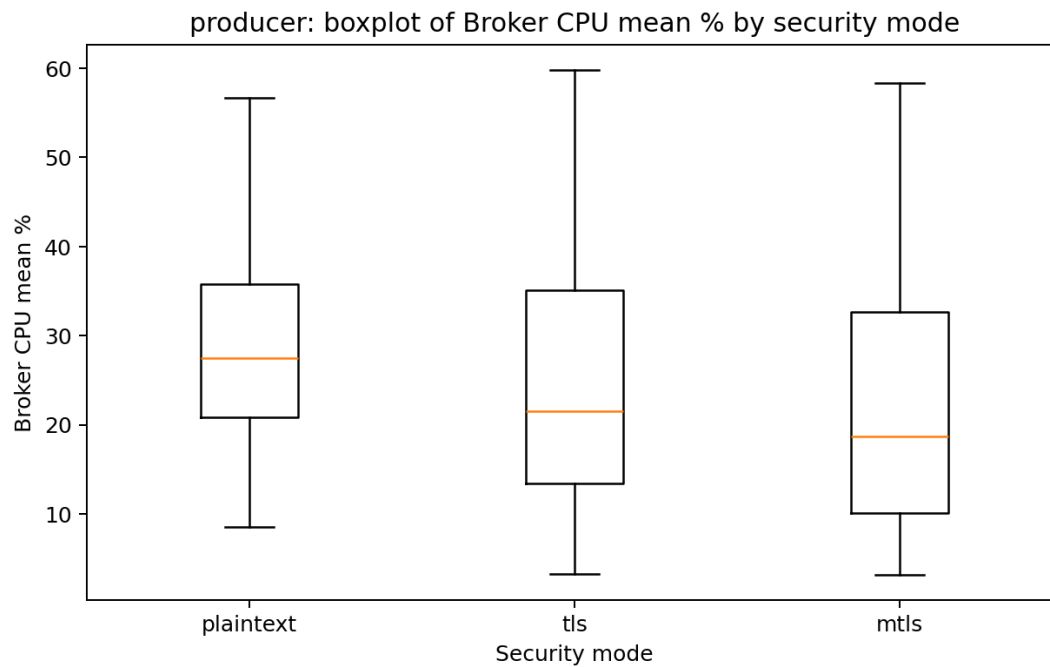


Figure D.7: Additional broker CPU distribution by security mode for producer runs.

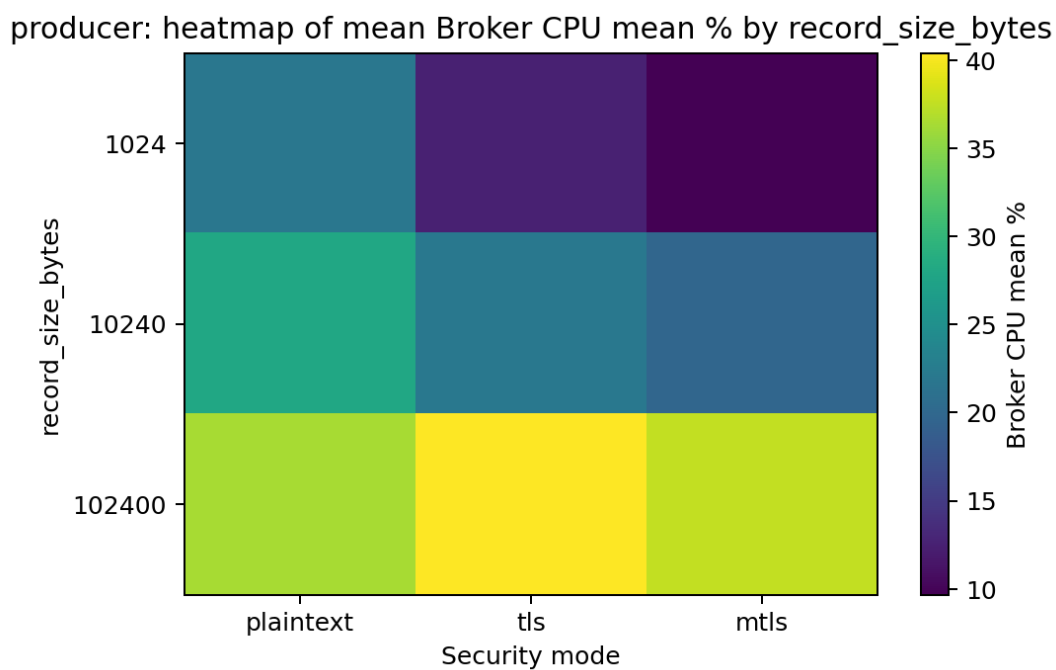


Figure D.8: Additional broker CPU heatmap by record size and security mode for producer runs. This supports the host telemetry finding that broker CPU does not simply increase under TLS and mTLS.

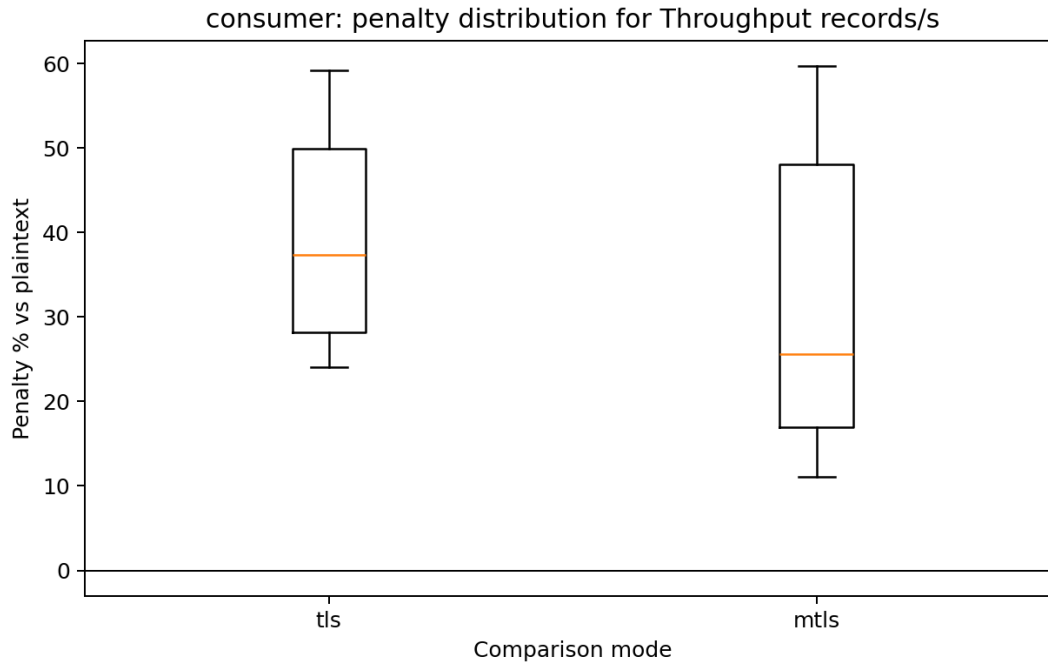


Figure D.9: Additional consumer throughput-overhead distribution for the targeted validation slice. Positive values mean lower throughput than the matched plaintext run.

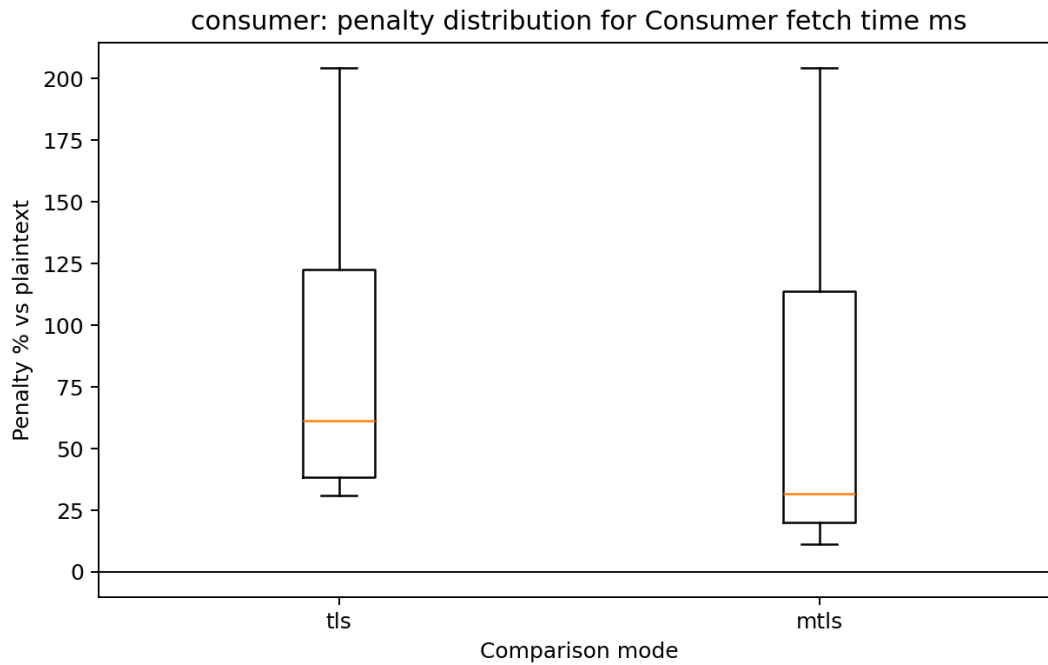


Figure D.10: Additional consumer fetch-time overhead distribution for the targeted validation slice.

The following disk and network figures provide additional support for the host telemetry discussion. They should be read as supporting evidence for resource behaviour, not as separate primary findings.

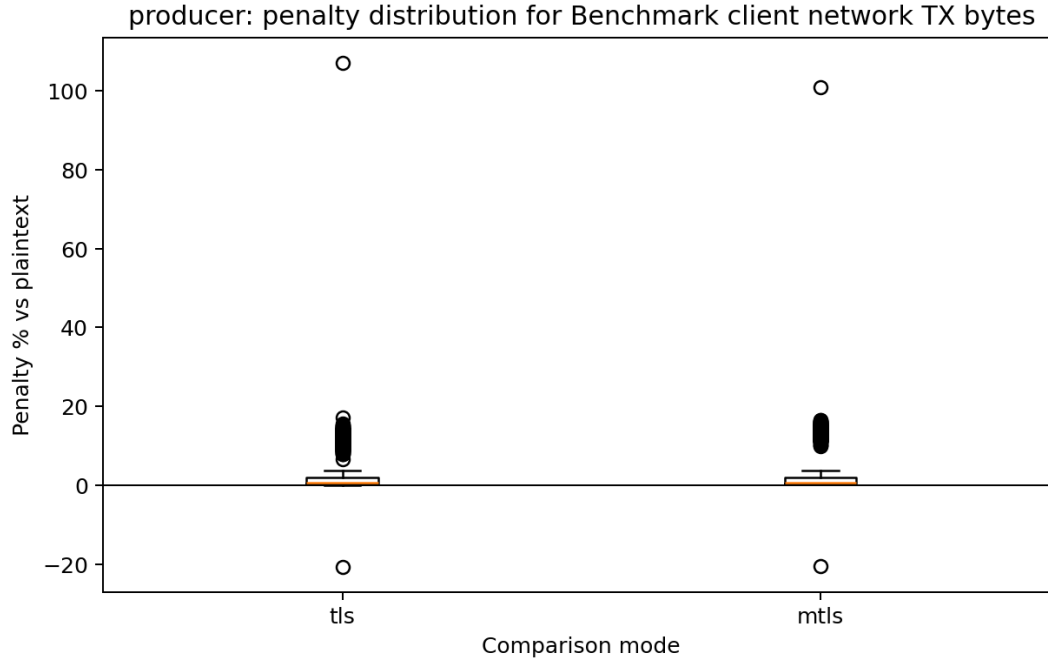


Figure D.11: Additional benchmark-client network transmit overhead distribution for producer runs. Positive values mean higher transmit volume than the matched plaintext run.

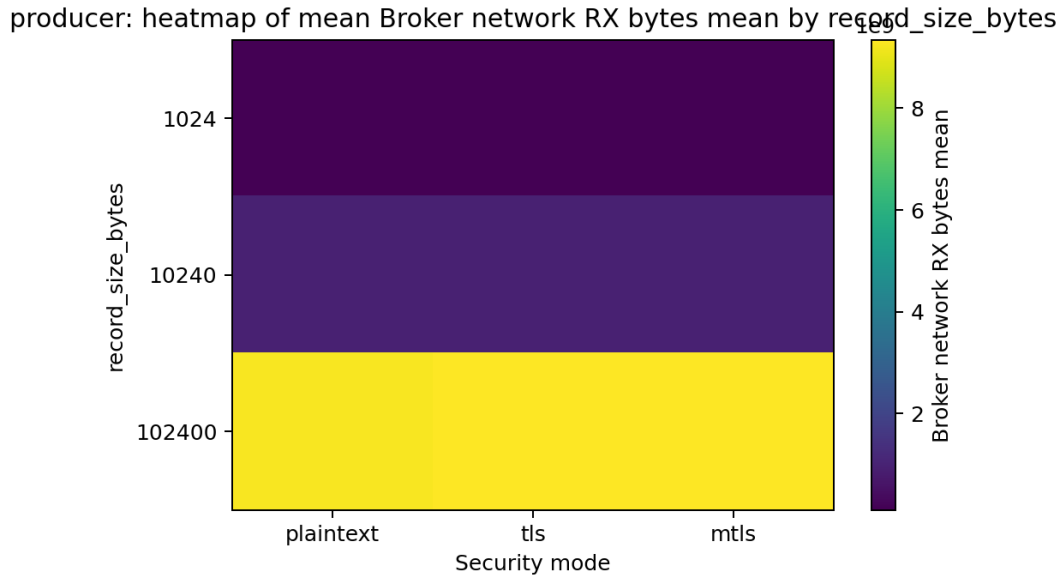


Figure D.12: Additional broker network receive heatmap by record size and security mode for producer runs. This supports the comparison of network behaviour across matched security modes.

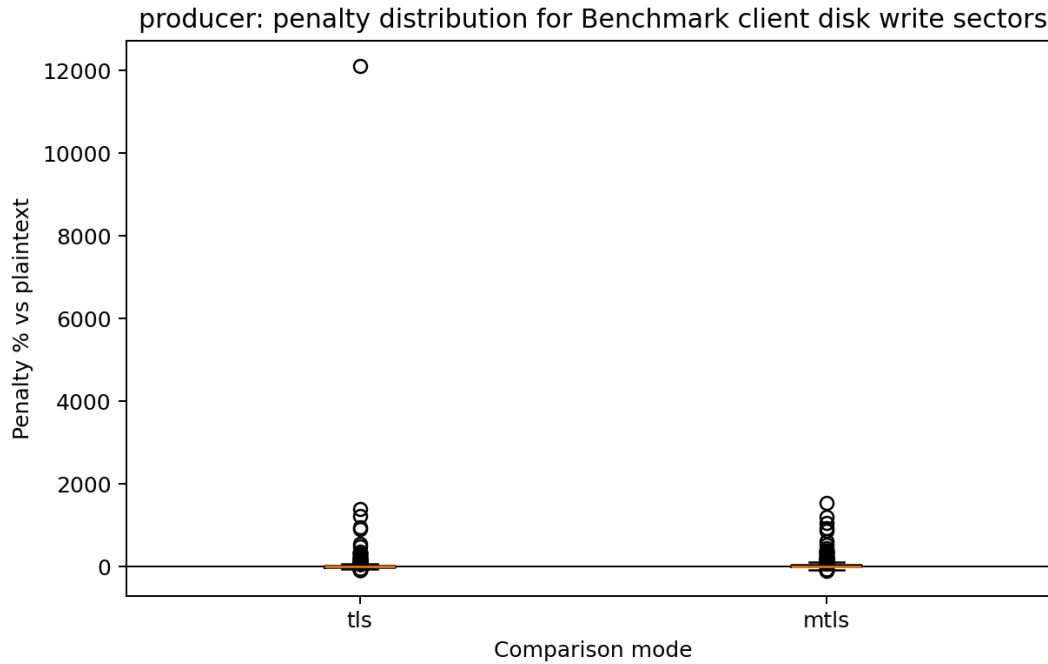


Figure D.13: Additional benchmark-client disk-write overhead distribution for producer runs. Positive values mean more write-sector activity than the matched plaintext run.

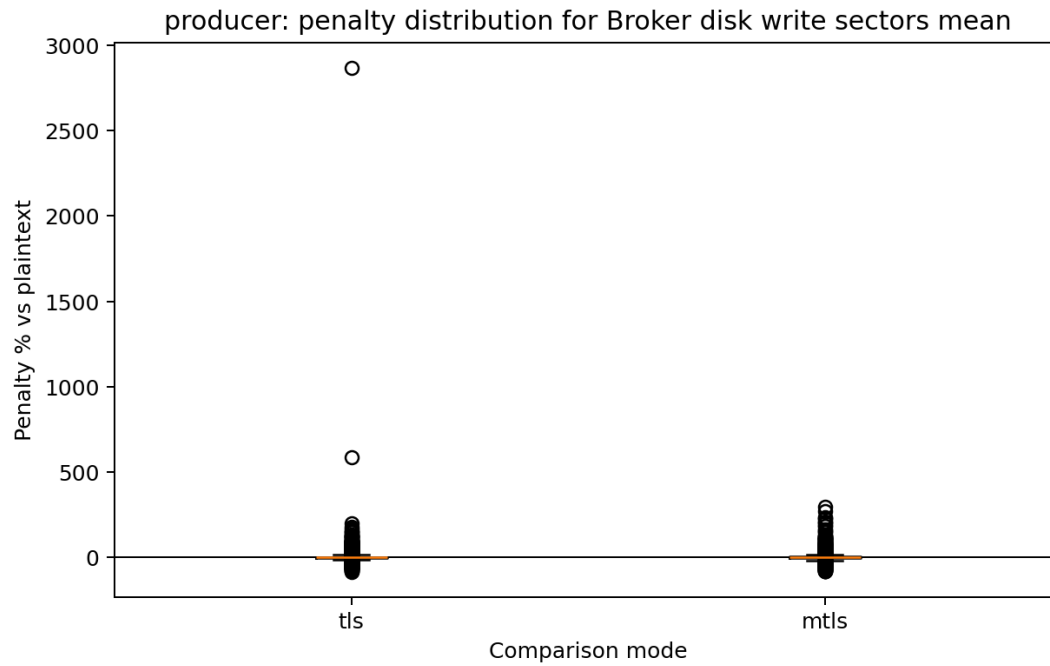


Figure D.14: Additional broker disk-write overhead distribution for producer runs. This supports the host telemetry comparison without treating disk activity as the main source of transport-security overhead.

Mode	Metric	n	Mean penalty	Mean 95% CI	Median penalty	Median 95% CI
tls	Throughput records/s	1728	10.833%	[10.168, 11.493]	3.774%	[1.240, 6.112]
tls	Average latency ms	1728	952.233%	[901.366, 1004.036]	530.304%	[487.917, 571.041]
tls	Maximum latency ms	1728	79.301%	[77.888, 80.690]	79.362%	[77.720, 81.056]
tls	Benchmark client CPU %	1727	2.494%	[-0.797, 8.418]	0.606%	[0.478, 0.814]
tls	Broker CPU mean %	1725	-17.062%	[-18.401, -15.750]	-21.059%	[-23.011, -19.065]
mtls	Throughput records/s	1728	9.566%	[8.901, 10.232]	0.119%	[0.056, 0.858]
mtls	Average latency ms	1728	957.814%	[905.554, 1010.044]	576.306%	[540.583, 619.176]
mtls	Maximum latency ms	1728	91.797%	[90.085, 93.447]	88.032%	[87.127, 89.539]
mtls	Benchmark client CPU %	1727	-0.547%	[-3.938, 5.315]	-0.075%	[-0.200, 0.068]
mtls	Broker CPU mean %	1726	-27.417%	[-28.873, -26.036]	-31.872%	[-34.326, -28.299]

Table D.17: Bootstrap confidence intervals for key producer penalties

Mode	Metric	n	Worse rows	Mean penalty	Effect dz	Sign-test p
tls	Throughput records/s	1728	74.306%	10.833%	0.777	<0.001
tls	Average latency ms	1728	99.421%	952.233%	0.857	<0.001
tls	Maximum latency ms	1728	98.148%	79.301%	2.648	<0.001
tls	Benchmark client CPU %	1727	59.641%	2.494%	0.022	<0.001
tls	Broker CPU mean %	1725	26.551%	-17.062%	-0.611	<0.001
mtls	Throughput records/s	1728	67.708%	9.566%	0.667	<0.001
mtls	Average latency ms	1728	99.306%	957.814%	0.858	<0.001
mtls	Maximum latency ms	1728	98.553%	91.797%	2.574	<0.001
mtls	Benchmark client CPU %	1727	48.929%	-0.547%	-0.005	0.386
mtls	Broker CPU mean %	1726	18.192%	-27.417%	-0.893	<0.001

Table D.18: Matched-pair effect summary for key producer penalties

Mode	Metric	n	Mean penalty	Mean 95% CI	Median penalty	Median 95% CI
tls	Throughput records/s	24	38.704%	[34.176, 43.403]	37.363%	[28.646, 45.526]
tls	Consumer fetch time ms	24	79.582%	[60.222, 101.119]	61.313%	[39.203, 106.154]
tls	Benchmark client CPU %	24	-4.464%	[-11.407, 0.522]	-2.549%	[-4.874, -1.496]
tls	Broker CPU mean %	24	-15.760%	[-23.302, -8.750]	-23.195%	[-27.689, -1.292]
mtls	Throughput records/s	24	31.813%	[25.370, 38.463]	25.551%	[17.466, 46.275]
mtls	Consumer fetch time ms	24	69.100%	[46.040, 93.697]	31.545%	[20.172, 109.393]
mtls	Benchmark client CPU %	24	-25.187%	[-29.776, -20.687]	-25.221%	[-35.814, -14.458]
mtls	Broker CPU mean %	24	-46.344%	[-55.019, -37.638]	-45.616%	[-66.049, -26.066]

Table D.19: Bootstrap confidence intervals for key consumer penalties

Mode	Metric		n	Worse rows	Mean penalty	Effect dz	Sign-test p
tls	Throughput records/s		24	100.000%	38.704%	3.228	<0.001
tls	Consumer	fetch	24	100.000%	79.582%	1.504	<0.001
tls	Benchmark	client	24	20.833%	-4.464%	-0.288	0.008
tls	Broker	CPU mean	24	16.667%	-15.760%	-0.842	0.002
mtls	Throughput records/s		24	100.000%	31.813%	1.886	<0.001
mtls	Consumer	fetch	24	100.000%	69.100%	1.128	<0.001
mtls	Benchmark	client	24	0.000%	-25.187%	-2.129	<0.001
mtls	Broker	CPU mean	24	0.000%	-46.344%	-2.118	<0.001

Table D.20: Matched-pair effect summary for key consumer penalties

Dataset	Mode	Metric	Factor	Lowest level	Highest level	Spread
consumer	mtls	Consumer fetch time ms	record_size_bytes	102400	1024	100.708%
consumer	mtls	Consumer fetch time ms	consumer_count	6	1	18.608%
consumer	mtls	Throughput records/s	record_size_bytes	102400	1024	30.894%
consumer	mtls	Throughput records/s	consumer_count	6	1	3.929%
consumer	tls	Consumer fetch time ms	record_size_bytes	102400	1024	80.759%
consumer	tls	Consumer fetch time ms	trial_index	1	3	33.883%
consumer	tls	Throughput records/s	record_size_bytes	102400	1024	20.455%
consumer	tls	Throughput records/s	trial_index	1	3	3.849%
producer	mtls	Average latency ms	batch_size	16384	131072	946.412%
producer	mtls	Average latency ms	producer_count	1	12	841.512%
producer	mtls	Throughput records/s	throughput_limit_records_per_sec	1000	10000	15.484%
producer	mtls	Throughput records/s	producer_count	1	12	14.500%
producer	tls	Average latency ms	batch_size	16384	131072	945.197%
producer	tls	Average latency ms	producer_count	1	12	811.945%
producer	tls	Throughput records/s	throughput_limit_records_per_sec	1000	10000	16.676%
producer	tls	Throughput records/s	producer_count	1	12	14.316%

Table D.21: Top factor-sensitivity results for selected producer and consumer penalties

Overall, Appendix D provides the extended numerical and visual evidence behind the main Chapter 4 discussion. The interpretation remains in Chapter 4, while these tables and figures provide the supporting detail needed for transparency and completeness.

Appendix E

Repository and Reproducibility Notes

The repository for the artefact is available at github.com/VictoriousWealth/apache-kafka-artefact. Where possible, the submitted version should be linked to the final commit or release used to generate the dissertation results.

This appendix summarises how the artefact repository is organised and how reproducibility is supported. The repository separates infrastructure provisioning, Kafka deployment, benchmark execution, telemetry collection, result parsing, and export. This was done so that the artefact can be inspected as a set of clear components rather than as one large script.

The artefact is also designed around explicit configuration files and generated execution plans. This means that benchmark rows are not created manually during execution. Instead, the configuration files define the experiment space, the plan generator expands this into JSONL rows, and the executor runs those rows while recording started, completed, and failed runs.

E.1 Repository Structure

The main repository directories are shown in Table E.1. Source files, configuration files, orchestration scripts, and documentation are part of the artefact. Generated execution state is stored separately under `.orchestration/`, because it depends on the current AWS deployment and local execution environment.

Repository path	Purpose
<code>infrastructure/terraform/</code>	Contains Terraform configuration for AWS networking, security groups, Kafka broker instances, and the benchmark client.
<code>deploy/kafka/</code>	Contains Kafka installation scripts, broker configuration templates, client configuration, TLS/mTLS setup, and benchmark runner scripts.
<code>config/baselines/</code>	Contains baseline workload configurations used as fixed defaults.
<code>config/sweeps/</code>	Contains one-factor-at-a-time exploratory sweep definitions.
<code>config/factorials/</code>	Contains factorial experiment definitions for the final producer campaign and consumer validation slice.
<code>scripts/orchestration/</code>	Contains scripts for inventory generation, deployment, benchmark execution, parsing, aggregation, export, and campaign control.
<code>docs/</code>	Contains supporting artefact documentation, including architecture, methodology, result schema, reproducibility notes, and campaign operating notes.
<code>results/</code>	Contains local benchmark outputs, parsed results, summaries, and exported tables or plots.
<code>.orchestration/</code>	Contains generated local execution state, plans, checkpoints, Terraform outputs, inventory files, and deployment metadata.
<code>scripts/analysis/</code>	Contains post-processing scripts for matched comparison, bootstrap confidence intervals, matched-pair summaries, and factor-sensitivity exports.

Table E.1: Repository structure

E.2 Configuration-Driven Execution

Benchmark execution is driven by configuration files rather than manually constructed commands. Baseline and factorial JSON files define the workload parameters, while generated JSONL plans define the exact benchmark rows to execute. This makes the campaign design inspectable before execution and reduces the risk of changing parameters accidentally between security modes.

The final producer campaign is generated from `config/factorials/security-overhead-final.json`. The targeted consumer validation slice is generated from `config/factorials/consumer-security-slice.json`. The generated plans are stored under `.orchestration/`, including `.orchestration/security-overhead-final-plan.jsonl` and `.orchestration/consumer-security-slice-plan.jsonl`.

This structure supports reproducibility because each result can be traced back to a configuration file, generated plan row, deterministic run identifier, metadata file, result directory, and summary row. A benchmark result is therefore not only a value in a table, but an output linked to a specific workload and deployment configuration.

E.3 Deployment Reproducibility

AWS infrastructure is provisioned using Terraform. The Terraform layer defines the VPC, subnet, security groups, Kafka broker instances, and benchmark client instance. After provisioning, orchestration scripts export the Terraform outputs and convert them into local inventory and cluster metadata files.

Kafka deployment is automated through shell scripts. These scripts install Kafka, render broker configuration, configure KRaft metadata, start broker services, and validate readiness. Kafka data-plane traffic uses private VPC addresses, while SSH orchestration uses public addresses. This separation prevents Kafka benchmark traffic from using public broker addresses while still allowing the local workstation to manage the cluster.

Broker readiness is checked using Kafka API availability rather than only `systemd` process status. This is important because a broker process can be running before the broker is ready to serve Kafka requests. API-level readiness checks reduce the risk of starting benchmark rows against a partially available cluster.

E.4 Checkpointing and Resumability

The benchmark campaigns are long-running and can be interrupted by SSH timeouts, local machine sleep, network issues, or cloud-instance disruption. The executor therefore records campaign state using `started.jsonl`, `completed.jsonl`, `failures.jsonl` where applicable, and checkpoint files under `.orchestration/`.

A row is only treated as complete after the remote benchmark has finished, result files have been copied back locally, raw output has been parsed, and the checkpoint has been updated. This means that a row appearing in `started.jsonl` is not enough for it to be treated as complete. The completed file and checkpoint are the authoritative record of successful execution.

When a campaign resumes, the executor skips rows that are already recorded as complete. This prevents completed rows from being rerun and makes it practical to execute large campaigns across multiple sessions. It also protects the result set from partial local state, because result copying is completed before a row is marked as finished.

E.5 Result Traceability

Each benchmark run writes a dedicated result directory. The directory name is derived from the run identifier, which encodes the main workload configuration. This makes the directory self-describing and reduces ambiguity when inspecting individual runs.

The run directory contains raw Kafka benchmark logs, workload metadata, parsed metrics, and host telemetry. The raw logs are retained so that parsed values can be checked against the original benchmark output. The `metadata.json` file records workload context, while `result.json` stores the parsed structured result for the run.

At result-set level, `summary.json` and `summary.csv` aggregate completed runs into structured and tabular formats. These files are used for analysis, comparison, and dissertation-ready exports. This gives two levels of traceability: individual run inspection through run directories, and full result-set analysis through summary files.

E.6 Generated State and Version Control

Not every file produced by the artefact is suitable for version control. Generated state under `.orchestration/` may include environment-specific inventory files, Terraform outputs, deployment checkpoints, generated plans, SSH key paths, certificates, and cluster metadata. These files are useful for local execution but do not represent permanent source code.

Sensitive files such as SSH keys, AWS credentials, generated certificates, keystores, truststores, and environment-specific deployment outputs are excluded from version control where appropriate. This is necessary for security and also improves reproducibility, because those files describe one live deployment rather than the general artefact design.

Large raw result sets are also treated carefully. Historical result sets are retained locally for traceability, but final analysis should use the telemetry-enabled result sets from the final plaintext, TLS, and mTLS campaigns. Older non-telemetry outputs are useful for development history but should not be mixed with final evaluation data.

E.7 Analysis Export Reproducibility

The final analysis also uses derived comparison outputs under `results/final-processed/comprehensive-result-pack/`. These include matched producer and consumer comparison files, overhead-long CSV files, dissertation tables, figures, and statistical summaries.

The statistical summaries used in Chapter 4 are generated by

`scripts/analysis/export_statistical_analysis.py`. This script reads the matched comparison outputs and produces bootstrap confidence intervals, matched-pair effect summaries, and factor-sensitivity tables under `results/final-processed/comprehensive-result-pack/statistics/`.

This means the reported confidence intervals and factor-sensitivity results are not manually calculated values. They are reproducible derived outputs linked to the matched result files. The generated `statistics/manifest.json` records the exported statistical files, bootstrap sample count, and random seed used for the final analysis.

E.8 Rerunning the Artefact

To rerun the artefact, the infrastructure must first be provisioned through Terraform. Terraform outputs are then exported into local orchestration metadata. The benchmark client and Kafka brokers are prepared using the orchestration scripts, after which the required security mode is deployed.

For factorial campaigns, the relevant JSON configuration is converted into a JSONL execution plan. The executor then runs the plan using security-mode and broker-count filters where required. If execution is interrupted, the same command can be rerun with the same checkpoint file so that completed rows are skipped.

The exact command depends on the campaign phase and security mode. However, the reproducibility principle remains the same across all phases: the same configuration file, generated plan, result-set name, and checkpoint file should be used when resuming a campaign.

After aggregation, the statistical export stage can be rerun using `scripts/analysis/export_statistical_analysis.py`. The same matched comparison files and script settings should be used when regenerating the bootstrap intervals, matched-pair summaries, and factor-sensitivity tables reported in Chapter 4.

E.9 Reproducibility Limitations

The artefact improves reproducibility, but it does not remove all sources of experimental variation. AWS EC2 instances are shared cloud resources, and their performance can vary because of cloud scheduling, network conditions, and background host activity. The benchmark results should therefore be interpreted as controlled cloud measurements rather than deterministic hardware constants.

The generated execution state also depends on campaign history. For example, an

interrupted run may appear in `started.jsonl` without appearing in `completed.jsonl`. This is expected behaviour and is handled by the executor, but it means that completed rows and checkpoint files should be used when reporting final campaign progress.

Finally, the artefact evaluates selected Kafka transport-security configurations rather than every possible secure Kafka deployment. Plaintext, TLS, and mTLS provide a controlled basis for comparing transport-security overhead, but they do not represent every possible Zero Trust Architecture, certificate-management strategy, authorisation policy, or production Kafka deployment pattern.